高并发核心技术Redis

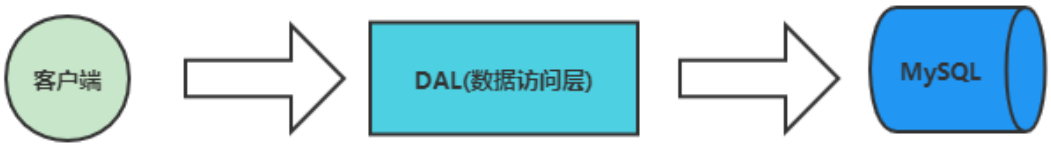
一、Redis概述

1 NoSQL介绍

1.1 NoSQL由来

任何技术的出现都是一步一步演进出来的。

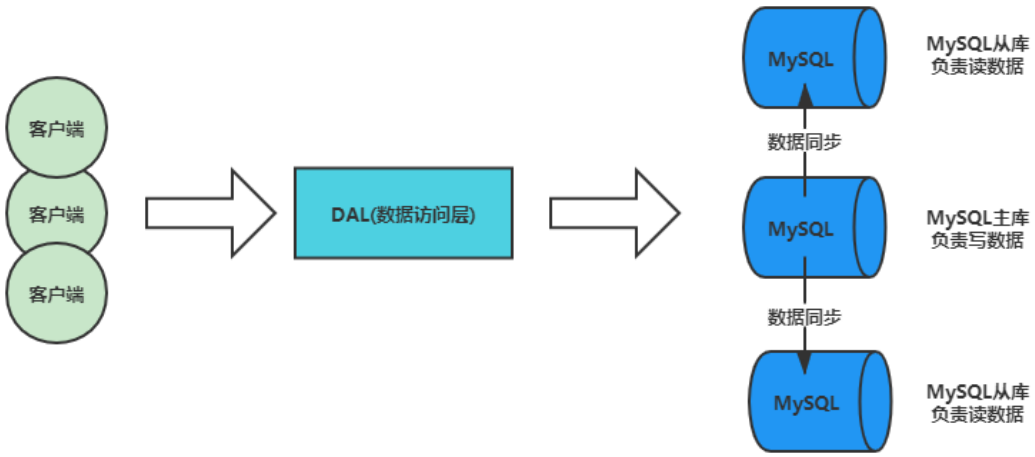
1. 在互联网诞生初期，一个网站一个应用访问量都不大， 使用单机Mysql数据库可以应对。



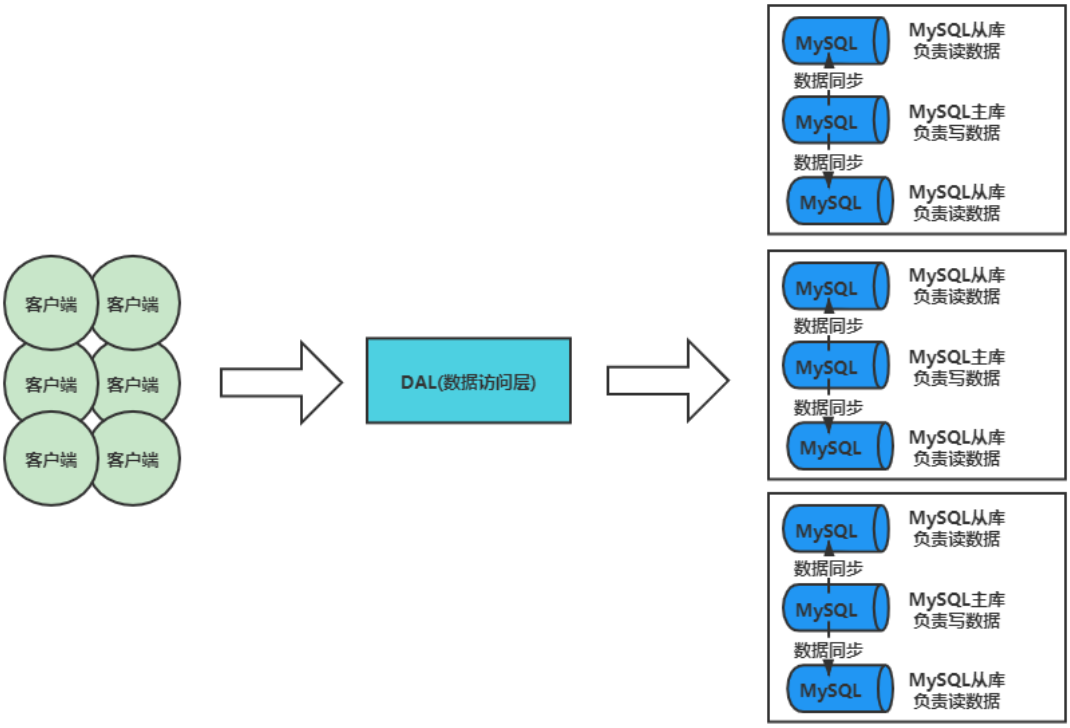
2. 随着用户的不断增多，单机MySQL可能无法放下，服务器也可能承受不住

   一个网站，大多数的情况都是查询，为了减少数据库的压力，引入读写分离策略，让主数据库处理事务性增、改、删操作，而从数据库处理SELECT查询操作。

   通过读写分离的方式，解决问题。

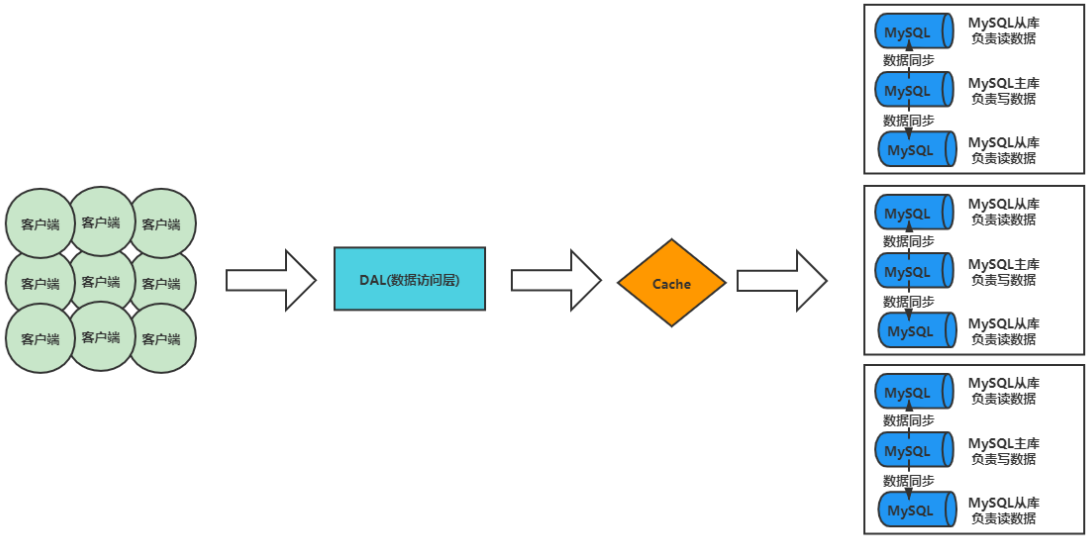


3. Mysql读的问题得到了缓解，写的压力依然存在，开始通过分库分表和MySQL集群来解决写的问题。



4. 当前，数据量越来越大、数据类型也越来越复杂，Mysql数据库在面对数据量很多、变化很大数据，如大文本字段、图片等，MySQL效率就低了，并且扩展性差，大数据量场景下IO压力大，结构更改困难。如果有一种数据库专门来存储此类数据，分担读取数据的压力，MySQL压力就会变得很小。

   NoSQL数据库就应运而生，可以很好的解决上述问题。



1.2 什么是NoSQL

> NoSQL = Not Only SQL 不仅仅是SQL

泛指非关系型数据库，对应对比关系型数据库，NoSQL没有sql语句使用，不需要设计表结构，没有表关系。

很多数据类型如：用户的个人信息，社交网络，地理位置，这些数据类型的存储不需要一个固定的模式，类似于Map<String,Object>使用键值对来控制。

1.3 NoSQL的特点

1. 方便扩展（数据之间没有关系，很好扩展）
2. 大数据量高性能（Redis一秒写8万次，读取11万次，NoSQL缓存记录级，是一种细粒度的缓存，性能比较高）
3. 数据类型多样性（不需要事先设计数据库，如果是数据量特别大的表，设计繁琐）
4. 传统RDBMS 与 NoSQL

```
RDBMS：
  - 结构化数据
  - 结构化查询语句SQL
  - 数据和关系都存储在单独的表中
  - 数据操纵语言,数据定义语言
  - 严格的一致性
  - 基础事务
NoSQL：
  - 没有固定的查询语言
  - 键值对存储、列存储、文档存储、图关系数据库
  - 最终一致性
  - CAP定理和BASE理论
  - 高性能，高可用，高可扩
```

1.4 NoSQL的四大分类

分类	常见数据库	典型应用场景	优点	缺点
键值对（key-value）	Tokyo Cabinet/Tyrant, **Redis**, Voldemort, Oracle BDB	内容缓存，主要用于处理大量数据的高访问负载，也用于一些日志系统等等。	查找速度快	数据无结构化，通常只被当作字符串或者二进制数据
文档型数据库	CouchDB, **MongoDB**	Web应用（与Key-Value类似，Value是结构化的，不同的是数据库能够了解Value的内容）	数据结构要求不严格，表结构可变，不需要像关系型数据库一样需要预先定义表结构	功能相对局限
列存储数据库	Cassandra, **HBase**, Riak	分布式的文件系统	查找速度快，可扩展性强，更容易进行分布式扩展	查询性能不高，而且缺乏统一的查询语法。
图形数据库	Neo4J, InfoGrid, Infinite Graph	社交网络，推荐系统等。专注于构建关系图谱	利用图结构相关算法。比如最短路径寻址，N度关系查找等	很多时候需要对整个图做计算才能得出需要的信息，而且这种结构不太好做分布式的集群方案。

2 Redis概述

2.1 Redis是什么

1. Redis（Remote Dictionary Server 远程字段服务）是一个开源的使用ANSI C语言编写、支持网络、内存亦可持久化的key-value数据库，并提供多种语言的API。
2. Redis是一个**key-value**存储系统，它支持存储的value类型相对更多，包括**string、list、set、zset**（sorted set --有序集合）和**hash**。这些数据结构都支持push/pop、add/remove及取交集并集和差集及更丰富的操作，而且这些操作都是原子性的。在此基础上，Redis支持各种不同方式的排序。为了保证效率，数据都是缓存在内存中，Redis会周期性的把更新的数据写入磁盘或者把修改操作写入追加的记录文件，并且在此基础上实现了master-slave（主从）同步。
3. Redis提供了java、C/C++、PHP、JavaScript、Perl、Object-C、Python、Ruby、Erlang等客户端，使用很方便。

2.2 Redis能干嘛

1. 读写效率高，用于高速缓存
2. 发布，订阅消息（消息通知）
3. 地图信息分析
4. 活动排行榜或计数
5. .....

2.3 Redis特点

1. 多样的数据类型：Redis不仅仅支持简单的key-value类型的数据，同时还提供list，set，zset，hash等数据结构的存储。
2. Redis支持数据的持久化，可以将内存中的数据保存在磁盘中，重启的时候可以再次加载进行使用。
3. Redis的所有操作都是原子性的。
4. 支持主从复制及集群。

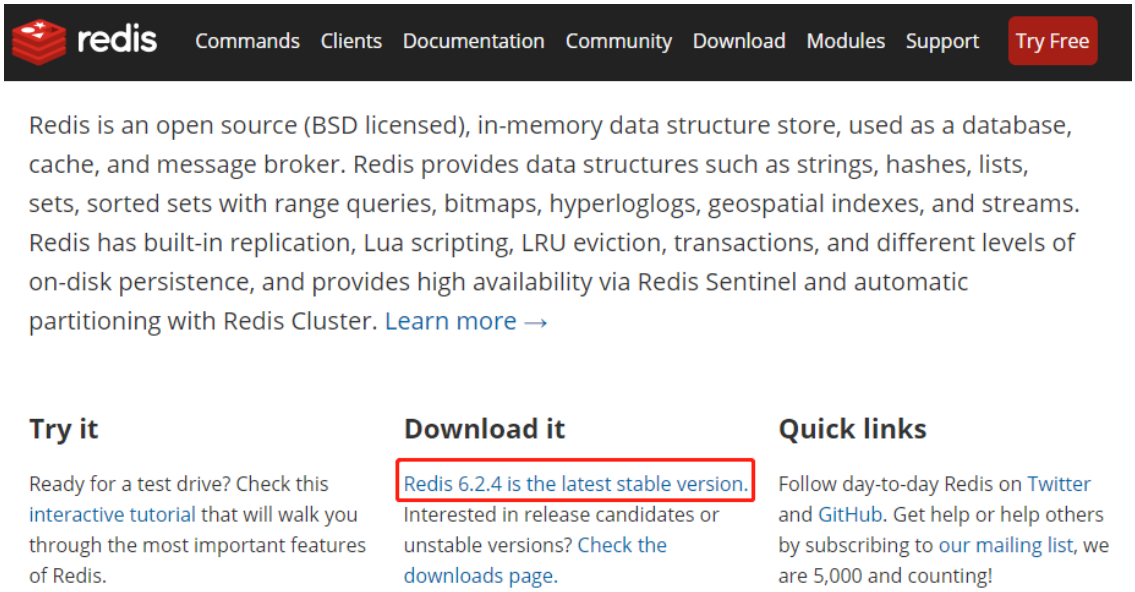
3 Redis安装

3.1 下载地址

Redis官方网址：https://redis.io/

3.2 下载Redis

1. 下载6.2.4 for Linux



2. redis-6.2.4.tar.gz上传至CentOS并解压，解压后得到redis-6.2.4目录

   解压命令：

```
tar -zxvf redis-6.2.4.tar.gz
```

3.3 安装GCC

安装C语言编译环境

```
dnf group install "Development Tools"
```

通过使用`gcc --version`命令打印 GCC 版本，来验证 GCC 编译器是否被成功安装:

```
gcc --version
```

3.4 安装Redis

1. 编译Redis

   在redis-6.2.4目录下执行：

```
make
```

   出现下图代表编译成功

```
    CC gopher.o
    CC tracking.o
    CC connection.o
    CC tls.o
    CC sha256.o
    CC timeout.o
    CC setcpuaffinity.o
    CC monotonic.o
    CC mt19937-64.o
    LINK redis-server
    INSTALL redis-sentinel
    CC redis-cli.o
    CC cli_common.o
    LINK redis-cli
    CC redis-benchmark.o
    LINK redis-benchmark
    INSTALL redis-check-rdb
    INSTALL redis-check-aof

Hint: It's a good idea to run 'make test' ;)

make[1]: 离开目录“/download/redis-6.2.4/src”
```

2. 安装Redis

   在redis-6.2.4目录下执行：

```
make install
```

   出现下图代表安装成功

```
[root@localhost redis-6.2.4]# make install
cd src && make install
make[1]: 进入目录“/download/redis-6.2.4/src”
    CC Makefile.dep

Hint: It's a good idea to run 'make test' ;)

    INSTALL redis-server
    INSTALL redis-benchmark
    INSTALL redis-cli
make[1]: 离开目录“/download/redis-6.2.4/src”
```

3. 安装目录： **/usr/local/bin**

```
[root@localhost bin]# cd /usr/local/bin/
[root@localhost bin]# ls
redis-benchmark  redis-check-aof  redis-check-rdb  redis-cli  redis-sentinel  redis-server
[root@localhost bin]#
```

   redis-benchmark：Redis自带的基准性能测试工具

   redis-check-aof：对有问题的 AOF 文件进行修复，AOF和RDB文件后面会说明

   redis-check-rdb：对有问题的 RDB文件进行修复

   redis-sentinel：Redis集群使用

   redis-cli：客户端

   redis-server：服务器启动

4. 服务启动

   前台启动：/usr/local/bin下执行

```
./redis-server
```

```
[root@localhost bin]# ./redis-server
64830:C 28 Jun 2021 06:20:16.361 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
64830:C 28 Jun 2021 06:20:16.361 # Redis version=6.2.4, bits=64, commit=00000000, modified=0, pid=64830, ju
st started
64830:C 28 Jun 2021 06:20:16.361 # Warning: no config file specified, using the default config. In order to
 specify a config file use ./redis-server /path/to/redis.conf
64830:M 28 Jun 2021 06:20:16.361 * Increased maximum number of open files to 10032 (it was originally set t
o 1024).
64830:M 28 Jun 2021 06:20:16.361 * monotonic clock: POSIX clock_gettime
                _.-``__ ''-._
           _.-``    `.  `_.  ''-._           Redis 6.2.4 (00000000/0) 64 bit
       .-`` .-```.  ```\/    _.,_ ''-._
      (    '      ,       .-`  |`,    )     Running in standalone mode
      |`-._`-...-` __...-.``-._|'` _.-'|     Port: 6379
      |    `-._   `._    /     _.-'    |     PID: 64830
       `-._    `-._  `-./  _.-'    _.-'
      |`-._`-._    `-.__.-'    _.-'_.-'|
      |    `-._`-._        _.-'_.-'    |           https://redis.io
       `-._    `-._`-.__.-'_.-'    _.-'
      |`-._`-._    `-.__.-'    _.-'_.-'|
      |    `-._`-._        _.-'_.-'    |
       `-._    `-._`-.__.-'_.-'    _.-'
           `-._    `-.__.-'    _.-'
               `-._        _.-'
                   `-.__.-'

64830:M 28 Jun 2021 06:20:16.362 # WARNING: The TCP backlog setting of 511 cannot be enforced because /proc
/sys/net/core/somaxconn is set to the lower value of 128.
64830:M 28 Jun 2021 06:20:16.362 # Server initialized
64830:M 28 Jun 2021 06:20:16.362 # WARNING overcommit_memory is set to 0! Background save may fail under lo
w memory condition. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or
 run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
64830:M 28 Jun 2021 06:20:16.362 * Loading RDB produced by version 6.2.4
64830:M 28 Jun 2021 06:20:16.362 * RDB age 6 seconds
64830:M 28 Jun 2021 06:20:16.362 * RDB memory usage when created 0.77 Mb
64830:M 28 Jun 2021 06:20:16.362 * DB loaded from disk: 0.000 seconds
64830:M 28 Jun 2021 06:20:16.362 * Ready to accept connections
```

后台启动：

拷贝redis-6.2.4目录中的redis.conf文件到其他目录

```
mkdir /usr/local/myredis
cp redis.conf /usr/local/myredis/redis.conf
```

设置/usr/local/myredis/redis.conf文件中的**daemonize**属性，由no改为yes

在/usr/local/bin下执行

```
./redis-server /usr/local/myredis/redis.conf
```

通过ps aux | grep redis-server查看服务是否启动

```
[root@localhost bin]# ps aux | grep redis-server
root       65187  0.0  0.4  62184  3240 ?        Ssl  06:49   0:00 ./redis-server 127.0.0.1:6379
root       65241  0.0  0.1  12348  1120 pts/1    R+   06:51   0:00 grep --color=auto redis-server
[root@localhost bin]#
```

5. 客户端启动

/usr/local/bin下执行

```
./redis-cli
```

```
[root@localhost bin]# ./redis-cli
127.0.0.1:6379>
```

ping命令可以检测服务器是否正常（服务器返回PONG）

```
ping
```

```
[root@localhost bin]# ./redis-cli
127.0.0.1:6379> ping
PONG
```

4 Redis基本知识

1. 端口6379的由来

   6379 = Merz

   Merz全名Alessia Merz，是意大利的一位广告女郎。

2. 默认有16个数据库，且初始状态默认选择0号数据库(即第一个数据库)。
3. 可以使用select 进行数据库切换。

```
select 8 切换到8号数据库
```

```
127.0.0.1:6379> select 8
OK
127.0.0.1:6379[8]>
```

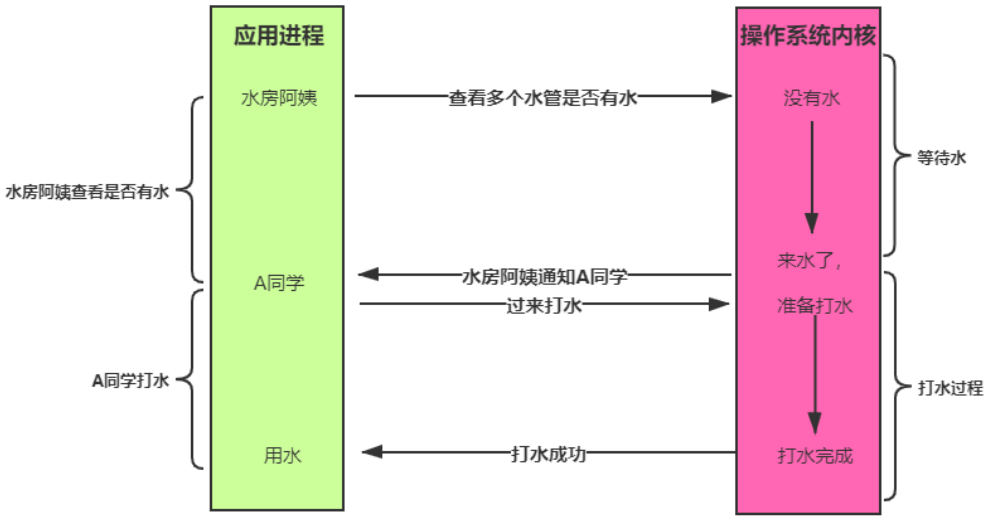
4. 统一密码管理，所有库密码都一致。
5. dbsize-查看当前数据库的key数量。
6. flushdb-清空当前库。

7. flushall-清空所有库。
8. 为什么Redis是单线程且效率极高：

   1）.绝大部分请求是纯粹的内存操作。

   2）.避免了不必要的上下文切换和竞争条件，也不存在多进程或者多线程导致的切换而消耗 CPU，不用去考虑。各种锁的问题，不存在加锁释放锁操作。

   3）.使用IO多路复用技术，可以处理并发的连接。



多路复用I/O模型

二、Redis数据类型

1 key键

1. **keys *** 查看当前库中所有的key 。
2. **exists key** 判断某个key是否存在。

   可以设置多个key，只返回存在的个数，但不返回哪一个存在/不存在。

```
exists k1  查看k1是否存在，如果存在返回1
exists k1 k2 k3 查看k1 k2 k3是否存在，如果k1 k2存在，k3不存在，则返回2
```

3. **move key db** 将当前数据库的 key 移动到给定的数据库 db 当中。

```
move k1 8 将k1从当前数据库移动到8号库
```

4. **type key** 查看当前key 所储存的值的类型。

   返回当前key所储存的值的类型，如string 、list等。
5. **del key** 删除已存在的key，不存在的 key 会被忽略。

   可以设置多个key，返回删除成功的个数。

```
del k1  删除k1，如果成功返回1，失败返回0
del k1 k2 k3 删除k1 k2 k3，如果k1 k2存在，k3不存在，则返回2
```

6. **expire key time** 给key设置time秒的过期时间。

设置成功返回 1 。 当 key 不存在返回 0。

```
expire k1 10 给k1设置10秒后过期
```

7. **ttl key** 以秒为单位返回 key 的剩余过期时间。

   当 key 不存在时，返回 -2 。 当 key 存在但没有设置剩余生存时间时，返回 -1 。 否则，以秒为单位，返回 key 的剩余生存时间。

8. **persist key** 移除给定 key 的过期时间，使得 key 永不过期。

   当过期时间移除成功时，返回 1 。 如果 key 不存在或 key 没有设置过期时间，返回 0 。

2 五大数据类型-String（字符串）

2.1 简介

String是Redis最基本的类型，一个key对应一个value。

String是二进制安全的，意味着String可以包含任何数据，比如序列化对象或者一张图片。

String最多可以放512M的数据。

2.2 常用命令

1. **set key value** 用于设置给定 key 的值。如果 key 已经存储其他值， set 就重写旧值，且无视类型。

```
set k1 v1    向Redis中设置一个k1的键值对
set k1 100   将k1的值由v1重置为100
```

2. **get key** 用于获取指定 key 的值。如果 key 不存在，返回 nil 。
3. **append key value** 将给定的value追加到key原值末尾。

   如果 key 已经存在并且是一个字符串， append 命令将 value 追加到 key 原来的值的末尾。

   如果 key 不存在， append 就简单地将给定 key 设为 value ，就像执行 set key value 一样。

```
append k1 v1 在Redis中不存在k1，所以直接设置k1的值为v1
append k1 v2 向k1的值末尾添加一个v2，最终结果为v1v2
```

4. **strlen key** 获取指定 key 所储存的字符串值的长度。当 key 储存的不是字符串值时，返回一个错误。
5. **setex key time value** 给指定的 key 设置值及time 秒的过期时间。如果 key 已经存在， setex命令将会替换旧的值，并设置过期时间。

```
setex k1 10 v1 向Redis中设置一个k1的键值对并且10秒后过期
```

6. **setnx key value**当key不存在时，设置给定 key 的值。如果key存在，则没有任何影响。

```
setnx k1 v1 向Redis中设置一个k1的键值对
setnx k1 v2 Redis中存在k1，则没有影响，k1的值仍然为v1
```

7. **incr key** 将 key 中储存的数字值增一。

   如果 key 不存在，那么 key 的值会先被初始化为 0 ，然后再执行 incr 操作。

如字符串类型的值不能表示为数字、或者是其他类型，那么返回一个错误。

```
incr k1 因为Redis中不存在k1，所以先初始化为0，再递增，值为1
incr k1 存在k1，递增后k1的值为2
set k2 v2
incr k2 因为k2不为数值，Redis返回一个错误
```

8. **decr key**将 key 中储存的数字值减一。

   如果 key 不存在，那么 key 的值会先被初始化为 0 ，然后再执行 decr 操作。

   如字符串类型的值不能表示为数字、或者是其他类型，那么返回一个错误。

```
decr k1 因为Redis中不存在k1，所以先初始化为0，再递增，值为-1
decr k1 存在k1，递增后k1的值为-2
set k2 v2
decr k2 因为k2不为数值，Redis返回一个错误
```

9. **incrby/decrby key step** 将key存储的数字值按照step进行增减。

   如果 key 不存在，那么 key 的值会先被初始化为 0 ，然后再执行 incrby/decrby 命令。

   如字符串类型的值不能表示为数字、或者是其他类型，那么返回一个错误。

10. **mset key1 value1 key2 value2 .....**同时设置一个或多个 key-value 。

```
mset k1 v1 k2 v2 k3 v3 同时向Redis中设置了k1 k2 k3
```

11. **mget key1 key2 .....**返回所有(一个或多个)给定 key 的值。

    如果给定的 key 里面，有某个 key 不存在，那么这个 key 返回特殊值 nil 。

```
mget k1 k2 同时获取k1 k2
```

12. **msetnx key1 value1 key2 value2 .....**用于所有给定 key 都不存在时，同时设置一个或多个 key-value 。

    msetnx具有原子性特性，有一个失败，则都失败。

```
msetnx k1 v1 k2 v2 向Redis中设置k1 k2两个键值对
msetnx k1 v2 k3 v2 Redis中存在k1，k1设置失败，由于原子性特性，k3也设置失败
```

13. **getrange key start end**用于获取存储在指定 key 中字符串的子字符串。字符串的截取范围由 start 和 end 两个偏移量决定(包括 start 和 end 在内)。

```
set java helloworld 设置一个key为java，value为helloworld的值
getrange java 0 3 获取索引0-3的值，结果为hell
```

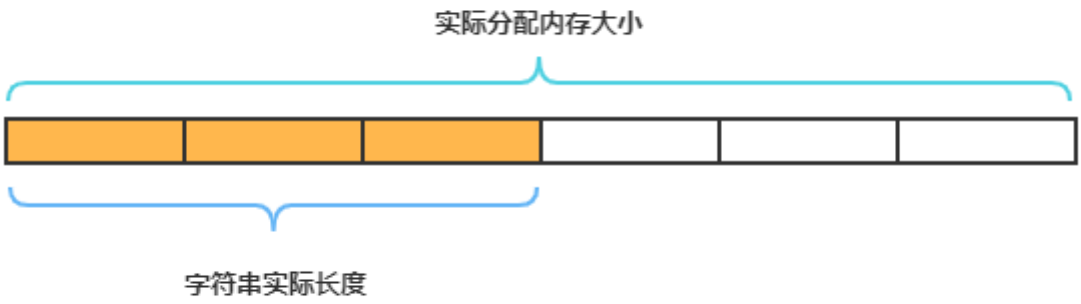
14. **setrange key offset value**用指定的字符串重写给定 key 所储存的字符串值，重写的位置从偏移量 offset 开始。

```
set java helloworld 设置一个key为java，value为helloworld的值
setrange java 5 baizhan 从偏移位置5（w）开始，用baizhan重写key
```

2.3 String底层数据结构

String底层数据结构是简单动态字符串（simple dynamic string,SDS）。

类似于JAVA中的ArrayList，采用预分配方式来减少内存的频繁分配。



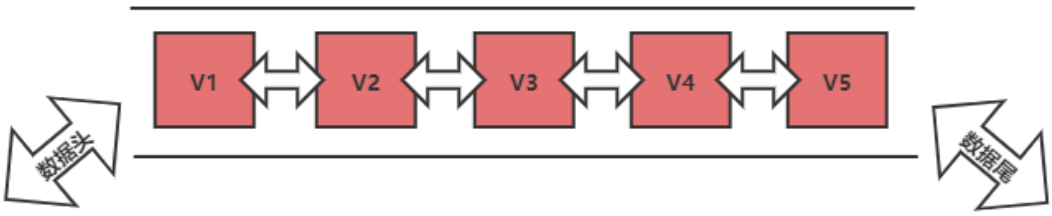
如图，内存实际大小一般都要高于字符串实际大小。当字符串长度小于1M时，扩容都是加倍现有的空间，如果超过1M，扩容时每次只会多扩1M的空间。字符串最大长度为512M。

3 五大数据类型-List（列表）

3.1 简介

List是简单的字符串列表，按照插入顺序排序。你可以添加一个元素到列表的头部（左边）或者尾部（右边）。

底层是一个双向链表，对两段操作性能极高，通过索引操作中间的节点性能较差。

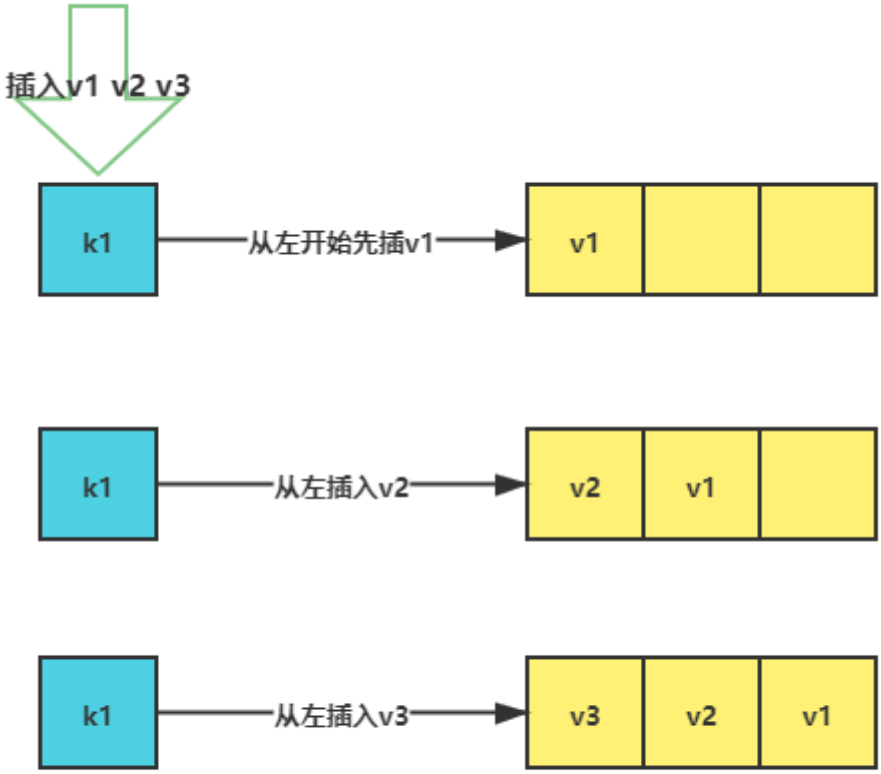


一个List最多可以包含 $2^{32}-1$个元素 （每个列表超过40亿个元素）。

3.2 常用命令

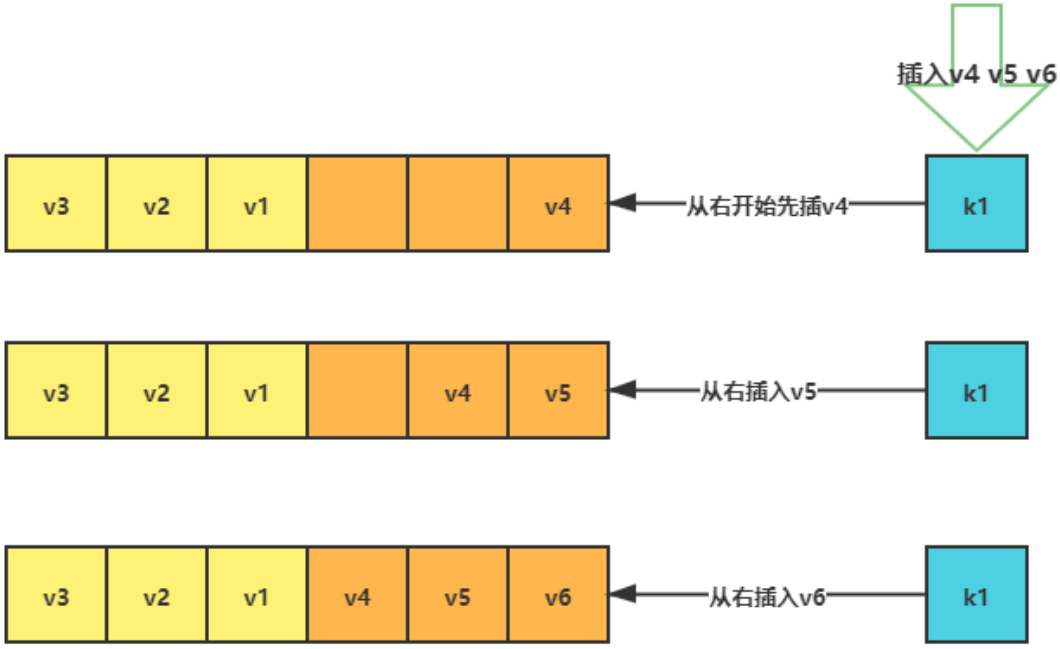
1. **lpush/ rpush key1 value1 value2 value3.....**从左边（头部）/右边（尾部）插入一个或多个值。

```
lpush k1 v1 v2 v3 从左边放入v1 v2 v3
```



lpush过程

```
rpush k1 v4 v5 v6 从右边放入v4 v5 v6
```



rpush过程

2. **lrange key start end**返回key列表中的start和end之间的元素（包含start和end）。 其中 0 表示列表的第一个元素，-1表示最后一个元素。

```
lrange k1 0 2 取出列表里前3个值，结果为v3 v2 v1
lrange k1 0 -1 取出列表里全部值，结果为v3 v2 v1 v4 v5 v6
```

3. **lpop/rpop key**移除并返回第一个值/最后一个值。

   值在键在，值光键亡。

```
lpop k1 从列表中删除v3，并返回，当前列表全部值v2 v1 v4 v5 v6
rpop k1 从列表中删除v6，并返回，当前列表全部值v2 v1 v4 v5
```

4. **lindex key index** 获取列表index位置的值（从左开始）。
5. **llen key**获取列表长度。
6. **lrem key count value**从左边开始删除与value相同的count个元素。

```
lrem k1 2 v1 从左边开始删除k1列表中2个v1元素
```

7. **linsert key before/after value newvalue** 在列表中value值的前边/后边插入一个newvalue值（从左开始）。

```
linsert k1 before v1 v5 在v1前面插入一个v5
```

8. **lset key index value**将索引为index的值设置为value

4 五大数据类型-Set（集合）

4.1 简介

与List类似是一个列表功能，但Set是自动排重的，当需要存储一个列表数据，又不希望出现重复数据时，Set是一个很好的选择。

Set是String类型的无序集合，它底层其实是一个value为null的hash表，所以添加、删除、查找的时间复杂度都是O(1)。

一般来说，一个算法如果是O(1)，随着数据增加，查找数据的时间不变。

集合中最大的成员数为$2^{32}-1$（每个集合超过40亿个元素）。

4.2 常用命令

1. **sadd key value1 value2.....**将一个或多个元素添加到集合key中，已经存在的元素将被忽略。

```
sadd k1 v1 v2 v2 v3 v4 v5 v6向集合中添加值，最终只有v1 v2 v3 v4 v5 v6
```

2. **smembers key**取出该集合的所有元素。

```
smembers k1
```

3. **sismember key value**判断集合key中是否含有value元素，如有返回1，否则返回0。

```
sismember k1 v1
```

4. **scard key**返回该集合的元素个数。

```
scard k1
```

5. **srem key value1 value2.....**删除集合中的一个或多个成员元素，不存在的成员元素会被忽略。

```
srem k1 v1 v2 删除v1 v2
```

6. **spop key**随机删除集合中一个元素并返回该元素。

```
spop k1 随机删除一个元素，并返回
```

7. **srandmember key count**随机取出集合中count个元素，但不会删除。

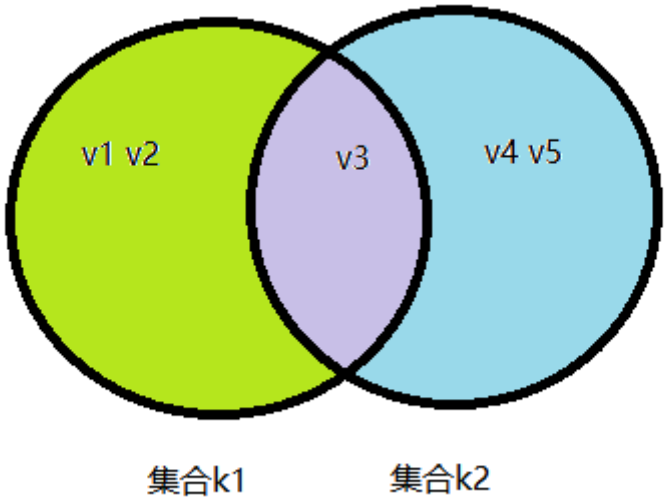
```
srandmember k1 2 随机取出集合中的2个元素
```

8. **smove sourcekey destinationkey value**将value元素从sourcekey集合移动到destinationkey集合中。

   如果 sourcekey集合不存在或不包含指定的 value元素，则 smove 命令不执行任何操作，仅返回 0 。

```
smove k1 k2 v5 将元素v5从集合k1中移动到集合k2
```

9. **sinter key1 key2**返回两个集合的交集元素。
10. **sunion key1 key2**返回两个集合的并集元素。
11. **sdiff key1 key2**返回两个集合的差集元素（key1中的，不包含key2）




```
sinter k1 k2 返回v3
sunion k1 k2 返回v1 v2 v3 v4 v5
sdiff k1 k2 返回v1 v2
sdiff k2 k1 返回v4 v5
```

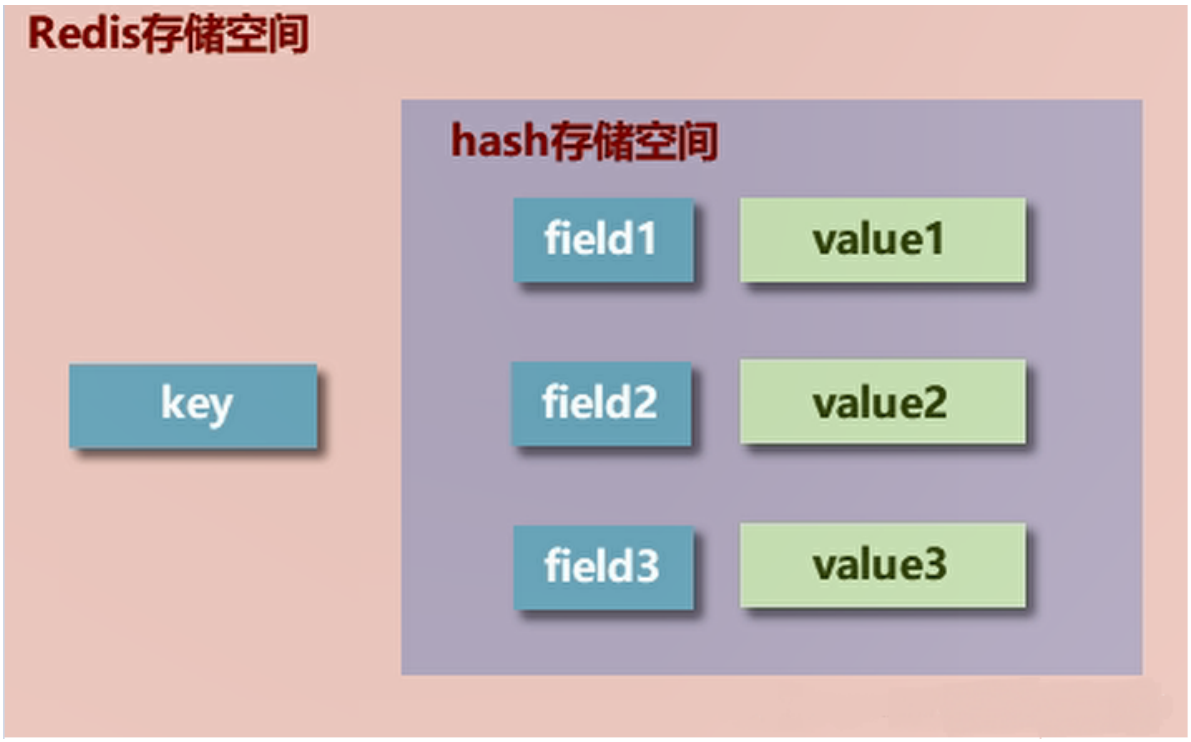
5 五大数据类型-Hash（哈希）

5.1 简介

Hash是一个键值对的集合。

Hash 是一个 String 类型的 field（字段） 和 value（值） 的映射表，hash 特别适合用于存储对象。

每个 Hash 可以存储 $2^{32}-1$键值对（40多亿）。



- Hash存储结构优化
  - 如果field数量较少，存储结构优化为类数组结构
  - 如果field数量较多，存储结构使用HashMap结构

5.2 常用命令

1. **hset key field value**给key集合中的field赋值value。

   如果哈希表不存在，一个新的哈希表被创建并进行 HSET 操作。

   如果字段已经存在于哈希表中，旧值将被重写。

   ```
   hset user name baizhan 创建一个key为user的哈希，并创建name字段，给name字段赋值为baizhan
   hset user age 3        在key为user的哈希中，创建age字段赋值为3
   hset user name shangxuetang 将key为user的哈希name字段修改为shangxuetang
   ```

2. **hget key field**从key哈希中，取出field字段的值。

   ```
   hget user name 从key为user的哈希中取出name字段的值，结果为shangxuetang
   ```

3. **hmset key field1 value1 field2 value2.....**批量设置哈希的字段及值。

   ```
   hmset user1 name bjsxt age 15  创建一个key为user1的哈希，有两个字段name和age
   ```

4. **hexists key field** 判断指定key中是否存在field

   如果哈希表含有给定字段，返回 1 。 如果哈希表不含有给定字段，或 key 不存在，返回 0 。

```
hexists user name 返回1
```

5. **hkeys key**获取该哈希中所有的field。
6. **hvals key**获取该哈希中所有的value。
7. **hincrby key field increment**为哈希表key中的field字段的值加上增量increment。

   增量也可以为负数，相当于对指定字段进行减法操作。

   如果哈希表的 key 不存在，一个新的哈希表被创建并执行 hincrby 命令。

   如果指定的字段不存在，那么在执行命令前，字段的值被初始化为 0 。

   对一个储存字符串值的字段执行 hincrby 命令将造成一个错误。

```
hincrby user age 10 对user中的age字段做运算，增加10
```

8. **hdel key field1 field2.....**删除哈希表 key 中的一个或多个指定字段，不存在的字段将被忽略。

   返回被成功删除字段的数量，不包括被忽略的字段。

```
hdel user name age 删除user中的name和age字段
```

9. **hsetnx key field value**给key哈希表中不存在的的字段赋值 。

   如果哈希表不存在，一个新的哈希表被创建并进行 hsetnx 操作。

   如果字段已经存在于哈希表中，操作无效。

   如果 key 不存在，一个新哈希表被创建并执行 hsetnx 命令。

```
hsetnx user name
```

6 五大数据类型-Zset（有序集合）

6.1 简介

Zset与Set非常相似，是一个没有重复元素的String集合。

不同之处是Zset的每个元素都关联了一个分数（score），这个分数被用来按照从低分到高分的方式排序集合中的元素。集合的元素是唯一的，但分数可以重复。

因为元素是有序的，所以可以根据分数（score）或者次序（position）来获取一个范围内的元素。

6.2 常用命令

1. **zadd key score1 value1 score2 value2.....**将一个或多个元素（value）及分数（score）加入到有序集key中。

   如果某个元素已经是有序集的元素，那么更新这个元素的分数值，并通过重新插入这个元素，来保证该元素在正确的位置上。

   分数值可以是整数值或双精度浮点数。

   如果有序集合 key 不存在，则创建一个空的有序集并执行 zadd 操作。

```
zadd k1 100 java 200 c++ 300 python 400 php
```

2. **zrange key start end [withscores]**返回key集合中的索引start和索引end之间的元素（包含start和end）。

其中元素的位置按分数值递增(从小到大)来排序。 其中 0 表示列表的第一个元素，-1表示最后一个元素。

withscores是可选参数，是否返回分数。

```
zrange k1 0 -1 返回集合中所有元素
zrange k1 0 -1 withscores 返回集合中所有元素，并携带元素分数
```

3. **zrangebyscore key minscore maxscore [withscores]**返回key集合中的分数minscore 和分数maxscore 之间的元素（包含minscore 和maxscore ）。其中元素的位置按分数值递增(从小到大)来排序。

```
zrangebyscore k1 200 400 返回200-400分之间的元素递增排序
```

4. **zrevrangebyscore key maxscore minscore [withscores]**返回key集合中的分数maxscore和分数minxscore 之间的元素（包含maxscore和minxscore ）。其中元素的位置按分数值递减(从大到小)来排序。

```
zrevrangebyscore k1 400 200 返回200-400分之间的元素递减法排序
```

5. **zincrby key increment value**为元素value的score加上increment的值。

```
zincrby k1 50 java 给java元素加上50分
```

6. **zrem key value**删除该集合下value的元素。

```
zrem k1 php 删除php
```

7. **zcount key minscore maxscore**统计该集合在minscore 到maxscore分数区间中元素的个数。

```
zcount k1 100 300 统计100分到300分中间元素的个数
```

8. **zrank key value**返回value在集合中的排名，从0开始。

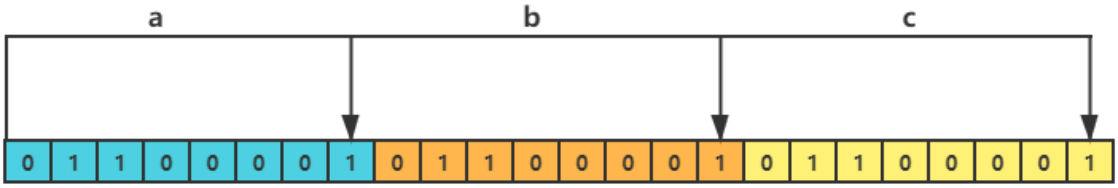
```
zrank k1 c++ 返回c++排名
```

7 新数据类型-Bitmaps

7.1 简介

在计算机中，用二进制（位）作为存储信息的基本单位，1个字节等于8位。

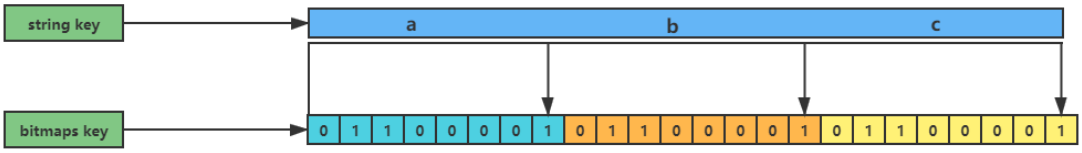
例如 "abc" 字符串是由 3 个字节组成，计算机存储时使用其二进制表示，"abc"分别对应的ASCII码是97、98、99，对应的二进制是01100001、01100010、01100011，在内存中表示如下：



合理地使用 位 能够有效地提高内存使用率和开发效率。

Redis提供了Bitmaps这个 “数据结构” 可以实现对位的操作：

1. Bitmaps 本身不是一种数据结构，实际上它就是字符串（key 对应的 value 就是上图中的一串二进制），但是它可以对字符串的位进行操作。
2. Bitmaps 单独提供了一套命令，所以在 Redis 中使用 Bitmaps 和使用字符串的方法不太相同。可以把 Bitmaps 想象成一个以位为单位的数组，数组的每个单元只能存储 0 和 1，数组的下标在Bitmaps中叫做偏移量。



7.2 常用命令

1. **setbit key offset value**设置Bitmaps中某个偏移量的值。

   偏移量从0开始，且value值只能为0或1。

   ```
   setbit sign 0 1 设置sign的第一位值为1
   setbit sign 1 1 设置sign的第二位值为1
   setbit sign 2 0 设置sign的第三位值为0
   setbit sign 3 1 设置sign的第四位值为1
   ```

2. **getbit key offset** 获取Bitmaps中某个偏移量的值。

   获取key的offset 的值。

   ```
   getbit sign 3 获取偏移量为3的值，结果为1
   ```

   如果偏移量未设置值，则也返回0。

   ```
   getbit sign 99 获取偏移量为99的值，结果为0
   ```

3. **bitcount key [start end]**统计字符串被设置为1的bit数量。一般情况下，给定的整个字符串都会被进行统计，可以选择通过额外的start和end参数，指定**字节组**范围内进行统计（包括start和end），0表示第一个元素，-1表示最后一个元素。

   ```
   bitcount sign 获取整个字符串被设置为1的bit数量，结果为3
   ```

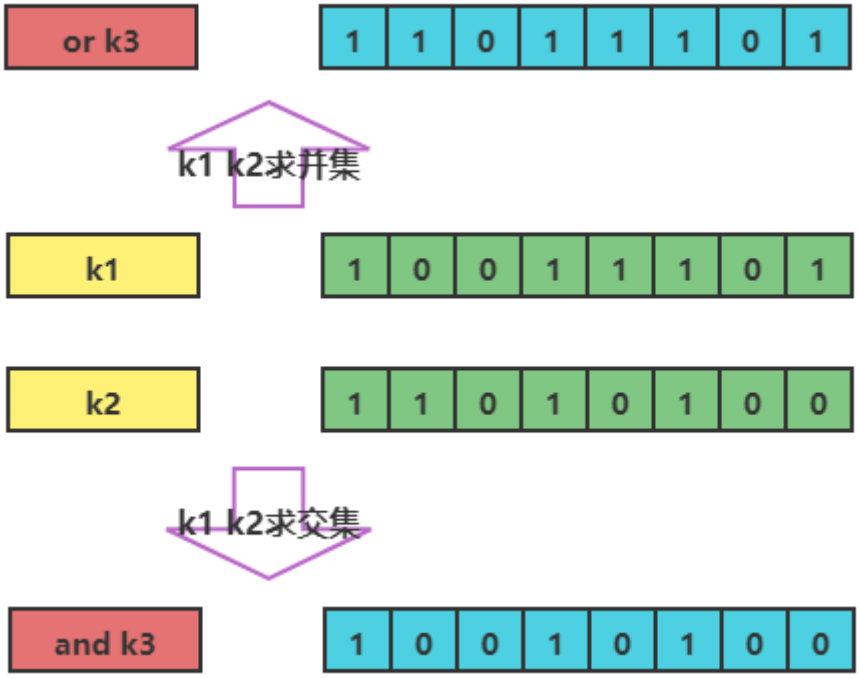
   如：当前存在一个key为k1的bitmaps存储着[00000001,00000001,00000010,00000011]，分别对应[1,1,2,3]。

```
setbit num 7 1
setbit num 15 1
setbit num 22 1
setbit num 30 1
setbit num 31 1

bitcount num 1 2 统计索引1、2两个字节组中bit=1的数量，即统计00000001,00000010中bit=1的数量，结果为2
bitcount num 1 3 统计索引1、2、3三个字节组中bit=1的数量，即统计00000001,00000010,00000011中bit=1的数量，结果为4
bitcount num 0 -1 统计所有的字节组中bit=1的数量，结果为5
```

setbit设置或获取的是bit（位）的位置，bitcount计算的是byte（字节）位置。

4. **bitop and/or destkey sourcekey1 sourcekey2.....**将多个bitmaps通过求交集/并集方式合并成一个新的bitmaps。

```
bitop and k3 k1 k2 通过求交集将k1 k2合并成k3
bitop or k3 k1 k2  通过求并集将k1 k2合并成k3
```



8 新数据类型-Geospatia

8.1 简介

GEO，Geographic,地理信息的缩写。

该类型就是元素的二维坐标，在地图上就是经纬度。Redis基于该类型，提供了经纬度设置、查询、范围查询、距离查询、经纬度Hash等常见操作。

8.2 常用命令

1. **geoadd key longitude latitude member [longitude latitude member .....]**用于存储指定的地理空间位置，可以将一个或多个经度(longitude)、纬度(latitude)、位置名称(member)添加到指定的 key 中。

   ```
   geoadd chinacity 116.405285 39.904989 beijing 将北京的经纬度和名称添加到chinacity
   geoadd chinacity 104.065735 30.659462 chengdu 121.472644 31.231706 shanghai
   将成都和上海的经纬度、名称添加到chinacity
   ```

   有效的经度：-180 ~ +180 有效的纬度：-85.05 ~ +85.05，当设置的经度纬度值超过范围会报错。

   两级无法直接添加。

   一般会直接下载城市数据，直接通过java程序直接一次性导入。

2. **geopos** key member [member .....]**从给定的 key 里返回所有指定名称(member)的位置（经度和纬度），不存在的返回 nil。

   ```
   geopos chinacity shanghai beijing 返回chinacity中名称为shanghai和beijing的经纬度
   ```

3. **geodist key member1 member2 [m|km|ft|mi]**用于返回两个给定位置之间的距离。

   最后一个距离单位参数说明：

   - m： 米，默认单位。
   - km： 千米。
   - mi： 英里。
   - ft： 英尺。

   ```
   geodist chinacity shanghai beijing 返回shanghai和beijing之间的距离，结果
   1067597.9668，单位米
   geodist chinacity shanghai chengdu km 返回shanghai和chengdu之间的距离，结果
   1660.0198，单位是千米
   ```

4. **georadius key longitude latitude radius m|km|ft|mi** 以给定的经纬度（longitude latitude）为中心， 返回键包含的位置元素当中， 与中心的距离不超过给定最大距离（radius ）的所有位置元素。

   ```
   georadius chinacity 116 40 1200 km 获取经纬度116 40为中心，在chinacity内1200公里
   范围内的所有元素。结果shanghai beijing
   ```

9 新数据类型-Hyperloglog

9.1 简介

在我们做站点流量统计的时候一般会统计页面UV(独立访客:unique visitor)和PV(即页面浏览量：page view)。

什么是基数?

数据集｛1，2，5，7，5，7，9｝，那么这个数据集的基数集为｛1，2，5，7，9｝，基数（不重复元素）为5，基数估计就是在误差可接受范围内，快速计算基数。

如果是通过Redis来处理，我们可以使用String类型然后自增计数即可达到统计PV，统计UV可以使用Set，每个用户id是唯一的可以放到这个集合里。

以上方案虽然结果准确，但随着数据不断增加，导致占用的内存空间越来越大，对于非常大的数据集是不合适的。

Hyperloglog 是一种基数估算统计，在输入元素的数量特别巨大时，计算基数所需的空间是固定的，并且很小。

在Redis中，每个Hyperloglog 只占用12KB内存，就可以计算接近2^{64}个不同元素的基数。

因为HyperLogLog 只会更具输入元素来计算基数，而不会存储输入元素本身，所以Hyperloglog 不能像集合那样，返回输入的各个元素。

9.2 常用命令

1. **pfadd key element1 element2.....**将所有元素参数添加到 Hyperloglog 数据结构中。

   如果至少有个元素被添加返回 1， 否则返回 0。

   ```
   pfadd book1 java c++  添加两个元素，当前book1数量为2
   pfadd book1 java php  添加一个元素，当前book1数量为3
   ```

2. **pfcount key1 key2.....**计算Hyperloglog 近似基数，可以计算多个Hyperloglog ，统计基数总数。

   ```
   pfcount book1 计算book1的基数，结果为3
   pfadd book2 chinese math 添加两个元素到book2中
   pfcount book1 book2 统计两个key的基数总数，结果为5
   ```

3. **pfmerge destkey sourcekey1 sourcekey2.....**将一个或多个Hyperloglog（sourcekey1） 合并成一个Hyperloglog （destkey ）。

   比如每月活跃用户可用每天活跃用户合并后计算。

   ```
   pfmerge book book1 book2 将book1和book2合并成book，结果为5
   ```

10 Redis配置文件详解

10.1 units单位

配置大小单位，开头定义基本度量单位，只支持bytes，大小写不敏感。

```
# Note on units: when memory size is needed, it is possible to specify
# it in the usual form of 1k 5GB 4M and so forth:
#
# 1k => 1000 bytes
# 1kb => 1024 bytes
# 1m => 1000000 bytes
# 1mb => 1024*1024 bytes
# 1g => 1000000000 bytes
# 1gb => 1024*1024*1024 bytes
#
# units are case insensitive so 1GB 1Gb 1gB are all the same.

```

10.2 #### INCLUDES

Redis只有一个配置文件，如果多个人进行开发维护，那么就需要多个这样的配置文件，这时候多个配置文件就可以在此通过 include /path/to/local.conf 配置进来，而原本的 redis.conf 配置文件就作为一个总闸。

```
################################## INCLUDES ###################################

# Include one or more other config files here.  This is useful if you
# have a standard template that goes to all Redis servers but also need
# to customize a few per-server settings.  Include files can include
# other files, so use this wisely.
#
# Note that option "include" won't be rewritten by command "CONFIG REWRITE"
# from admin or Redis Sentinel. Since Redis always uses the last processed
# line as value of a configuration directive, you'd better put includes
# at the beginning of this file to avoid overwriting config change at runtime.
#
# If instead you are interested in using includes to override configuration
# options, it is better to use include as the last line.
#
# include /path/to/local.conf
# include /path/to/other.conf
```

10.3 #### NETWORK

1. bind：绑定redis服务器网卡IP，默认为127.0.0.1,即本地回环地址。访问redis服务只能通过本机的客户端连接，而无法通过远程连接。如果bind选项为空的话，那会接受所有来自于可用网络接口的连接。

```
# IF YOU ARE SURE YOU WANT YOUR INSTANCE TO LISTEN TO ALL THE INTERFACES
# JUST COMMENT OUT THE FOLLOWING LINE.
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
bind 127.0.0.1 -::1
```

2. protected-mode：本机保护模式，值为yes时只能本机访问不能远程访问。

```
# By default protected mode is enabled. You should disable it only if
# you are sure you want clients from other hosts to connect to Redis
# even if no authentication is configured, nor a specific set of interfaces
# are explicitly listed using the "bind" directive.
protected-mode yes
```

3. port：指定redis运行的端口，默认是6379。

```
# Accept connections on the specified port, default is 6379 (IANA #815344).
# If port 0 is specified Redis will not listen on a TCP socket.
port 6379
```

4. timeout：设置客户端连接时的超时时间，单位为秒。当客户端在这段时间内没有发出任何指令，那么关闭该连接。默认值为0，表示不关闭。

```
# Close the connection after a client is idle for N seconds (0 to disable)
timeout 0
```

10.4 #### GENERAL

1. daemonize：设置为yes表示指定Redis以守护进程的方式启动（后台启动）。

```
# By default Redis does not run as a daemon. Use 'yes' if you need it.
# Note that Redis will write a pid file in /var/run/redis.pid when daemonized.
# When Redis is supervised by upstart or systemd, this parameter has no impact.
daemonize yes
```

2. pidfile：配置PID文件路径，当redis作为守护进程运行的时候，它会把 pid 默认写到 /var/redis/run/redis_6379.pid 文件里面。

```
# Note that on modern Linux systems "/run/redis.pid" is more conforming
# and should be used instead.
pidfile /var/run/redis_6379.pid
```

3. loglevel： 定义日志级别。默认值为notice，有如下4种取值：
   - debug（记录大量日志信息，适用于开发、测试阶段）。
   - verbose（较多日志信息）。
   - notice（适量日志信息，使用于生产环境）。
   - warning（仅有部分重要、关键信息才会被记录）。

```
# Specify the server verbosity level.
# This can be one of:
# debug (a lot of information, useful for development/testing)
# verbose (many rarely useful info, but not a mess like the debug level)
# notice (moderately verbose, what you want in production probably)
# warning (only very important / critical messages are logged)
loglevel notice
```

4. logfile： 配置log文件地址,默认打印在命令行终端的窗口上。

```
# Specify the log file name. Also the empty string can be used to force
# Redis to log on the standard output. Note that if you use standard
# output for logging but daemonize, logs will be sent to /dev/null
logfile ""
```

5. databases：设置数据库的数目。

```
# Set the number of databases. The default database is DB 0, you can select
# a different one on a per-connection basis using SELECT <dbid> where
# dbid is a number between 0 and 'databases'-1
databases 16
```

10.5 #### SECURITY

1. requirepass：设置redis连接密码。

   比如: requirepass 123 表示redis的连接密码为123。

```
# The requirepass is not compatable with aclfile option and the ACL LOAD
# command, these will cause requirepass to be ignored.
#
# requirepass foobared
```

10.6 其他配置

1. maxclients： 设置客户端最大并发连接数，默认无限制，Redis可以同时打开的客户端连接数为Redis进程可以打开的最大数量。 如果设置 maxclients为0 ， 表示不作限制。当客户端连接数到达限制时，Redis会关闭新的连接并向客户端返回错误信息。

```
# IMPORTANT: When Redis Cluster is used, the max number of connections is also
# shared with the cluster bus: every node in the cluster will use two
# connections, one incoming and another outgoing. It is important to size the
# limit accordingly in case of very large clusters.
#
# maxclients 10000
```

2. maxmemory：设置Redis的最大内存，如果设置为0 。表示不作限制。通常是配合maxmemory-policy参数一起使用。

```
# In short... if you have replicas attached it is suggested that you set a lower
# limit for maxmemory so that there is some free RAM on the system for replica
# output buffers (but this is not needed if the policy is 'noeviction').
#
# maxmemory <bytes>
```

3. maxmemory-policy： 当内存使用达到maxmemory设置的最大值时，redis使用的内存清除策略。

清除策略包括：

- volatile-lru：利用LRU算法移除设置过过期时间的key (LRU:最近使用 Least Recently Used)
- allkeys-lru：利用LRU算法移除任何key
- volatile-random：移除设置过过期时间的随机key
- allkeys-random：移除随机key
- volatile-ttl：移除即将过期的key(minor TTL)
- noeviction：不移除任何key，只是返回一个写错误，默认选项

```
# MAXMEMORY POLICY: how Redis will select what to remove when maxmemory
# is reached. You can select one from the following behaviors:
#
# volatile-lru -> Evict using approximated LRU, only keys with an expire set.
# allkeys-lru -> Evict any key using approximated LRU.
# volatile-lfu -> Evict using approximated LFU, only keys with an expire set.
# allkeys-lfu -> Evict any key using approximated LFU.
# volatile-random -> Remove a random key having an expire set.
# allkeys-random -> Remove a random key, any key.
# volatile-ttl -> Remove the key with the nearest expire time (minor TTL)
# noeviction -> Don't evict anything, just return an error on write operations.
#
# LRU means Least Recently Used
# LFU means Least Frequently Used
#
# Both LRU, LFU and volatile-ttl are implemented using approximated
# randomized algorithms.
#
# Note: with any of the above policies, when there are no suitable keys for
# eviction, Redis will return an error on write operations that require
# more memory. These are usually commands that create new keys, add data or
# modify existing keys. A few examples are: SET, INCR, HSET, LPUSH, SUNIONSTORE,
# SORT (due to the STORE argument), and EXEC (if the transaction includes any
# command that requires memory).
#
# The default is:
#
# maxmemory-policy noeviction
```

11 发布与订阅

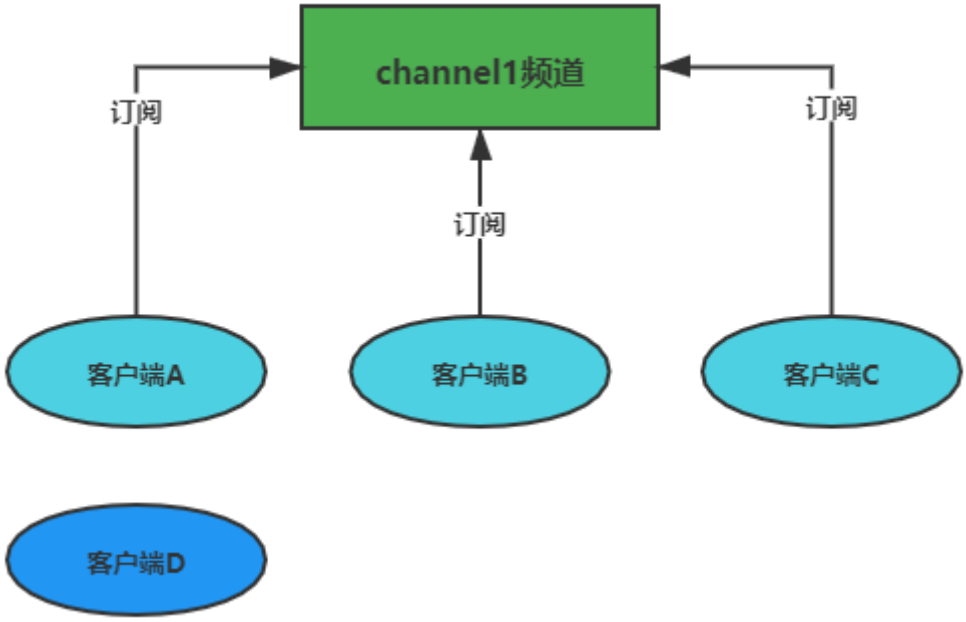
11.1 什么是发布与订阅

Redis 发布订阅 (pub/sub) 是一种消息通信模式：发送者 (pub) 发送消息，订阅者 (sub) 接收消息。

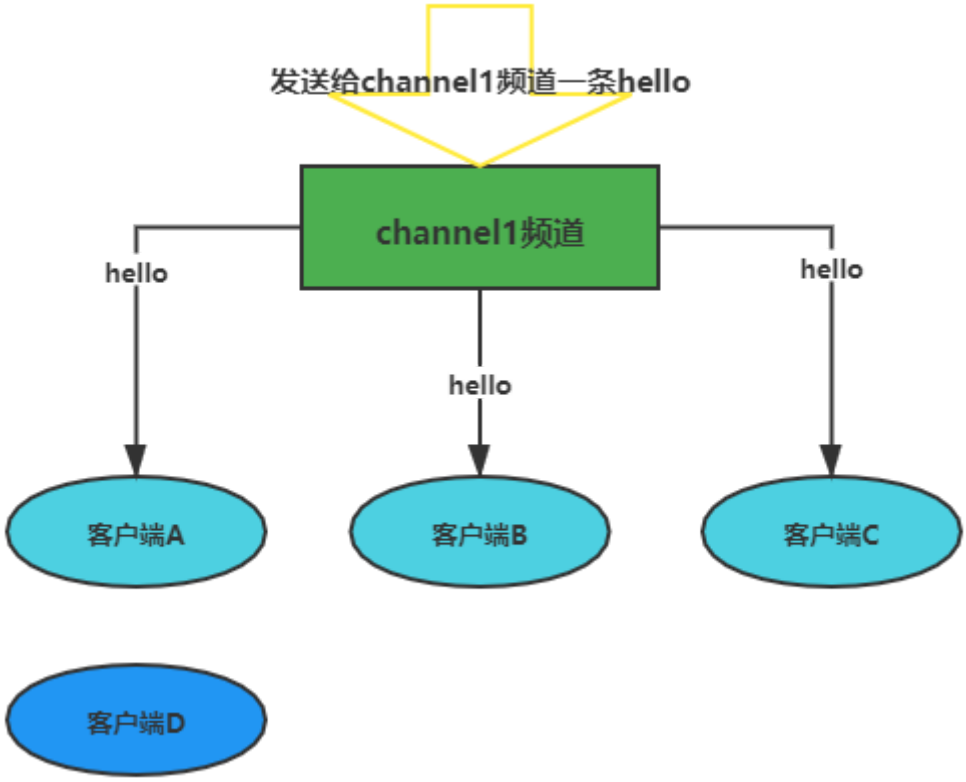
Redis 客户端可以订阅任意数量的频道。

11.2 Redis的发布与订阅

1. 客户端订阅频道



2. 当给这个频道发送消息后，消息就会发送给订阅的客户端



3. Redis中发布与订阅命令

   订阅：subscribe channel 订阅频道channel。

   发布：publish channel msg向频道channel 发送一条msg消息。

11.3 发布与订阅命令行实现

1. 打开一个客户端订阅channel 1频道。

```
127.0.0.1:6379> subscribe channel1
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel1"
3) (integer) 1
```

2. 打开另一个客户端给channel 1频道发送一条hello消息。

```
127.0.0.1:6379> publish channel1 hello
(integer) 1
127.0.0.1:6379>
```

返回1代表订阅者数量。

3. 打开第一个客户端可以看到发送的消息

```
127.0.0.1:6379> subscribe channel1
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel1"
3) (integer) 1
1) "message"
2) "channel1"
3) "hello"
```

客户端只能收到订阅以后发送的消息。

三、Redis持久化

由于Redis的数据都存放在内存中，如果没有配置持久化，Redis重启后数据就全丢失了，于是需要开启Redis的持久化功能，将数据保存到磁盘上，当Redis重启后，可以从磁盘中恢复数据。

Redis提供了两个不同形式的持久化方式：

- RDB(Redis DataBase)
- AOF(Append Only File)

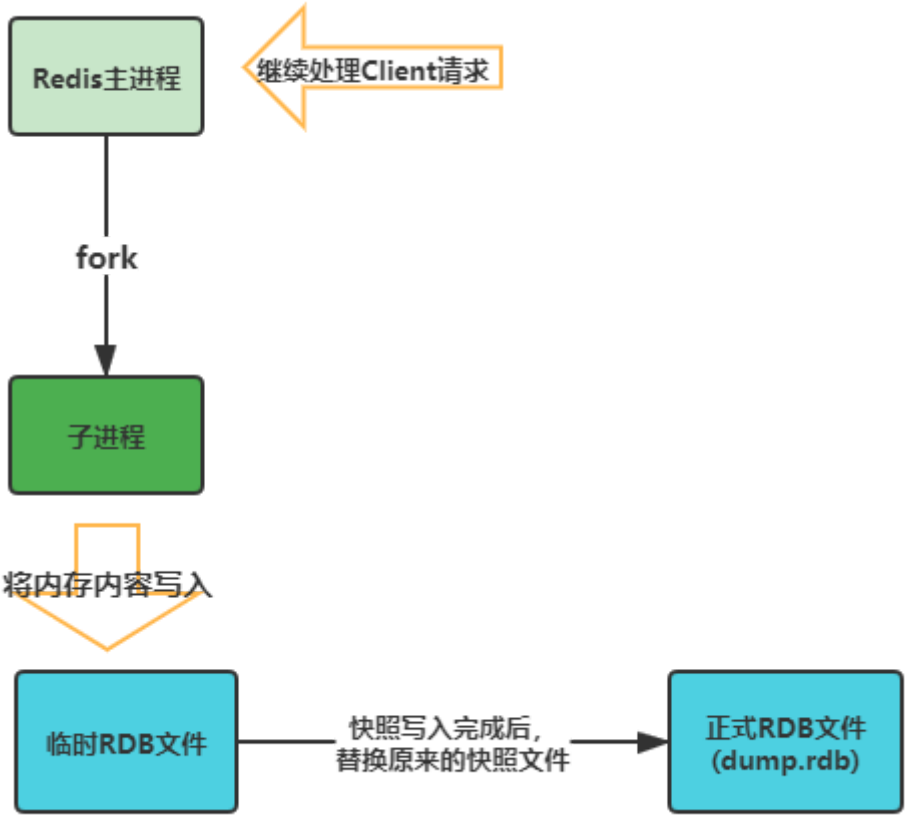
1 持久化操作-RDB

1.1 RDB是什么?

在指定的时间间隔内将内存的数据集快照写入磁盘，也就是行话讲的Snapshot快照，它恢复时是将快照文件直接读到内存里。

1.2 备份过程

Redis会单独创建（fork）一个子进程来进行持久化，会先将数据写入到一个临时文件中，待持久化过程都结束了，再用这个临时文件替换上次持久化好的文件。整个过程中，主进程是不进行任何IO操作的，这就确保了极高的性能如果需要进行大规模数据的恢复，且对于数据恢复的完整性不是非常敏感，那RDB方式要比AOF方式更加的高效。RDB的缺点是最后一次持久化后的数据可能丢失。



1.3 dump.rdb文件

1. RDB保存的文件，在redis.conf中配置文件名称，默认为dump.rdb。

```
# The filename where to dump the DB
dbfilename dump.rdb
```

2. rdb文件的保存位置，也可以修改。默认在Redis启动时命令行所在的目录下。

```
[root@localhost bin]# pwd
/usr/local/bin
[root@localhost bin]# ls
dump.rdb  redis-benchmark  redis-check-aof  redis-check-rdb  redis-cli  redis-sentinel  redis-server
[root@localhost bin]#
```

redis.conf中配置文件路径

```
# The DB will be written inside this directory, with the filename specified
# above using the 'dbfilename' configuration directive.
#
# The Append Only File will also be created inside this directory.
#
# Note that you must specify a directory here, not a file name.
dir ./
```

1.4 如何触发快照?

1.4.1 配置文件中默认的快照配置

```
360 ################################ SNAPSHOTTING  ################################
361
362 # Save the DB to disk.
363 #
364 # save <seconds> <changes>
365 #
366 # Redis will save the DB if both the given number of seconds and the given
367 # number of write operations against the DB occurred.
368 #
369 # Snapshotting can be completely disabled with a single empty string argument
370 # as in following example:
371 #
372 # save ""
373 #
374 # Unless specified otherwise, by default Redis will save the DB:
375 #   * After 3600 seconds (an hour) if at least 1 key changed
376 #   * After 300 seconds (5 minutes) if at least 100 keys changed
377 #   * After 60 seconds if at least 10000 keys changed
378 #
379 # You can set these explicitly by uncommenting the three following lines.
380 #
381 # save 3600 1
382 # save 300 100
383 # save 60 10000
```

1. 快照默认配置

```
save 3600 1：表示3600秒内（一小时）如果至少有1个key的值变化，则保存
save 300 100：表示300秒内（五分钟）如果至少有100个 key 的值变化，则保存
save 60 10000：表示60秒内如果至少有 10000个key的值变化，则保存
```

可以自己配置新的保存规则。

2. 例：给redis.conf添加新的快照策略，30秒内如果有5次key的变化，则触发快照。配置修改后，需要重启Redis服务。

```
380 #
381 # save 3600 1
382 # save 300 100
383 # save 60 10000
384
385 save 30 5
386
```

dump.rdb默认大小是92字节，里面会有一些基本信息。

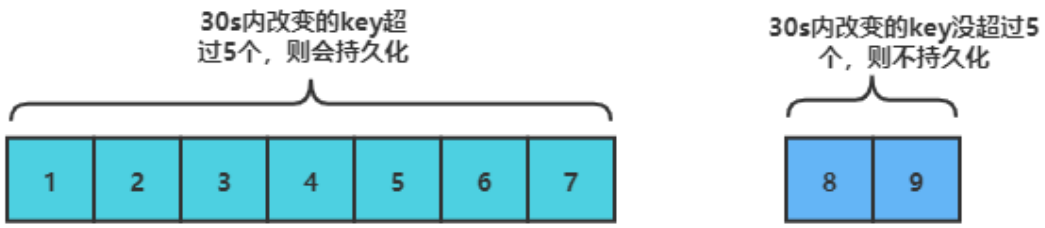
```
[root@localhost bin]# ll
总用量 24844
-rw-r--r--. 1 root root      92 7月   5 22:30 dump.rdb
-rwxr-xr-x. 1 root root  6549080 6月  28 04:55 redis-benchmark
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-aof -> redis-server
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-rdb -> redis-server
-rwxr-xr-x. 1 root root  6763376 6月  28 04:55 redis-cli
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-sentinel -> redis-server
-rwxr-xr-x. 1 root root 12118288 6月  28 04:55 redis-server
[root@localhost bin]#
```

30秒内设置5个以上的值。

```
set k1 v1
set k2 v2
set k3 v3
set k4 v4
set k5 v5
set k6 v6
set k7 v7
```

```
[root@localhost bin]# ll
总用量 24844
-rw-r--r--. 1 root root      132 7月   5 22:32 dump.rdb
-rwxr-xr-x. 1 root root  6549080 6月  28 04:55 redis-benchmark
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-aof -> redis-server
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-rdb -> redis-server
-rwxr-xr-x. 1 root root  6763376 6月  28 04:55 redis-cli
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-sentinel -> redis-server
-rwxr-xr-x. 1 root root 12118288 6月  28 04:55 redis-server
```

dump.rdb大小已经改变。



1.4.2 flushall

执行flushall命令，也会触发rdb规则。

1.4.3 save与bgsave

手动触发Redis进行RDB持久化的命令有两种：

1. save

   该命令会阻塞当前Redis服务器，执行save命令期间，Redis不能处理其他命令，直到RDB过程完成为止，不建议使用。

2. bgsave

   执行该命令时，Redis会在后台异步进行快照操作，快照同时还可以响应客户端请求。

这两个命令是在Redis客户端中执行，并不是redis.conf中修改。

1.4.4 stop-writes-on-bgsave-error

默认值是yes。当Redis无法写入磁盘的话，直接关闭Redis的写操作。

```
# However if you have setup your proper monitoring of the Redis server
# and persistence, you may want to disable this feature so that Redis will
# continue to work as usual even if there are problems with disk,
# permissions, and so forth.
stop-writes-on-bgsave-error yes
```

1.4.5 rdbcompression

默认值是yes。对于存储到磁盘中的快照，可以设置是否进行压缩存储。如果是的话，redis会采用LZF算法进行压缩。如果你不想消耗CPU来进行压缩的话，可以设置为关闭此功能，但是存储在磁盘上的快照会比较大。

```
# Compress string objects using LZF when dump .rdb databases?
# By default compression is enabled as it's almost always a win.
# If you want to save some CPU in the saving child set it to 'no' but
# the dataset will likely be bigger if you have compressible values or keys.
rdbcompression yes
```

1.4.6 rdbchecksum

默认值是yes。在存储快照后，我们还可以让redis使用CRC64算法来进行数据校验，但是这样做会增加大约10%的性能消耗，如果希望获取到最大的性能提升，可以关闭此功能。

```
# RDB files created with checksum disabled have a checksum of zero that will
# tell the loading code to skip the check.
rdbchecksum yes
```

1.5 恢复数据

只需要将rdb文件放在Redis的启动目录，Redis启动时会自动加载dump.rdb并恢复数据。

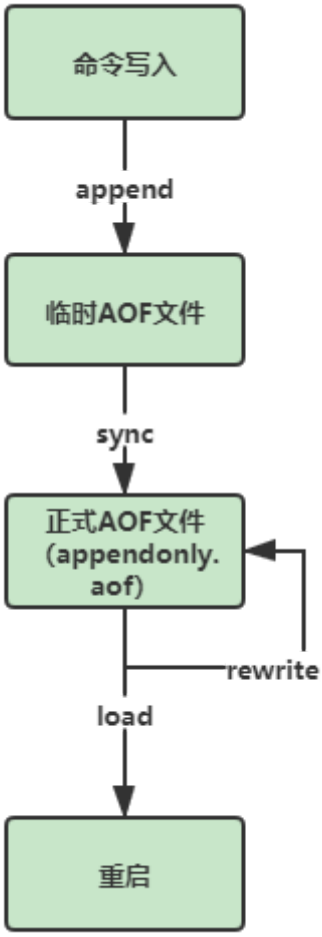
2 持久化操作-AOF

2.1 AOF是什么?

以日志的形式来记录每个写操作，将Redis执行过的所有写指令记录下来（读操作不记录），只允许加文件但不可以改写文件，redis启动之初会读取该文件重新构建数据，换言之，Redis重启的话就根据日志文件的内容将写指令从前到后执行一次以完成数据的恢复工作。

2.2 AOF持久化流程

1. 客户端的请求写命令会被append追加到AOF缓冲区内。
2. AOF缓冲区根据AOF持久化策略[always,everysec,no]将操作同步到磁盘的AOF文件中。
3. AOF文件大小超过重写策略或手动重写时，会对AOF文件rewrite重写，压缩AOF文件容量。
4. Redis服务重启时，会重新load加载AOF文件中的写操作达到数据恢复的目的。



2.3 AOF默认不开启

可以在redis.conf中配置文件名称，默认为appendonly.aof。

AOF文件的保存路径，同RDB的路径一致。

```
# Please check https://redis.io/topics/persistence for more information.

appendonly no

# The name of the append only file (default: "appendonly.aof")

appendfilename "appendonly.aof"
```

如果AOF和RDB同时启动，Redis默认读取AOF的数据。

2.4 AOF启动/修复/恢复

- 正常恢复
  1. 启动：设置Yes：修改默认的appendonly no，改为yes。
  2. 恢复：重启Redis然后重新加载。

例：设置appendonly为yes，配置修改后，需要重启Redis服务。

```
1253
1254 appendonly yes
```

服务器启动后，生成appendonly.aof文件，且大小为0。

```
[root@localhost bin]# ll
总用量 24844
-rw-r--r--. 1 root root       0 7月   6 01:33 appendonly.aof
-rw-r--r--. 1 root root     139 7月   6 01:32 dump.rdb
-rwxr-xr-x. 1 root root 6549080 6月  28 04:55 redis-benchmark
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-check-aof -> redis-server
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-check-rdb -> redis-server
-rwxr-xr-x. 1 root root 6763376 6月  28 04:55 redis-cli
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-sentinel -> redis-server
-rwxr-xr-x. 1 root root 12118288 6月  28 04:55 redis-server
[root@localhost bin]#
```

设置数据。

```
set k11 v11
set k12 v12
set k13 v13
set k14 v14
set k15 v15
```

appendonly.aof大小已经改变。

```
[root@localhost bin]# ll
总用量 24848
-rw-r--r--. 1 root root     226 7月   6 01:37 appendonly.aof
-rw-r--r--. 1 root root     132 7月   6 01:37 dump.rdb
-rwxr-xr-x. 1 root root 6549080 6月  28 04:55 redis-benchmark
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-check-aof -> redis-server
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-check-rdb -> redis-server
-rwxr-xr-x. 1 root root 6763376 6月  28 04:55 redis-cli
lrwxrwxrwx. 1 root root      12 6月  28 04:55 redis-sentinel -> redis-server
-rwxr-xr-x. 1 root root 12118288 6月  28 04:55 redis-server
[root@localhost bin]#
```

- 异常恢复
  1. 启动：设置Yes：修改默认的appendonly no，改为yes。
  2. 修复：如遇到AOF文件损坏，通过/user/local/bin/redis-check-aof --fix appendonly.aof进行恢复。
  3. 恢复：重启Redis然后重新加载。

例：服务启动和数据设置同上，模拟损坏appendonly.aof文件

```
vi appendonly.aof
```

在文件最后追加Hello World文本，破坏appendonly.aof原有格式，使其不可用。

```
k7
$2
v7
Hello World
```

重启Redis服务，并连接。由于aof文件被破坏，导致服务器启动失败。

```
[root@localhost bin]# ./redis-server ../myredis/redis.conf
[root@localhost bin]# ./redis-cli
Could not connect to Redis at 127.0.0.1:6379: Connection refused
not connected>
```

通过/user/local/bin/redis-check-aof --fix工具对appendonly.aof进行恢复。

```
./redis-check-aof --fix appendonly.aof
```

```
[root@localhost bin]# ./redis-check-aof --fix appendonly.aof
0x              e2: Expected prefix '*', got: 'H'
AOF analyzed: size=239, ok_up_to=226, ok_up_to_line=55, diff=13
This will shrink the AOF from 239 bytes, with 13 bytes, to 226 byte
Continue? [y/N]: y
Successfully truncated AOF
```

修复成功。再次查看aof文件，破坏的地方已经修复。再次启动服务器成功。

```
k7
$2
v7
~
"appendonly.aof" [dos] 54L, 226C
```

2.5 AOF同步频率设置

```
1280
1281 # appendfsync always
1282 appendfsync everysec
1283 # appendfsync no
```

1. appendfsync always

   始终同步，每次Redis的写入都会立刻记入日志，性能较差但数据完整性比较好。

2. appendfsync everysec

   每秒同步，每秒记入日志一次，如果宕机，本秒的数据可能丢失。

3. appendfsync no

   redis不主动进行同步，把同步时机交给操作系统。

2.6 Rewrite

1. AOF采用文件追加方式，文件会越来越大为避免出现此种情况，新增了重写机制，当AOF文件的大小超过所设定的阈值时，Redis就会启动AOF文件的内容压缩，只保留可以恢复数据的最小指令集。

   例：设置k1为0，然后incr 进行了4次，k1对应的值会是4，其实就相当于set k1 4

2. 重写虽然可以节约大量磁盘空间，减少恢复时间。但是每次重写还是有一定的负担的，因此设定Redis要满足一定条件才会进行重写。

   ```
   1322 # Specify a percentage of zero in order to disable the automatic AOF
   1323 # rewrite feature.
   1324
   1325 auto-aof-rewrite-percentage 100
   1326 auto-aof-rewrite-min-size 64mb
   ```

   redis.conf默认配置

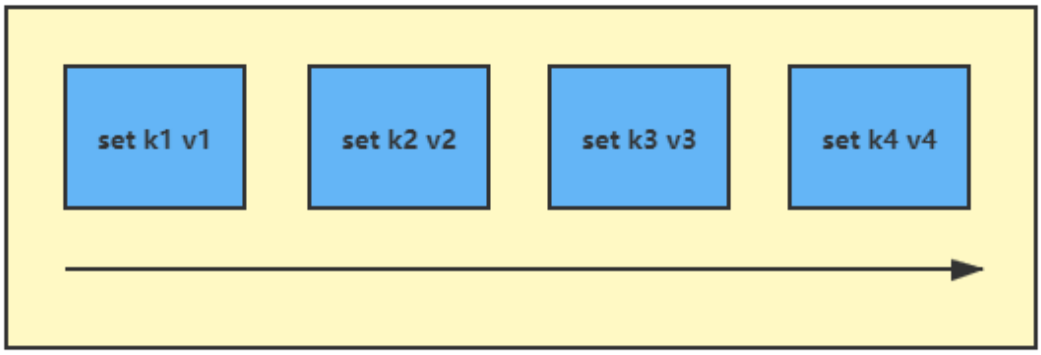
   auto-aof-rewrite-min-size：表示重写时，文件大小必须必这个值要大。

   auto-aof-rewrite-percentage：表示目前文件大小比上次重写后的文件大小大这么多才行。

四、Redis事务

1 Redis事务简介

1. Redis事务是一组命令的集合，一个事务中的所有命令都将被序列化，按照一次性、顺序性、排他性的执行一系列的命令。



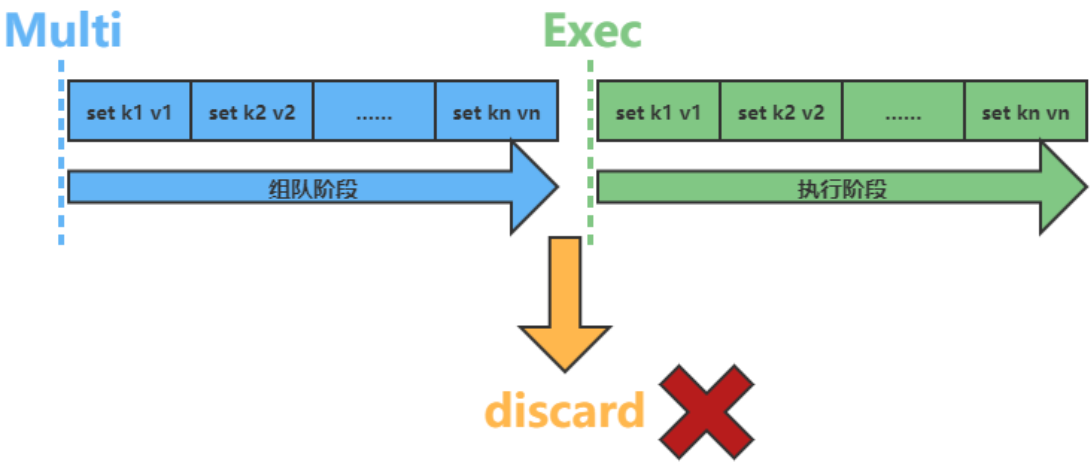
2. Redis单条命令保证原子性，但是事务不保证原子性，且没有回滚。事务中任意命令执行失败，其余的命令仍会被执行。
3. Redis事务没有隔离级别的概念。批量操作在执行前被放入缓存队列，并不会被实际执行，也就不存在事务内的查询要看到事务里的更新，事务外查询不能看到。
4. Redis事务的三个阶段：
    - 开始事务
    - 命令入队
    - 执行事务

2 Redis事务基本操作

2.1 Multi、Exec、discard

事务从输入Multi命令开始，输入的命令都会依次压入命令缓冲队列中，并不会执行，直到输入Exec后，Redis会将之前的命令缓冲队列中的命令依次执行。

组队过程中，可以通过discard来放弃组队。



例1：

```
multi     开始事务
set k1 v1 进行组队，并不执行
set k2 v2 进行组队，并不执行
exec      执行队列命令，依次设置k1 k2
```

```
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set k1 v1
QUEUED
127.0.0.1:6379(TX)> set k2 v2
QUEUED
127.0.0.1:6379(TX)> exec
1) OK
2) OK
```

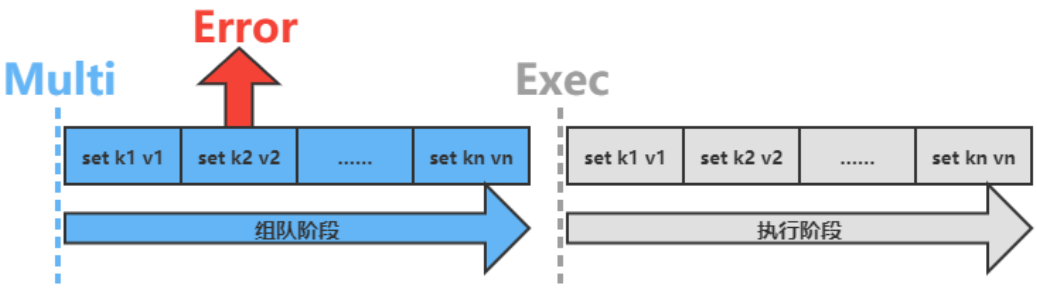
例2：

```
multi     开始事务
set k3 v3 进行组队，并不执行
set k4 v4 进行组队，并不执行
discard   取消组队，都不执行
```

```
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set k3 v3
QUEUED
127.0.0.1:6379(TX)> set k4 v4
QUEUED
127.0.0.1:6379(TX)> discard
OK
127.0.0.1:6379> get k3
(nil)
127.0.0.1:6379>
```

2.2 事务的错误处理

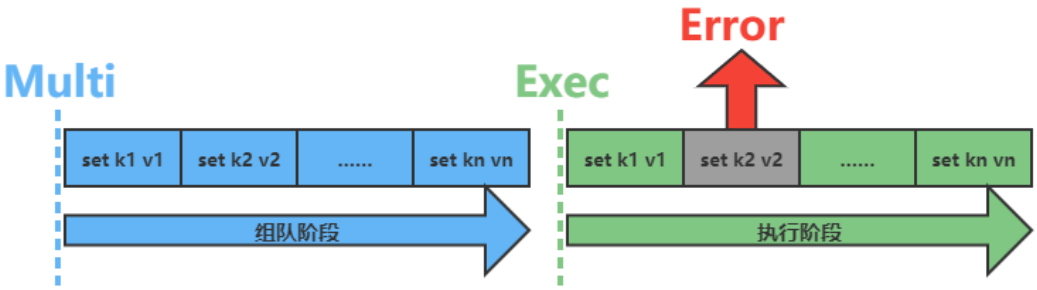
1. 组队阶段某个命令出现了错误，整个队列中的命令都不执行。



例：整个命令缓存队列都不会执行。

```
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set b1 v1
QUEUED
127.0.0.1:6379(TX)> set b2 v2
QUEUED
127.0.0.1:6379(TX)> set b3
(error) ERR wrong number of arguments for 'set' command
127.0.0.1:6379(TX)> exec
(error) EXECABORT Transaction discarded because of previous errors.
127.0.0.1:6379> get b1
(nil)
127.0.0.1:6379>
```

2. 执行阶段某个命令出现了错误，只有报错的命令不会执行，其他正常执行。



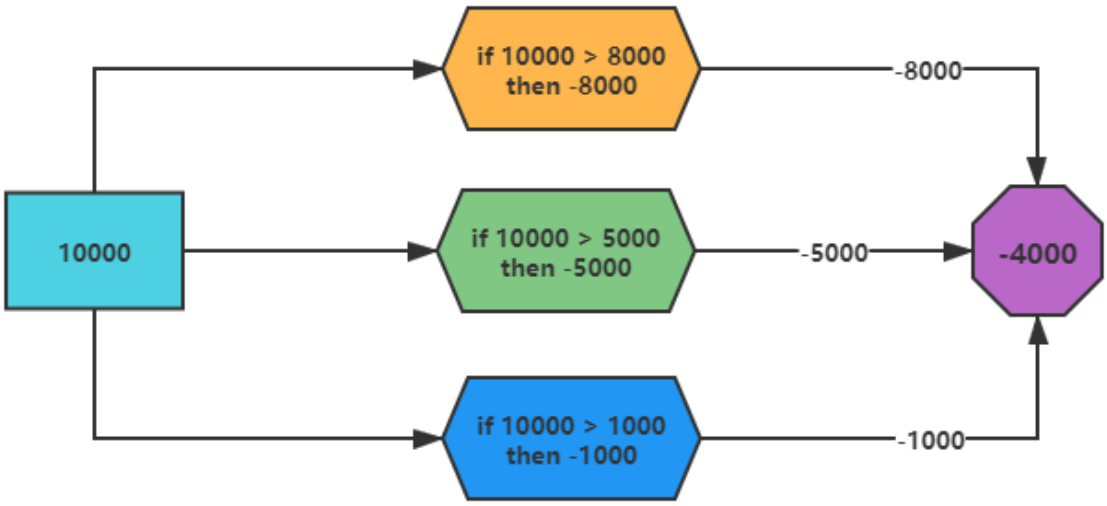
例：只有报错的命令没有执行。

```
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set c1 v1
QUEUED
127.0.0.1:6379(TX)> incr c1
QUEUED
127.0.0.1:6379(TX)> set c2 v2
QUEUED
127.0.0.1:6379(TX)> exec
1) OK
2) (error) ERR value is not an integer or out of range
3) OK
127.0.0.1:6379> mget c1 c2
1) "v1"
2) "v2"
127.0.0.1:6379>
```

3 悲观锁与乐观锁

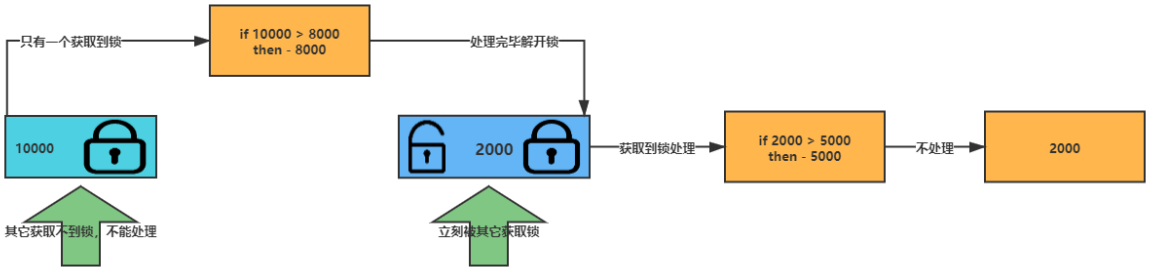
3.1 事务应用场景

- 一个请求想给余额减8000
- 一个请求想给余额减5000
- 一个请求想给余额减1000



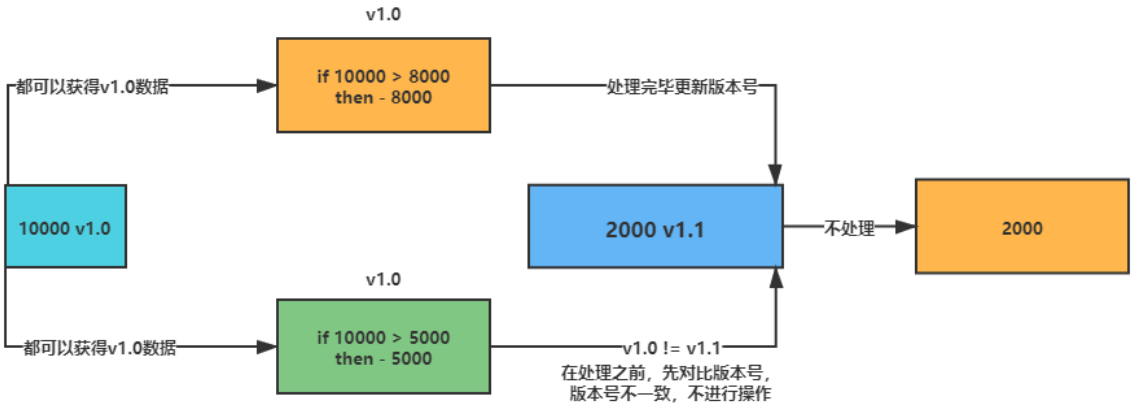
3.2 悲观锁（Pessimistic Lock）

每次去拿数据的时候都认为别人会修改，所以在每次拿数据的时候都会上锁，这样别人想拿这个数据就会阻塞，直到它拿到锁。传统关系型数据库就用到很多悲观锁，比如行锁、表锁等。



3.3 乐观锁（Optimistic Lock）

每次去拿数据的时候都认为别人不会修改，所以不会上锁，但在更新的时候会判断一下在此期间别人有没有去更新这个数据，可以使用版本号机制。乐观锁适用于多读的应用类型，可以提高吞吐量。Redis就是乐观锁机制实现事务的。



4 Redis中的乐观锁

4.1 watch key [key.....]

在执行multi之前，先执行watch监视一个或多个key，如果在事务执行之前这个（或这些）key被其它命令所改动，那么事务将被打断。

例：开启两个客户端

```
1号客户端
set money 10000 设置一个money的key
watch money 监视money
multi 开启事务
set money 100 修改money，暂时不执行事务

2号客户端
watch money 监视money
multi 开启事务
set money 110 修改money，暂时不执行事务

1号客户端
exec  执行事务，返回成功
```

```
2号客户端
exec    执行事务，更新失败
```

1号客户端

```
127.0.0.1:6379> set money 10000
OK
127.0.0.1:6379> watch money
OK
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set money 100
QUEUED
127.0.0.1:6379(TX)> exec
1) OK
127.0.0.1:6379>
```

2号客户端

```
127.0.0.1:6379> watch money
OK
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> set money 110
QUEUED
127.0.0.1:6379(TX)> exec
(nil)
127.0.0.1:6379>
```

4.2 unwatch

取消watch命令对所有key的监视

如果在执行watch命令之后，exec命令或discard命令先执行的话，那么就不需要再执行unwatch。

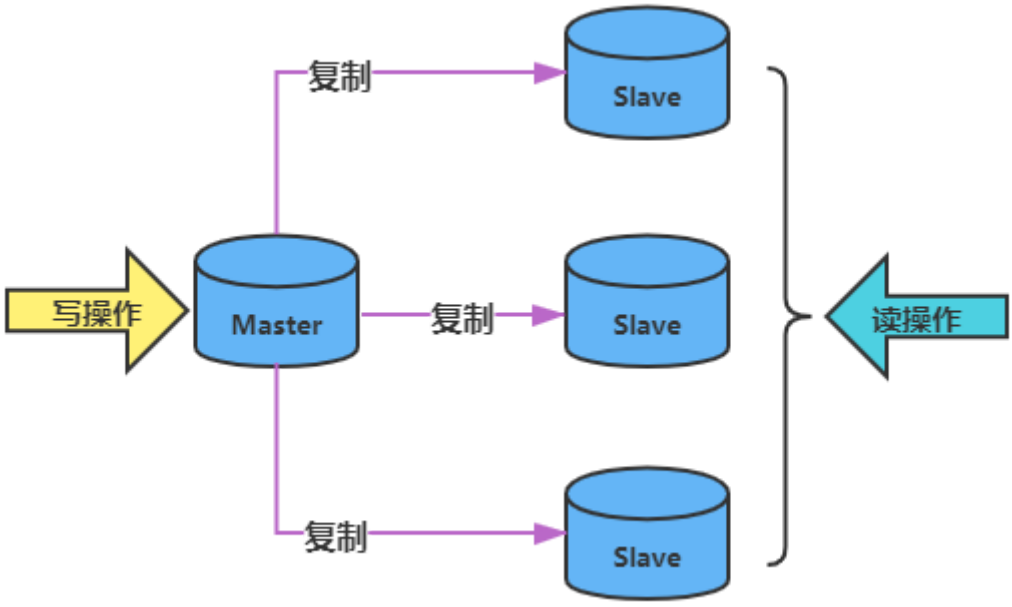
五、Redis主从复制

1 Redis主从复制简介

主从复制，是指将一台Redis服务器的数据，复制到其他的Redis服务器。前者称为主节点(Master)，后者称为从节点(Slave)；数据的复制是单向的，只能由主节点到从节点。

默认情况下，每台Redis服务器都是主节点；且一个主节点可以有多个从节点(或没有从节点)，但一个从节点只能有一个主节点。

Master以写为主，Slave以读为主。



主从复制的作用：

1. 数据冗余：主从复制实现了数据的热备份，是持久化之外的一种数据冗余方式。
2. 故障恢复：当主节点出现问题时，可以由从节点提供服务，实现快速的故障恢复；实际上是一种服务的冗余。
3. 负载均衡：在主从复制的基础上，配合读写分离，可以由主节点提供写服务，由从节点提供读服务（即写Redis数据时应用连接主节点，读Redis数据时应用连接从节点），分担服务器负载；尤其是在写少读多的场景下，通过多个从节点分担读负载，可以大大提高Redis服务器的并发量。
4. 高可用基石：除了上述作用以外，主从复制还是哨兵和集群能够实施的基础，因此说主从复制是Redis高可用的基础。

一般来说，要将Redis运用于工程项目中，只是用一台Redis是万万不能的，原因如下：

1. 从结构上，单个redis服务器会发生单点故障，并且一台服务器需要处理所有的请求负载，压力较大；
2. 从容量上，单个redis服务器内存容量有限，就算一台Redis服务器内容容量为256G，也不能将所有内存用做Redis存储内存，一般来说，**单台Redis最大使用内存不应该超过20G。**

2 Redis主从复制-一主多从

搭建一主二从的Redis服务器

环境搭建

1. 在同一台虚拟机上配置一主二从Redis服务器，由于单台机器，同一个端口只允许一个进程占用，所以需要修改其他两台Redis的端口。
2. 在/usr/local/myredis文件夹下，新创建三个redis的配置文件，分别为redis6379.conf、redis6380.conf、redis6381.conf，由文件名可知，redis的端口号分别为6379、6380、6381。

```
[root@localhost myredis]# ll
总用量 92
-rw-r--r--. 1 root root     0 7月  14 22:48 redis6379.conf
-rw-r--r--. 1 root root     0 7月  14 22:48 redis6380.conf
-rw-r--r--. 1 root root     0 7月  14 22:48 redis6381.conf
-rw-r--r--. 1 root root 93737 7月   6 01:32 redis.conf
[root@localhost myredis]#
```

   创建出3个空文件，redis.conf是原有的配置文件。

3. 在三个配置文件写入内容

- 可以通过include /usr/local/myredis/redis.conf将公共基础配置直接引入文件。

  将include /usr/local/myredis/redis.conf统一添加到这三个文件中
- 在各个文件中，添加对应的pidfile、port、dbfilename

  如：

  redis6379.conf中添加

  pidfile /var/run/redis_6379.pid

  port 6379

  dbfilename dump6379.rdb

```
include /usr/local/myredis/redis.conf
pidfile /var/run/redis_6379.pid
port 6379
dbfilename dump6379.rdb
~
~
~
~
~
```

  redis6380.conf中添加

  pidfile /var/run/redis_6380.pid

  port 6380

  dbfilename dump6380.rdb

```
include /usr/local/myredis/redis.conf
pidfile /var/run/redis_6380.pid
port 6380
dbfilename dump6380.rdb
~
~
~
~
~
```

  redis6381.conf中添加

  pidfile /var/run/redis_6381.pid

  port 6381

  dbfilename dump6381.rdb

```
include /usr/local/myredis/redis.con
pidfile /var/run/redis_6381.pid
port 6381
dbfilename dump6381.rdb
~
~
~
~
~
```

4. 通过不同的redis.conf文件，分别启动3个redis服务

```
./redis-server /usr/local/myredis/redis6379.conf
./redis-server /usr/local/myredis/redis6380.conf
./redis-server /usr/local/myredis/redis6381.conf
```

```
[root@localhost bin]# ./redis-server /usr/local/myredis/redis6379.conf
[root@localhost bin]# ./redis-server /usr/local/myredis/redis6380.conf
[root@localhost bin]# ./redis-server /usr/local/myredis/redis6381.conf
[root@localhost bin]#
```

通过ps -ef | grep redis命令查看redis服务，3个服务均已启动

```
[root@localhost bin]# ps -ef | grep redis
root        2791       1  0 23:24 ?        00:00:00 ./redis-server 127.0.0.1:6379
root        2850       1  0 23:29 ?        00:00:00 ./redis-server 127.0.0.1:6380
root        2856       1  0 23:29 ?        00:00:00 ./redis-server 127.0.0.1:6381
root        2879    2163  0 23:30 pts/0    00:00:00 grep --color=auto redis
[root@localhost bin]#
```

5. 在redis客户端中，通过info replication命令可以查看Redis服务器当前状态

   可以给redis-cli命令添加-p参数，来指定链接哪个服务器

```
./redis-cli -p 6379
./redis-cli -p 6380
./redis-cli -p 6381
```

```
127.0.0.1:6380> info replication
# Replication
role:master
connected_slaves:0
master_failover_state:no-failover
master_replid:cf6cdfcea6961af343fd9937f13672fc37d67735
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:0
second_repl_offset:-1
repl_backlog_active:0
repl_backlog_size:1048576
repl_backlog_first_byte_offset:0
repl_backlog_histlen:0
127.0.0.1:6380>
```

   所有的服务器目前都是master

6. 配从（6380、6381）不配主（6379）

   slaveof ip port成为某个实例的从服务器。

```
在从机6380和6381上执行：slaveof 127.0.0.1 6379
```

   在主库（6379）中再次执行info replication

```
127.0.0.1:6379> info replication
# Replication
role:master
connected_slaves:2
slave0:ip=127.0.0.1,port=6380,state=online,offset=210,lag=0
slave1:ip=127.0.0.1,port=6381,state=online,offset=210,lag=0
master_failover_state:no-failover
master_replid:4fa2ad14066f41496b3bb2ec4e78447fdc36736e
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:210
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:1
repl_backlog_histlen:210
127.0.0.1:6379>
```

   有两个从库，分别为6380和6381。

7. 在主库（6379）中写数据，可以在从库（6380、6381）中读取到。

注意：

- 在从库中进行写操作，会报错。
- 由于该主从复制在同一台虚拟机上搭建，所以需要修改端口号，如果在多台服务器上搭建主从复制，则需要修改相对应的ip。
- 如果在多台服务器上搭建主从复制，一定要开放远程链接。

3 Redis主从复制-复制原理

3.1 主从复制的一些问题

1. 如果Master断开（宕机），Slave依然连接着Master，可以正常使用读操作，但是没有写操作。如果Master恢复正常，Slave依旧可以直接获取Master写的信息。
2. 如果Slave断开（宕机），当该Slave重启成功，则会变为Master，需要通过slaveof 恢复成Slave，只要变为Slave，立刻可以从Master同步所有数据。

3.2 复制原理

- Slave启动成功连接到Master后会主动发送一个同步（sync）命令。
- Master接到Slave的命令，把Master数据进行持久化，把rdb文件发送给Slave，Slave拿到rdb进行读取。
- 每次Master进行写操作之后，会和Slave进行数据同步。

- **全量复制**：一般发生在Slave初始化阶段，这时Slave需要将Master上的所有数据都复制一份。
- **增量复制**：指Slave初始化后开始正常工作时主服务器发生的写操作同步到从服务器的过程。

3.3 薪火相传

上一个Slave（从机）是下一个Slave（从机）的Master（主机）。

优点：Slave同样可以接收其他Slave的连接和同步请求，那么该Slave作为了链条中下一个的Master, 可以有效减轻Master的压力，去中心化降低风险。

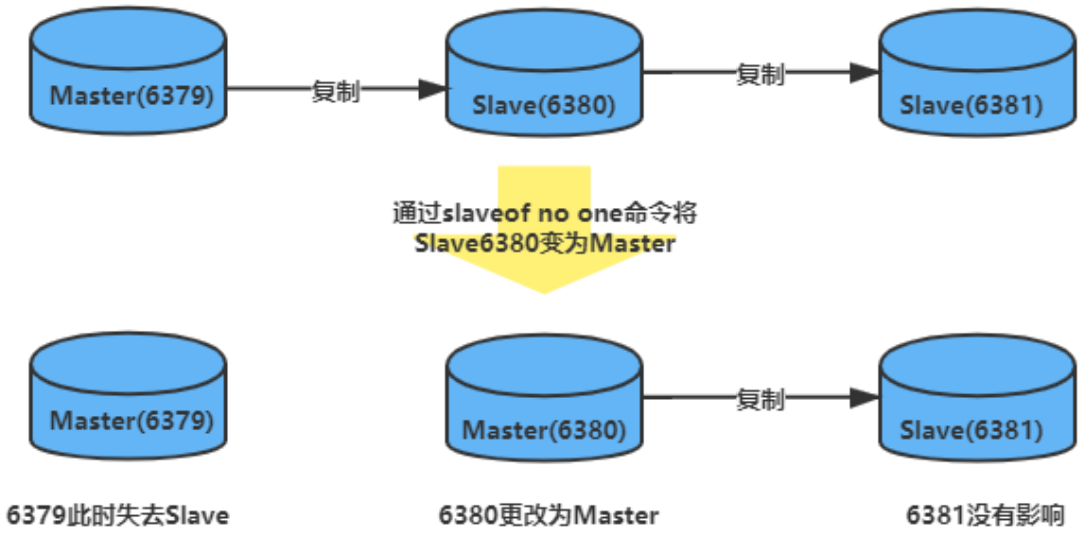
缺点：一旦某个Slave宕机，后面的Slave都无法备份。

注意：

- 也是通过slaveof ip port命令修改Master。
- 中途变更转向:会清除之前的数据，重新建立拷贝最新的。
- Slave6380本质上仍然是从库，只能读、不能写。

3.4 反客为主

当一个Master宕机后，后面的Slave可以立刻升为Master，其后面的Slave不用做任何修改。

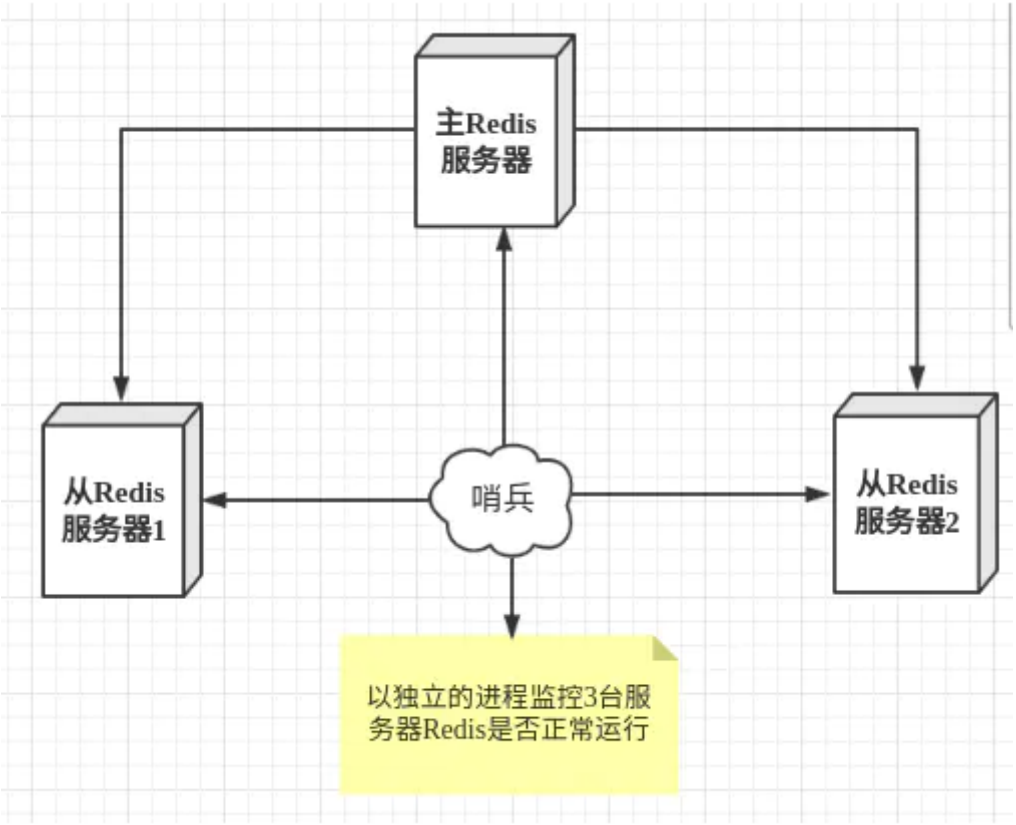


通过slaveof no one 将Slave变为Master。

4 Redis主从复制-哨兵模式（Sentinel）

反客为主的自动版，能够后台监控Master是否故障，如果故障了，根据投票数自动将Slave转换为Master。

哨兵模式是一种特殊的模式，首先Redis提供了哨兵的命令，哨兵是一个独立的进程，作为进程，它会独立运行。其原理是**哨兵通过发送命令，等待Redis服务器响应，从而监控运行的多个Redis实例。**



4.1 哨兵模式的使用

1. 将服务器调整为一主多从（6379带6380、6381）。

```
127.0.0.1:6379> info replication
# Replication
role:master
connected_slaves:2
slave0:ip=127.0.0.1,port=6380,state=online,offset=140,lag=1
slave1:ip=127.0.0.1,port=6381,state=online,offset=140,lag=1
master_failover_state:no-failover
master_replid:27a9fe5424cecd7a8ccbec700d27519c93e12e37
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:140
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:1
repl_backlog_histlen:140
```

2. 在/usr/local/myredis文件夹下创建sentinel.conf文件（名字一定不能错）。

```
[root@localhost myredis]# ls
redis6379.conf  redis6380.conf  redis6381.conf  redis.conf  sentinel.conf
[root@localhost myredis]#
```

3. 配置哨兵，sentinel.conf添加内容

   sentinel monitor myredis 127.0.0.1 6379 1

   其中：myredis 为监控对象起的服务器名称（随意），1代表至少有1个哨兵投票同意迁移

   如果Master存在密码，需要配置sentinel auth-pass 服务器名 密码

   sentinel auth-pass myredis 123456

```
sentinel monitor myredis 127.0.0.1 6379 1
~
~
~
```

4. 启动哨兵，在/usr/local/bin/文件夹下执行./redis-sentinel /usr/local/myredis/sentinel.conf命令。

```
[root@localhost bin]# ./redis-sentinel /usr/local/myredis/sentinel.conf
5948:X 15 Jul 2021 05:07:47.328 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
5948:X 15 Jul 2021 05:07:47.328 # Redis version=6.2.4, bits=64, commit=00000000, modified=0, pid=5948, just started
5948:X 15 Jul 2021 05:07:47.328 # Configuration loaded
5948:X 15 Jul 2021 05:07:47.328 * Increased maximum number of open files to 10032 (it was originally set to 1024).
5948:X 15 Jul 2021 05:07:47.328 * monotonic clock: POSIX clock_gettime

                Redis 6.2.4 (00000000/0) 64 bit

                Running in sentinel mode
                Port: 26379
                PID: 5948

                      https://redis.io

5948:X 15 Jul 2021 05:07:47.328 # WARNING: The TCP backlog setting of 511 cannot be enforced because /proc/sys/net/core/somaxconn is set to the lower value of 128.
5948:X 15 Jul 2021 05:07:47.329 # Sentinel ID is fa0e39fab561ab0a81fe871462e0e21e8a61fa88
5948:X 15 Jul 2021 05:07:47.329 # +monitor master myredis 127.0.0.1 6379 quorum 1
```

5. 模拟Master宕机，哨兵会切换Master

   - 通过kill命令关闭Master（6379）

```
[root@localhost bin]# ps -ef | grep redis
root         2850       1  0 7月14 ?        00:00:10 ./redis-server 127.0.0.1:6380
root         2856       1  0 7月14 ?        00:00:10 ./redis-server 127.0.0.1:6381
root         5451       1  0 04:18 ?         00:00:01 ./redis-server 127.0.0.1:6379
root         5988    2163  0 05:11 pts/0     00:00:00 ./redis-sentinel *:26379 [sentinel]
root         5994    4986  0 05:12 pts/1     00:00:00 grep --color=auto redis
[root@localhost bin]# kill -9 5451
[root@localhost bin]#
```

   - 等待一段时间，哨兵窗口就会输出信息

```
[root@localhost bin]# ./redis-sentinel /usr/local/myredis/sentinel.conf
5988:X 15 Jul 2021 05:11:54.077 # oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
5988:X 15 Jul 2021 05:11:54.077 # Redis version=6.2.4, bits=64, commit=00000000, modified=0, pid=5988, just started
5988:X 15 Jul 2021 05:11:54.077 # Configuration loaded
5988:X 15 Jul 2021 05:11:54.077 * Increased maximum number of open files to 10032 (it was originally set to 1024).
5988:X 15 Jul 2021 05:11:54.077 * monotonic clock: POSIX clock_gettime

                Redis 6.2.4 (00000000/0) 64 bit

                Running in sentinel mode
                Port: 26379
                PID: 5988

                      https://redis.io

5988:X 15 Jul 2021 05:11:54.078 # WARNING: The TCP backlog setting of 511 cannot be enforced because /proc/sys/net/core/somaxconn is set to the lower value of 128.
5988:X 15 Jul 2021 05:11:54.078 # Sentinel ID is fa0e39fab561ab0a81fe871462e0e21e8a61fa88
5988:X 15 Jul 2021 05:11:54.078 # +monitor master myredis 127.0.0.1 6379 quorum 1
5988:X 15 Jul 2021 05:13:22.620 # +sdown master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.620 # +odown master myredis 127.0.0.1 6379 #quorum 1/1
5988:X 15 Jul 2021 05:13:22.620 # +new-epoch 1
5988:X 15 Jul 2021 05:13:22.620 # +try-failover master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.621 # +vote-for-leader fa0e39fab561ab0a81fe871462e0e21e8a61fa88 1
5988:X 15 Jul 2021 05:13:22.621 # +elected-leader master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.621 # +failover-state-select-slave master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.677 # +selected-slave slave 127.0.0.1:6381 127.0.0.1 6381 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.677 * +failover-state-send-slaveof-noone slave 127.0.0.1:6381 127.0.0.1 6381 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:22.736 * +failover-state-wait-promotion slave 127.0.0.1:6381 127.0.0.1 6381 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:23.463 # +promoted-slave slave 127.0.0.1:6381 127.0.0.1 6381 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:23.463 # +failover-state-reconf-slaves master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:23.553 * +slave-reconf-sent slave 127.0.0.1:6380 127.0.0.1 6380 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:24.480 * +slave-reconf-inprog slave 127.0.0.1:6380 127.0.0.1 6380 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:24.480 * +slave-reconf-done slave 127.0.0.1:6380 127.0.0.1 6380 @ myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:24.547 # +failover-end master myredis 127.0.0.1 6379
5988:X 15 Jul 2021 05:13:24.547 # +switch-master myredis 127.0.0.1 6379 127.0.0.1 6381
5988:X 15 Jul 2021 05:13:24.547 * +slave slave 127.0.0.1:6380 127.0.0.1 6380 @ myredis 127.0.0.1 6381
5988:X 15 Jul 2021 05:13:24.547 * +slave slave 127.0.0.1:6379 127.0.0.1 6379 @ myredis 127.0.0.1 6381
5988:X 15 Jul 2021 05:13:54.627 # +sdown slave 127.0.0.1:6379 127.0.0.1 6379 @ myredis 127.0.0.1 6381
```

   根据窗口信息可知，Master由6379转换为6381，进入6381客户端执行info replication

```
[root@localhost bin]# ./redis-cli -p 6381
127.0.0.1:6381> info replication
# Replication
role:master
connected_slaves:1
slave0:ip=127.0.0.1,port=6380,state=online,offset=49078,lag=1
master_failover_state:no-failover
master_replid:d9193b4595aeeb7408db8c8a8682ed97a31543a9
master_replid2:27a9fe5424cecd7a8ccbec700d27519c93e12e37
master_repl_offset:49210
second_repl_offset:23576
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:127
repl_backlog_histlen:49084
127.0.0.1:6381>
```

6. 重新启动6379服务器后，自动转换为Slave。

```
[root@localhost bin]# ./redis-server ../myredis/redis6379.conf
[root@localhost bin]# ./redis-cli -p 6379
127.0.0.1:6379> info replication
# Replication
role:slave
master_host:127.0.0.1
master_port:6381
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
slave_repl_offset:59320
slave_priority:100
slave_read_only:1
replica_announced:1
connected_slaves:0
master_failover_state:no-failover
master_replid:d9193b4595aeeb7408db8c8a8682ed97a31543a9
master_replid2:0000000000000000000000000000000000000000
master_repl_offset:59320
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:59321
repl_backlog_histlen:0
127.0.0.1:6379>
```

4.2 复制延迟

由于所有的写操作都是先在Master上操作，然后同步更新到Slave上，所以从Master同步到Slave机器有一定的延迟，Slave机器数量的增加，会使延迟问题会更加严重。

六、Redis集群

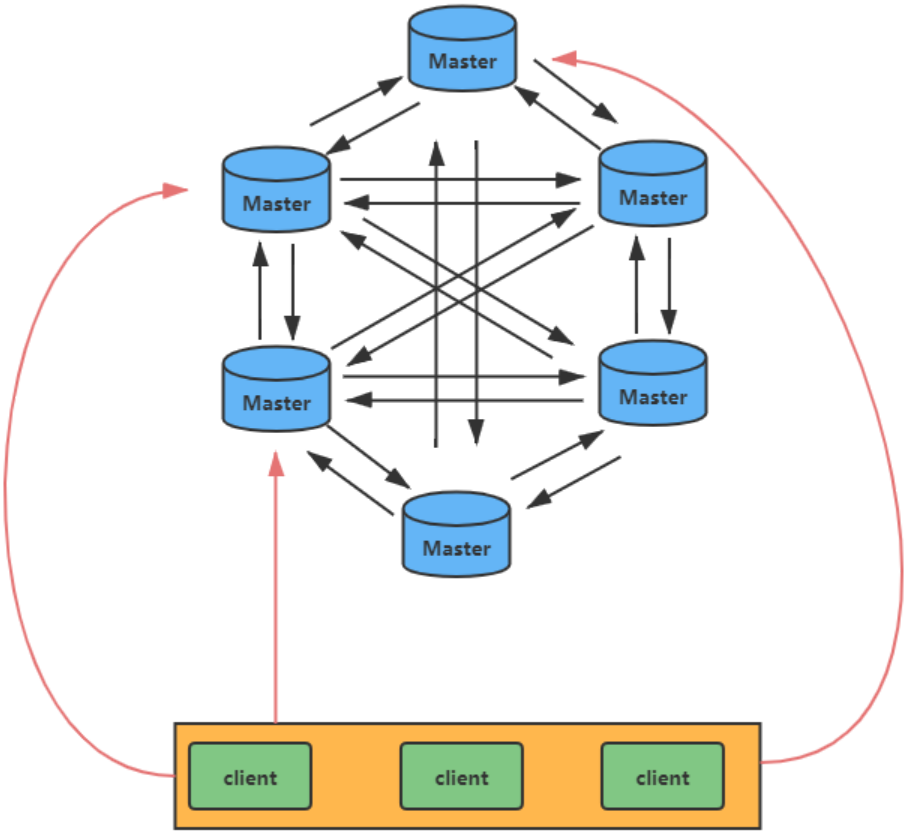
1 Redis集群简介

1.1 Redis集群（RedisCluster）

RedisCluster实现了对Redis的水平扩容，即启动N个Redis节点，将整个数据库分布存储在这N个节点当中，每个节点存储总数据的1/N。

RedisCluster通过分区（partition）来提供一定程度的可用性（availability）：即集群有一部分节点失效或者无法进行通讯，集群也可以继续处理命令。

Redis-Cluster采用无中心结构，每个节点保存数据和整个集群状态,每个节点都和其他所有节点连接，用来交换彼此的信息。



为了使得集群在一部分节点宕机或者无法与集群的大多数节点进行通讯的情况下， 仍然可以正常运作，Redis 集群对节点使用了主从复制功能。

2 Redis集群搭建

1. 删除持久化数据，aof、dump。
2. 创建6个实例，即在/usr/local/myredis文件夹下分别创建6379、6380、6381、6389、6390、6391的conf。
3. 在redis.conf中关闭appendonly。
4. 在不同的conf文件中，配置对应的内容
    - 可以通过include /usr/local/myredis/redis.conf将公共基础配置直接引入文件。

      将include /usr/local/myredis/redis.conf统一添加到这三个文件中
    - 在各个文件中，添加对应的pidfile、port、dbfilename、cluster-enabled（是否打开集群）、cluster-config-file（设定节点配置文件名）、cluster-node-timeout （设置节点失联时间，超过该时间（毫秒），集群自动进行主从切换）

      如：

      redis6379.conf中添加

      pidfile /var/run/redis_6379.pid

      port 6379

      dbfilename dump6379.rdb

      cluster-enabled yes

      cluster-config-file nodes-6379.conf

      cluster-node-timeout 15000

```
include /usr/local/myredis/redis.conf
pidfile "/var/run/redis_6379.pid"
port 6379
dbfilename "dump6379.rdb"
cluster-enabled yes
cluster-config-file nodes-6379.conf
cluster-node-timeout 15000
~
```

将其他文件按照相同规则创建出来。

5. 启动该6个redis，并确保是否全部生成nodes-xxxx.conf文件。（启动之前务必保证redis服务支持远程连接）

```
[root@localhost bin]# ./redis-server ../myredis/redis6379.conf
[root@localhost bin]# ./redis-server ../myredis/redis6380.conf
[root@localhost bin]# ./redis-server ../myredis/redis6381.conf
[root@localhost bin]# ./redis-server ../myredis/redis6389.conf
[root@localhost bin]# ./redis-server ../myredis/redis6390.conf
[root@localhost bin]# ./redis-server ../myredis/redis6391.conf
[root@localhost bin]# ps -ef |  grep redis
root       15163       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6379 [cluster]
root       15169       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6380 [cluster]
root       15175       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6381 [cluster]
root       15181       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6389 [cluster]
root       15187       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6390 [cluster]
root       15201       1  0 22:47 ?        00:00:00 ./redis-server 127.0.0.1:6391 [cluster]
root       15207    4986  0 22:48 pts/1    00:00:00 grep --color=auto redis
[root@localhost bin]#
```

```
[root@localhost bin]# ll
总用量 24864
-rw-r--r--. 1 root root      114 7月  15 22:47 nodes-6379.conf
-rw-r--r--. 1 root root      114 7月  15 22:47 nodes-6380.conf
-rw-r--r--. 1 root root      114 7月  15 22:47 nodes-6381.conf
-rw-r--r--. 1 root root      114 7月  15 22:40 nodes-6389.conf
-rw-r--r--. 1 root root      114 7月  15 22:40 nodes-6390.conf
-rw-r--r--. 1 root root      114 7月  15 22:40 nodes-6391.conf
-rwxr-xr-x. 1 root root  6549080 6月  28 04:55 redis-benchmark
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-aof -> redis-server
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-check-rdb -> redis-server
-rwxr-xr-x. 1 root root  6763376 6月  28 04:55 redis-cli
lrwxrwxrwx. 1 root root       12 6月  28 04:55 redis-sentinel -> redis-server
-rwxr-xr-x. 1 root root 12118288 6月  28 04:55 redis-server
```

6. 进入redis安装目录下的src文件夹

```
[root@localhost src]# pwd
/download/redis-6.2.4/src
[root@localhost src]#
```

7. 在该文件夹下执行命令

```
redis-cli --cluster create --cluster-replicas 1 192.168.56.31:6379
192.168.56.31:6380 192.168.56.31:6381 192.168.56.31:6389 192.168.56.31:6390
192.168.56.31:6391
```

此处使用真实ip地址，-replicas 1代表采用最简单的方式配置集群，一台主机，一台从机。

replicas表示每个master需要有几个slave。

```
[root@localhost src]# redis-cli --cluster create --cluster-replicas 1 192.168.56.31:6379 192.168.56.31:6380
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica 192.168.56.31:6390 to 192.168.56.31:6379
Adding replica 192.168.56.31:6391 to 192.168.56.31:6380
Adding replica 192.168.56.31:6389 to 192.168.56.31:6381
>>> Trying to optimize slaves allocation for anti-affinity
[WARNING] Some slaves are in the same host as their master
M: 986a9e0c374bfef0d3d826b9b33c3dde0e12133a 192.168.56.31:6379
   slots:[0-5460] (5461 slots) master
M: 2c7a3ddd9b921862119600d982615f417b4a7a0b 192.168.56.31:6380
   slots:[5461-10922] (5462 slots) master
M: 8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 192.168.56.31:6381
   slots:[10923-16383] (5461 slots) master
S: b2022abc7cf5b6ed50f8f5297bfdab2cb6e24f9e 192.168.56.31:6389
   replicates 8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13
S: ffc82180bddef727b83cb193995d515188f47c11 192.168.56.31:6390
   replicates 986a9e0c374bfef0d3d826b9b33c3dde0e12133a
S: 52033b4a26de0ad39a82889a1594e5804324abe1 192.168.56.31:6391
   replicates 2c7a3ddd9b921862119600d982615f417b4a7a0b
Can I set the above configuration? (type 'yes' to accept): yes
```

执行命令后，redis提供推荐的主从配置建议，执行同意。

```
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

集群创建成功。

8. 通过./redis-cli -c -p 6379可连接至集群（由于所有节点相通，任意端口号均可）

   在redis客户端中执行cluster nodes查看节点状态。

```
[root@localhost bin]# ./redis-cli -c -p 6379
127.0.0.1:6379> cluster nodes
8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 192.168.56.31:6381@16381 master - 0 1626405106000 3 connected 10923-16383
2c7a3ddd9b921862119600d982615f417b4a7a0b 192.168.56.31:6380@16380 master - 0 1626405106961 2 connected 5461-10922
52033b4a26de0ad39a82889a1594e5804324abe1 192.168.56.31:6391@16391 slave 2c7a3ddd9b921862119600d982615f417b4a7a0b 0 1626405106000 2 connected
986a9e0c374bfef0d3d826b9b33c3dde0e12133a 192.168.56.31:6379@16379 myself,master - 0 1626405105000 1 connected 0-5460
ffc82180bddef727b83cb193995d515188f47c11 192.168.56.31:6390@16390 slave 986a9e0c374bfef0d3d826b9b33c3dde0e12133a 0 1626405104000 1 connected
b2022abc7cf5b6ed50f8f5297bfdab2cb6e24f9e 192.168.56.31:6389@16389 slave 8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 0 1626405105954 3 connected
127.0.0.1:6379>
```

3 Redis集群操作

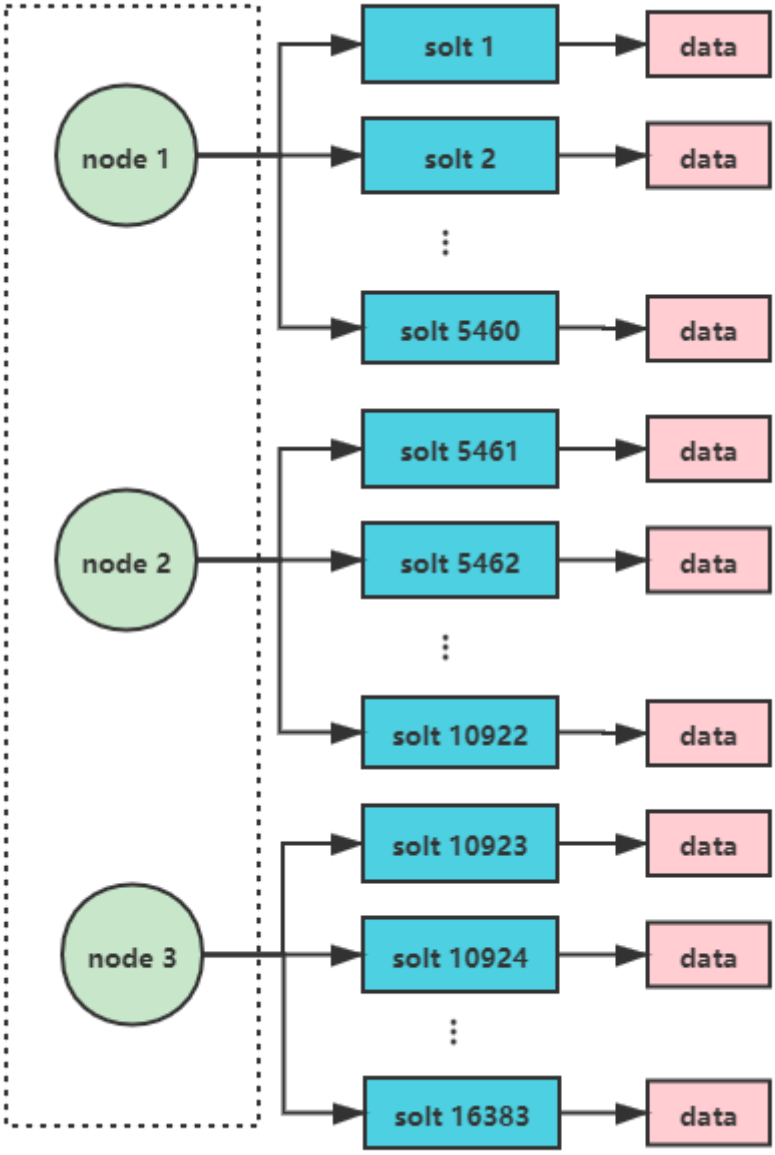
3.1 Slot

一个 Redis 集群包含 16384 个哈希槽（hash slot），每个键都属于这 16384 个哈希槽的其中一个。

集群使用公式 CRC16(key) % 16384 来计算键 key 属于哪个槽， 其中 CRC16(key) 语句用于计算键 key 的 CRC16 校验和 。

集群中的每个节点负责处理一部分哈希槽。 举个例子， 一个集群可以有三个节点， 其中：

- 节点 1 负责处理 0 号至 5500 号哈希槽。
- 节点 2 负责处理 5501 号至 11000 号哈希槽。
- 节点 3 负责处理 11001 号至 16384 号哈希槽。



3.2 命令执行

```
set k1 v1
```

```
127.0.0.1:6379> set k1 v1
-> Redirected to slot [12706] located at 192.168.56.31:6381
OK
192.168.56.31:6381>
```

根据k1计算出的槽值进行切换节点，并存入数据。

不在一个slot下的键值，是不能使用mget、mset等多建操作。

```
192.168.56.31:6381> mset k1 v1 k2 v2 k3 v3
(error) CROSSSLOT Keys in request don't hash to the same slot
192.168.56.31:6381>
```

可以通过{}来定义组的概念，从而是key中{}内相同内容的键值对放到同一个slot中

```
mset k1{test} v1 k2{test} v2 k3{test} v3
```

```
192.168.56.31:6381> mset k1{test} v1 k2{test} v2 k3{test} v3
-> Redirected to slot [6918] located at 192.168.56.31:6380
OK
192.168.56.31:6380>
```

3.3 故障恢复

1. 关闭6379服务器，进入redis-cli执行cluster nodes

```
127.0.0.1:6380> cluster nodes
ffc82180bddef727b83cb193995d515188f47c11 192.168.56.31:6390@16390 master - 0 1626407518000 7 connected 0-5460
b2022abc7cf5b6ed50f8f5297bfdab2cb6e24f9e 192.168.56.31:6389@16389 slave 8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 0 1626407516000 3 connec
ed
52033b4a26de0ad39a82889a1594e5804324abe1 192.168.56.31:6391@16391 slave 2c7a3ddd9b921862119600d982615f417b4a7a0b 0 1626407518875 2 connec
ed
8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 192.168.56.31:6381@16381 master - 0 1626407517866 3 connected 10923-16383
2c7a3ddd9b921862119600d982615f417b4a7a0b 192.168.56.31:6380@16380 myself,master - 0 1626407516000 2 connected 5461-10922
986a9e0c374bfef0d3d826b9b33c3dde0e12133a 192.168.56.31:6379@16379 master fail - 1626407473591 1626407466000 1 disconnected
```

   6379服务器fail，6390成为了新的master

2. 重新启动6379服务器，再次查看，6379成为了Slave

```
127.0.0.1:6380> cluster nodes
ffc82180bddef727b83cb193995d515188f47c11 192.168.56.31:6390@16390 master - 0 1626407647988 7 connected 0-5460
b2022abc7cf5b6ed50f8f5297bfdab2cb6e24f9e 192.168.56.31:6389@16389 slave 8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 0 1626407649000 3 connect
ed
52033b4a26de0ad39a82889a1594e5804324abe1 192.168.56.31:6391@16391 slave 2c7a3ddd9b921862119600d982615f417b4a7a0b 0 1626407648000 2 connect
ed
8e8aa41f0dd2b66c8a2348c377b0a751e8d53c13 192.168.56.31:6381@16381 master - 0 1626407648995 3 connected 10923-16383
2c7a3ddd9b921862119600d982615f417b4a7a0b 192.168.56.31:6380@16380 myself,master - 0 1626407648000 2 connected 5461-10922
986a9e0c374bfef0d3d826b9b33c3dde0e12133a 192.168.56.31:6379@16379 slave ffc82180bddef727b83cb193995d515188f47c11 0 1626407650002 7 connect
ed
```

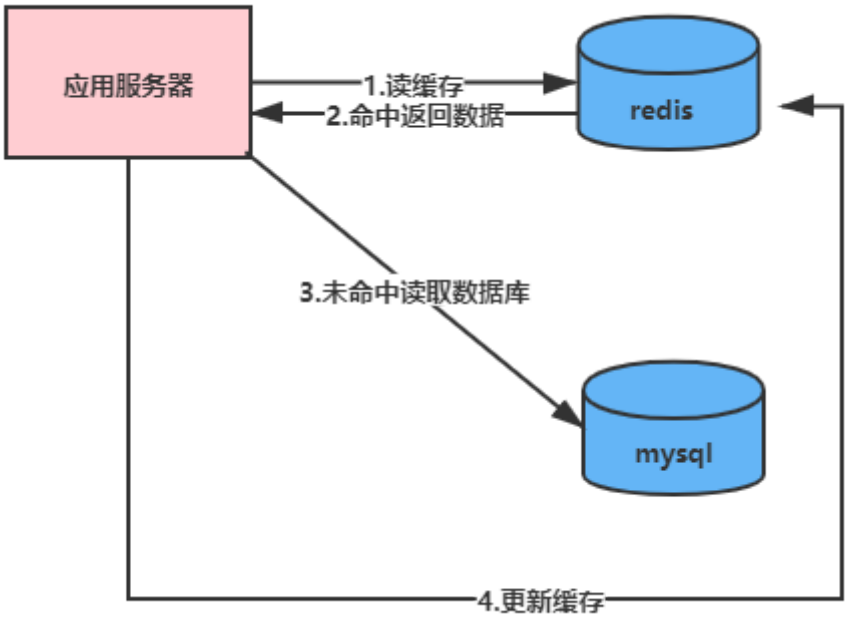
七、JAVA与Redis

1 Jedis操作Redis

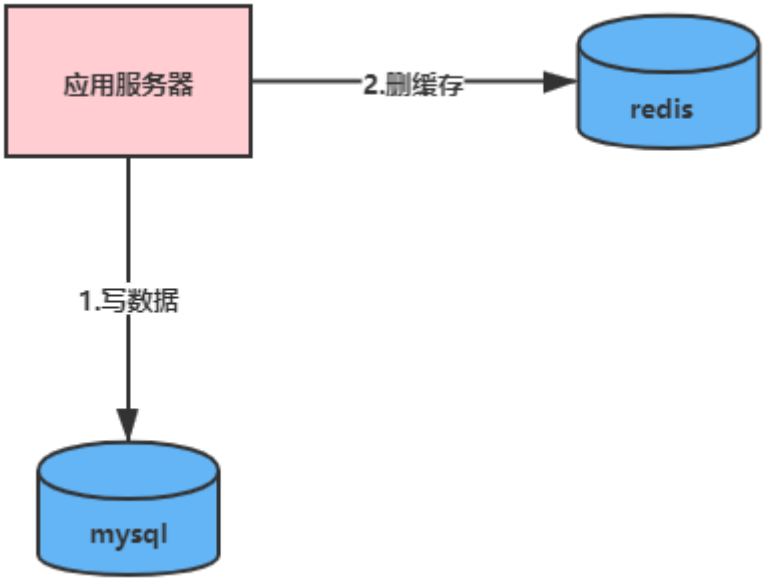
1.1 Cache Aside Pattern（缓存模式）

读：

1. 先读缓存，再读数据库
2. 如果缓存命中，则直接返回缓存数据
3. 如果缓存未命中，则访问数据库，并将数据重置回缓存，然后返回。



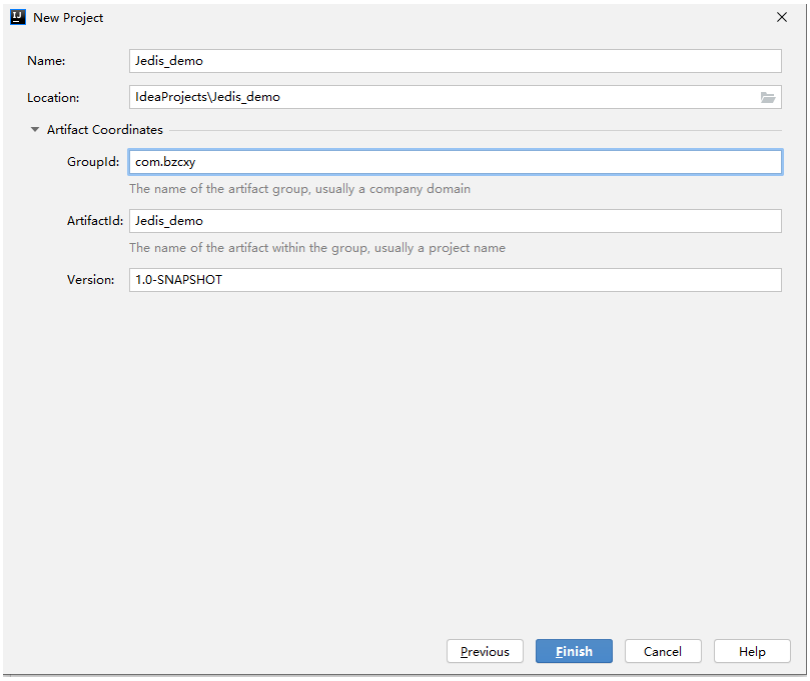
写：
先操作数据库写，再淘汰缓存（这里淘汰缓存是删除，而不是更新）



Jedis是Redis官方推荐的Java连接开发工具。要在Java开发中使用好Redis中间件，必须对Jedis熟悉才能写成漂亮的代码。

1.2 引入Jedis

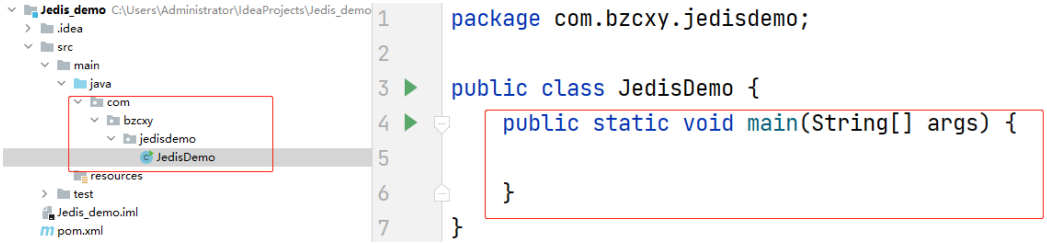
1. 启动redis并在IDEA中创建一个maven工程



2. 引入maven依赖

```
<dependency>
    <groupId>redis.clients</groupId>
    <artifactId>jedis</artifactId>
    <version>3.6.0</version>
</dependency>
```

3. 创建相关文件

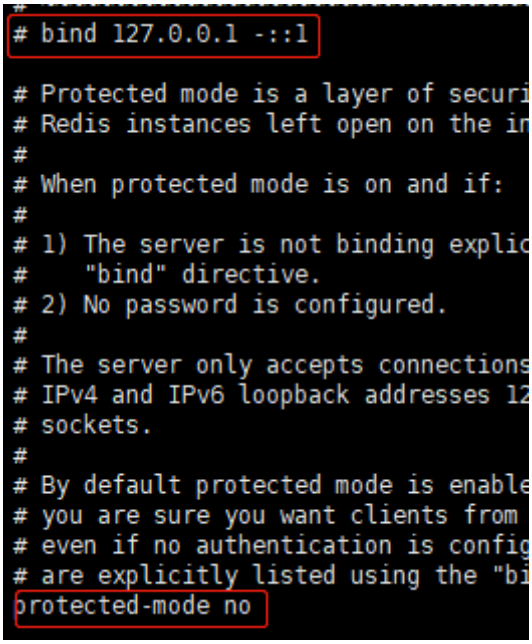


1.3 常用方法

1. Jedis连接到redis

```
Jedis jedis = new Jedis("192.168.56.31",6379); //第一个参数是ip地址，第二个参数是端口
```

- 在连接之前，需要开放redis连接服务，即关闭redis.conf中的bind本机和关闭本机保护模式



- 通过systemctl stop firewalld.service命令关闭CentOS8防火墙

```
Jedis jedis = new Jedis("192.168.56.31",6379);
String msg = jedis.ping();//通过ping()方法向redis发送一个ping命令，服务器返回一个Pong
System.out.println(msg);
jedis.close();//jedis使用完毕需要关闭
```

2. String

```
jedis.set("k1","v1"); //设置一个key
jedis.set("k2","1");//设置一个key
String res = jedis.get("k1");//获取一个key
Long ires =  jedis.incr("k2");//对某一个key自增
```

3. Keys

```
 Set<String> keys = jedis.keys("*");//返回所有的key
 Long time = jedis.ttl("k1");//返回该key剩余过期时间
```

4. List

```
jedis.lpush("list1","v1","v2","v3");//向list中添加数据
List<String> list = jedis.lrange("list1",0,-1 );//返回list全部数据
```

5. Set

```
jedis.sadd("set1" ,"v1","v2","v2","v3");//向set中添加数据
jedis.smembers("set1");//查看该集合中有多少个元素
```

6. Hash

```
jedis.hset("user","age","25");//设置一个hash
jedis.hvals("user");//获取该key的所有value
```

7. Zset

```
jedis.zadd("zset1",100,"java");//向zset中添加一条数据
jedis.zrange("zset1",0,-1);//获取所有的值
```

8. Bitmaps

```
jedis.setbit("b1",0, "1");//将b1偏移量为0的位设置为1
jedis.getbit("b1",0);//获取b1偏移量为0的位
```

9. Geospatia

```
jedis.geoadd("chinacity",130,110,"beijing");//添加一条地理信息数据
```

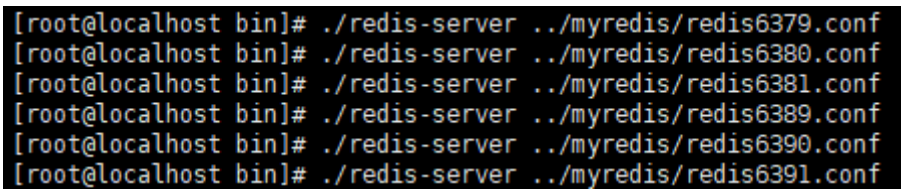
10. Hyperloglog

```
jedis.pfadd("book","c++","java","php");//将所有元素参数添加到 Hyperloglog 数据结构中。
```

其实jedis中的方法基本同redis命令一致。

2 JedisCluster操作Redis集群

1. 启动redis集群

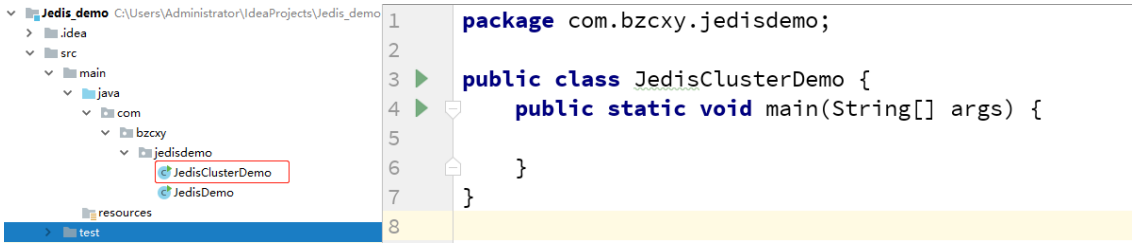


```
[root@localhost bin]# ./redis-server ../myredis/redis6379.conf
[root@localhost bin]# ./redis-server ../myredis/redis6380.conf
[root@localhost bin]# ./redis-server ../myredis/redis6381.conf
[root@localhost bin]# ./redis-server ../myredis/redis6389.conf
[root@localhost bin]# ./redis-server ../myredis/redis6390.conf
[root@localhost bin]# ./redis-server ../myredis/redis6391.conf
```

进入redis安装目录下的src文件夹执行

```
redis-cli --cluster create --cluster-replicas 1 192.168.56.31:6379
192.168.56.31:6380 192.168.56.31:6381 192.168.56.31:6389 192.168.56.31:6390
192.168.56.31:6391
```

2. 在IDEA中新建JedisClusterDemo类



3. JedisCluster连接到redis集群

```
//HostAndPort具体ip和端口只要是集群中的就可
HostAndPort hp = new HostAndPort("192.168.56.31",6380);
//需要用HostAndPort对象作为参数，连接到集群
JedisCluster jedisCluster = new JedisCluster(hp);
```

4. JedisCluster操作集群

```
jedisCluster.set("b1","v1");//向节点中设置一个值
String val = jedisCluster.get("b1");//从节点中获取一个值
```

3 SpringData整合Redis

1. 通过IDEA初始化一个Springboot工程，并添加SpringDataRedis组件

   pom.xml中有

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-pool2</artifactId>
    <version>2.6.0</version>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-core</artifactId>
    <version>2.12.3</version>
</dependency>
<dependency>
     <groupId>com.fasterxml.jackson.core</groupId>
     <artifactId>jackson-databind</artifactId>
     <version>2.12.3</version>
</dependency>
<dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-test</artifactId>
     <scope>test</scope>
</dependency>
```

2. 在application.properties中配置

```
#Redis服务器连接地址
spring.redis.host=192.168.56.31
#Redis服务器连接端口
```

```
spring.redis.port=6379
#Redis服务器连接密码（默认为空）
spring.redis.password=
#连接池最大连接数（使用负值表示没有限制）
spring.redis.pool.max-active=8
#连接池最大阻塞等待时间（使用负值表示没有限制）
spring.redis.pool.max-wait=-1
#连接池中的最大空闲连接
spring.redis.pool.max-idle=8
#连接池中的最小空闲连接
spring.redis.pool.min-idle=0
#连接超时时间（毫秒）
spring.redis.timeout=30000
```

3. 创建配置类

   常见序列化器：

   - StringRedisSerializer: 简单的字符串序列化
   - GenericJackson2JsonRedisSerializer: 可以将任何对象泛化为字符串并序列化（对象必须有无参构造方法）
   - JdkSerializationRedisSerializer: 序列化java对象（需要序列化的类必须实现Serializable接口）
   - Jackson2JsonRedisSerializer：序列化对象（序列化带泛型的数据时，会以map的结构进行存储，反序列化是不能将map解析成对象）

```
@Configuration
public class RedisConfig{
    @Bean
    public RedisTemplate<String, Serializable>
redisTemplate(LettuceConnectionFactory connectionFactory)
    {
        RedisTemplate<String, Serializable> redisTemplate = new
RedisTemplate<>();
        //给key设置StringRedisSerializer序列化器
        redisTemplate.setKeySerializer(new StringRedisSerializer());
        //给value设置GenericJackson2JsonRedisSerializer序列化器
        redisTemplate.setValueSerializer(new
GenericJackson2JsonRedisSerializer());
        redisTemplate.setConnectionFactory(connectionFactory);
        return redisTemplate;
    }
}
```

4. 启动Redis单机版
5. 编写unit

```
public class User {
    private Integer id;
    private String name;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
```

```
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public User(Integer id, String name) {
        this.id = id;
        this.name = name;
    }

    public User(){}
}
```

```
    @Autowired
    private RedisTemplate redisTemplate;
    @Test
    void contextLoads() {
        User u = new User(15,"百战");
        redisTemplate.opsForValue().set("user",u);
        User u1 = (User) redisTemplate.opsForValue().get("user");
        System.out.println(u1.getName());
    }
```

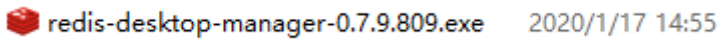
八、Redis企业级解决方案

1 RedisDesktopManager

一款好用的Redis桌面管理工具，支持命令控制台操作，以及常用，查询key，rename，delete等操作。

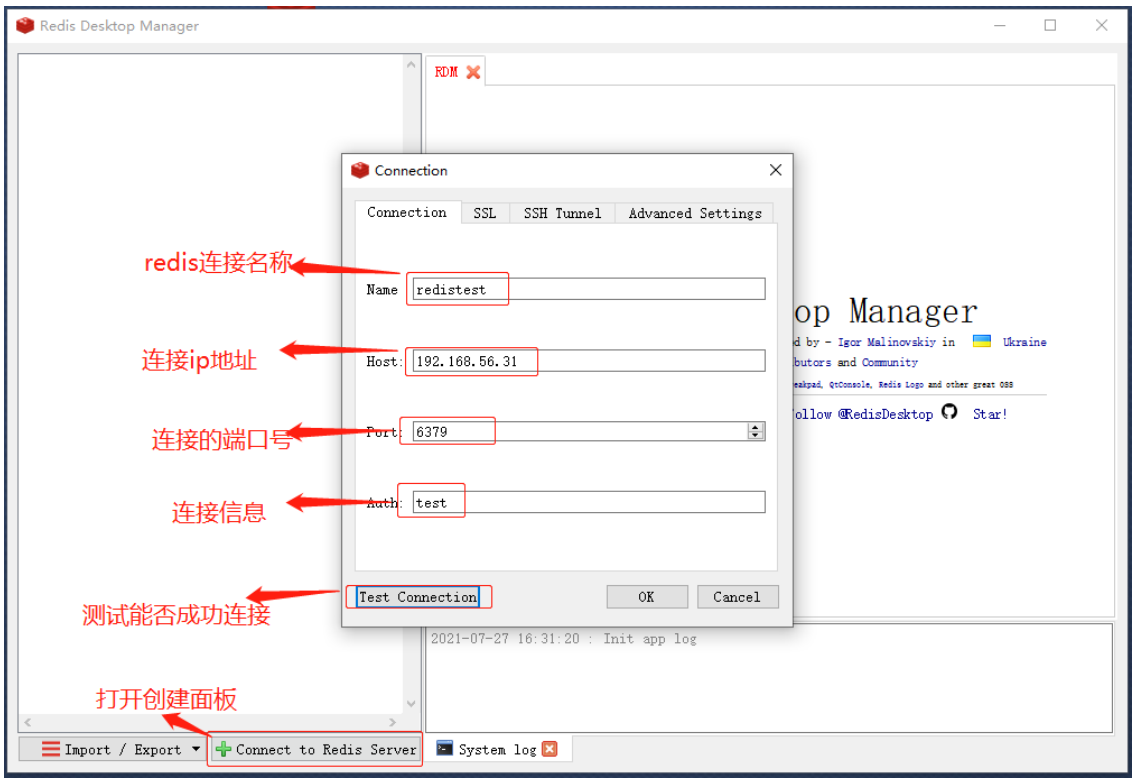
RedisDesktopManager不支持集群操作。

1. 傻瓜式安装该工具

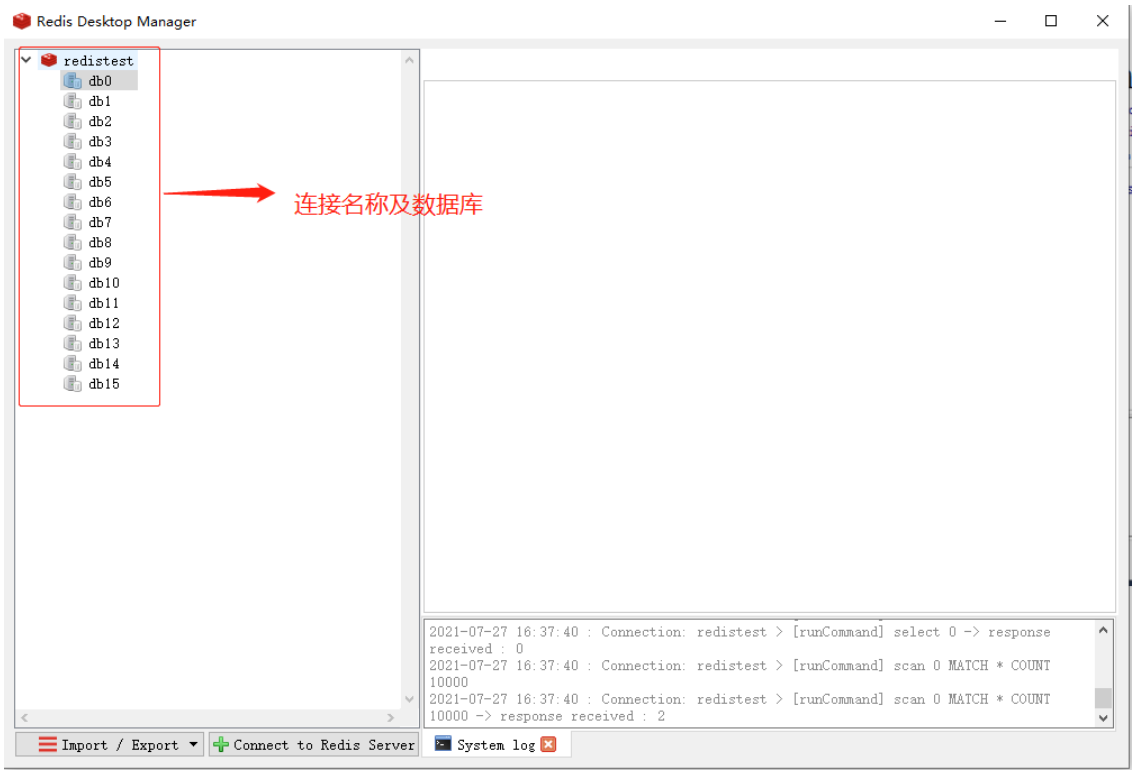


2. 安装成功后，启动该工具

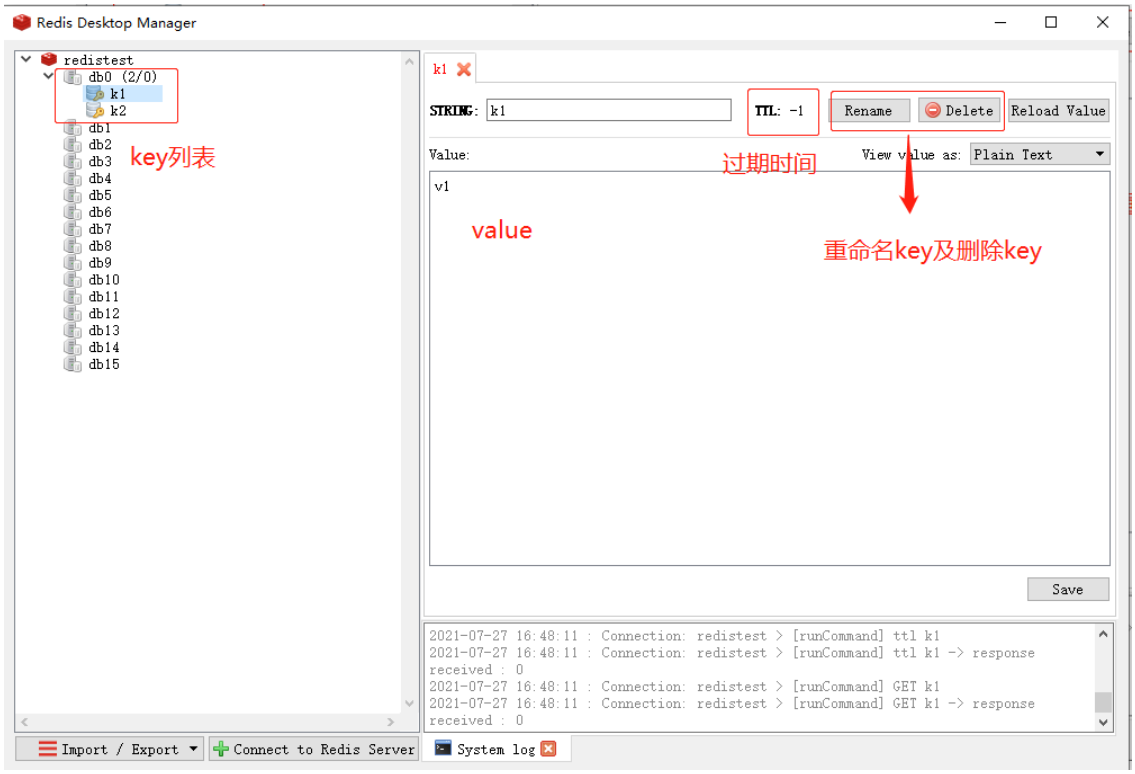
3. 启动后，创建新的连接



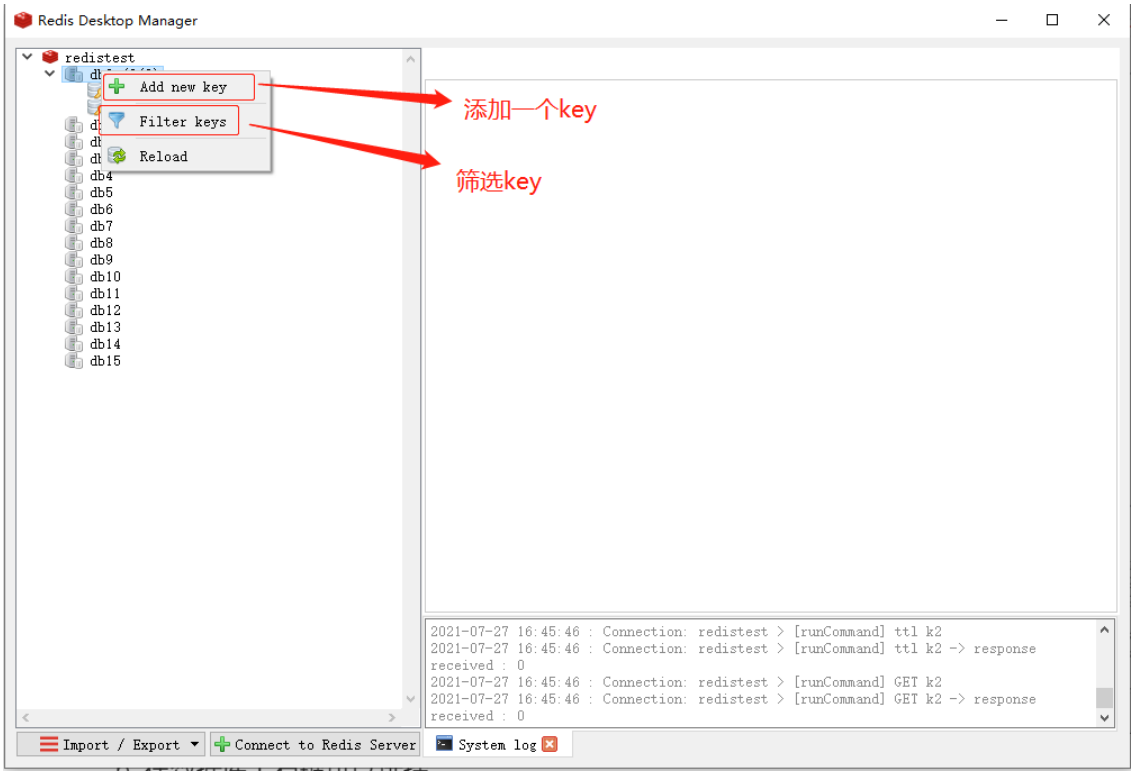
4. 创建连接成功

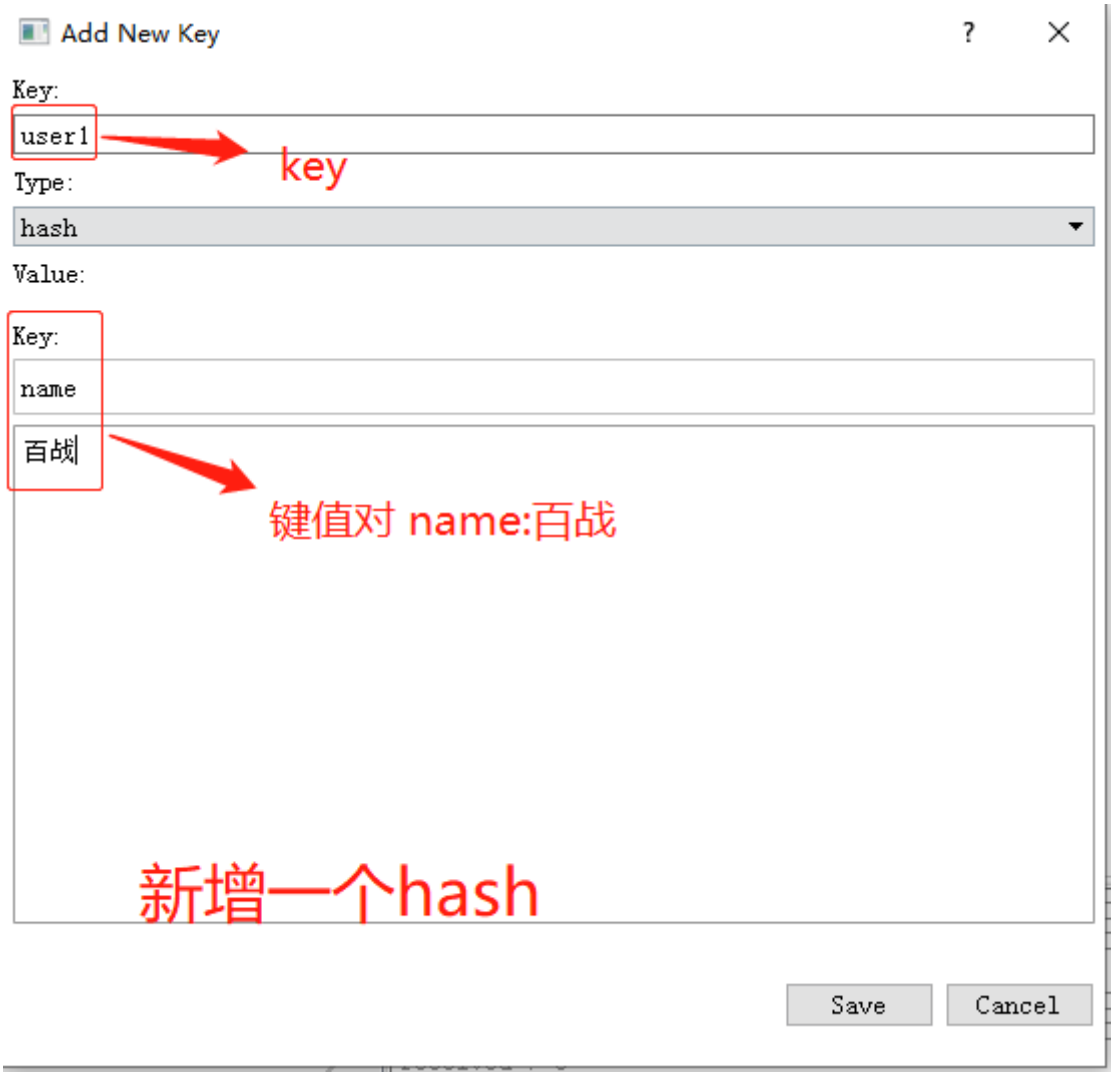


5. 双击进入数据库，可以查看所有的key及value、过期时间，同时可以修改key或删除key

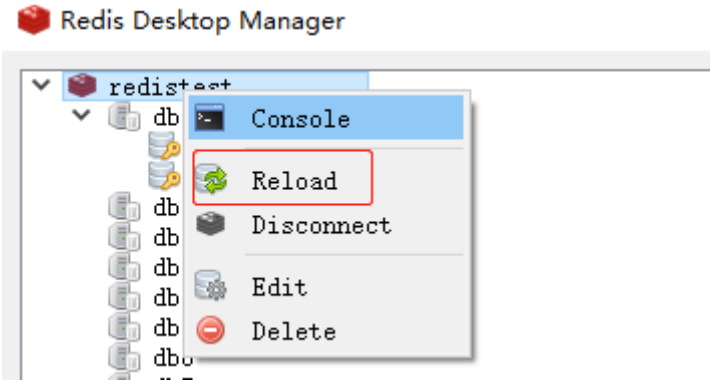


6. 在数据库上右键可以选择新增key或筛选key





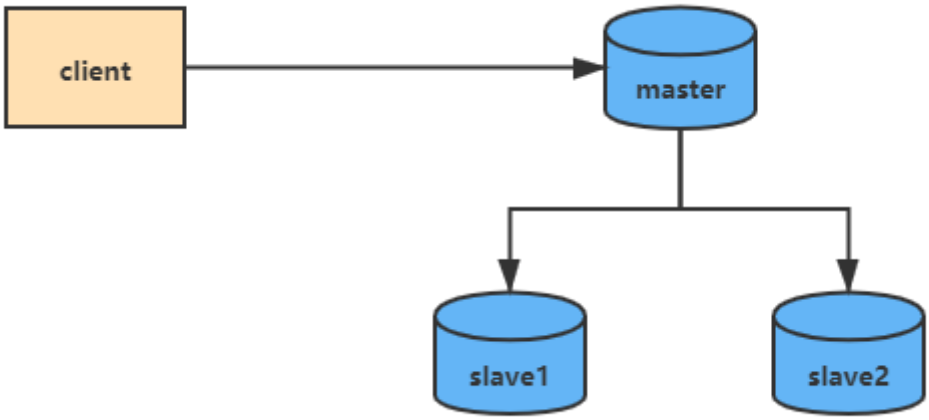
注：对key更改后(新增、修改、删除)，需要通过右键**连接名**->reload重新加载



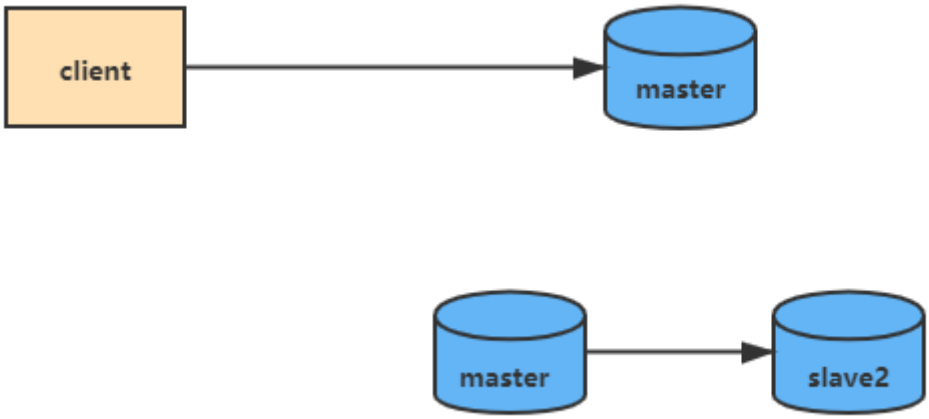
2 Redis脑裂

2.1 概念

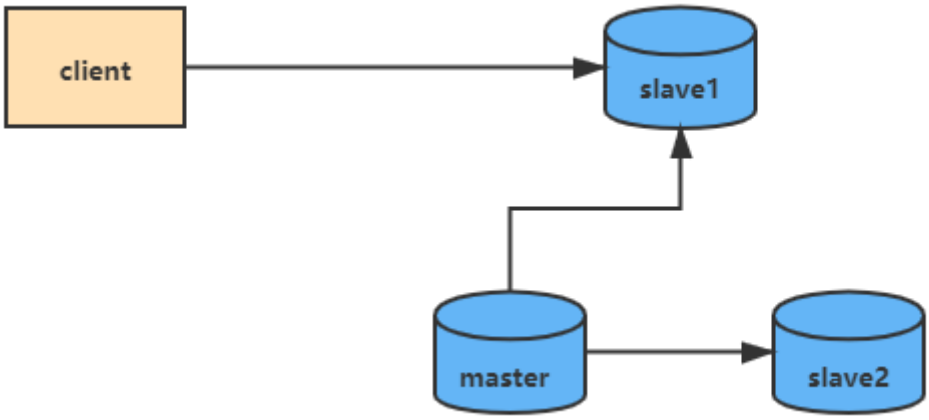
1. 假设现在有三台机器，分别安装了redis服务，结构如图



2. 如果此时master服务器所在区域网络通信出现异常，导致和两台slave机器无法正常通信，但是和客户端的连接是正常的。那么sentinel就会从两台slave机器中选举其中一个作为新的master来处理客户端请求。



3. 这个时候，已经存在两台master服务器，client发送的数据会持续保存在旧的master服务器中，而新的master和slave中没有新的数据。如果一分钟以后，网络恢复正常，服务之间能够正常通信。此时，sentinel会把旧的master会变成新的master的slave节点。



4. 问题出现了，我们都知道，slave会从master中同步数据，保持主从数据一致。这个时候，变成了slave节点的旧master会丢失掉通信异常期间从客户端接收到的数据。

2.2 解决方案

redis.conf配置参数：

```
min-replicas-to-write 1
min-replicas-max-lag 10
```

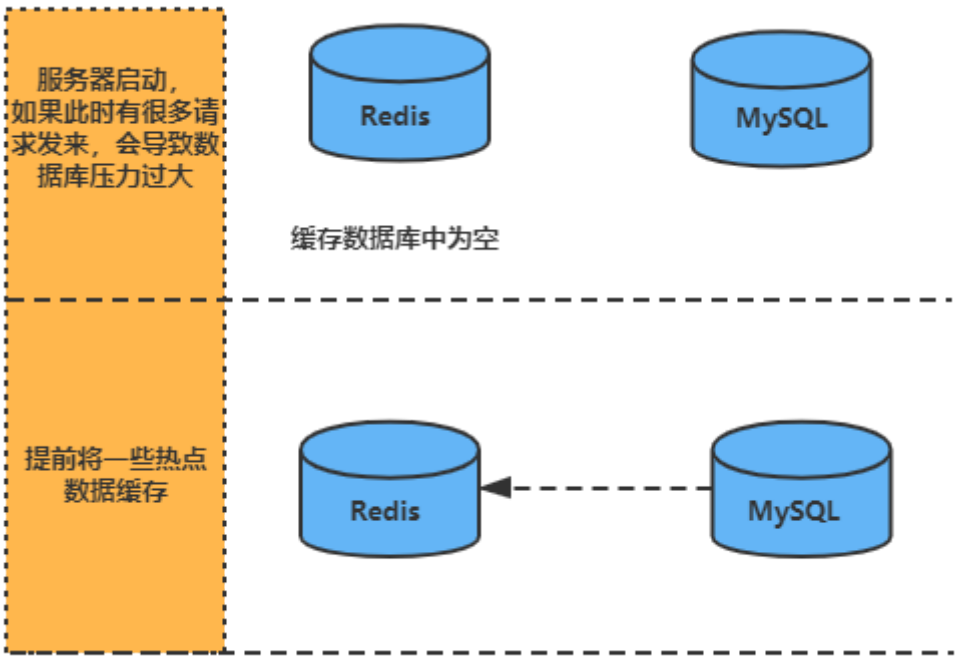
第一个参数表示最少的slave节点为1个

第二个参数表示数据复制和同步的延迟不能超过10秒

配置了这两个参数：如果发生脑裂：原master会在客户端写入操作的时候拒绝请求。这样可以避免大量数据丢失。

3 缓存预热

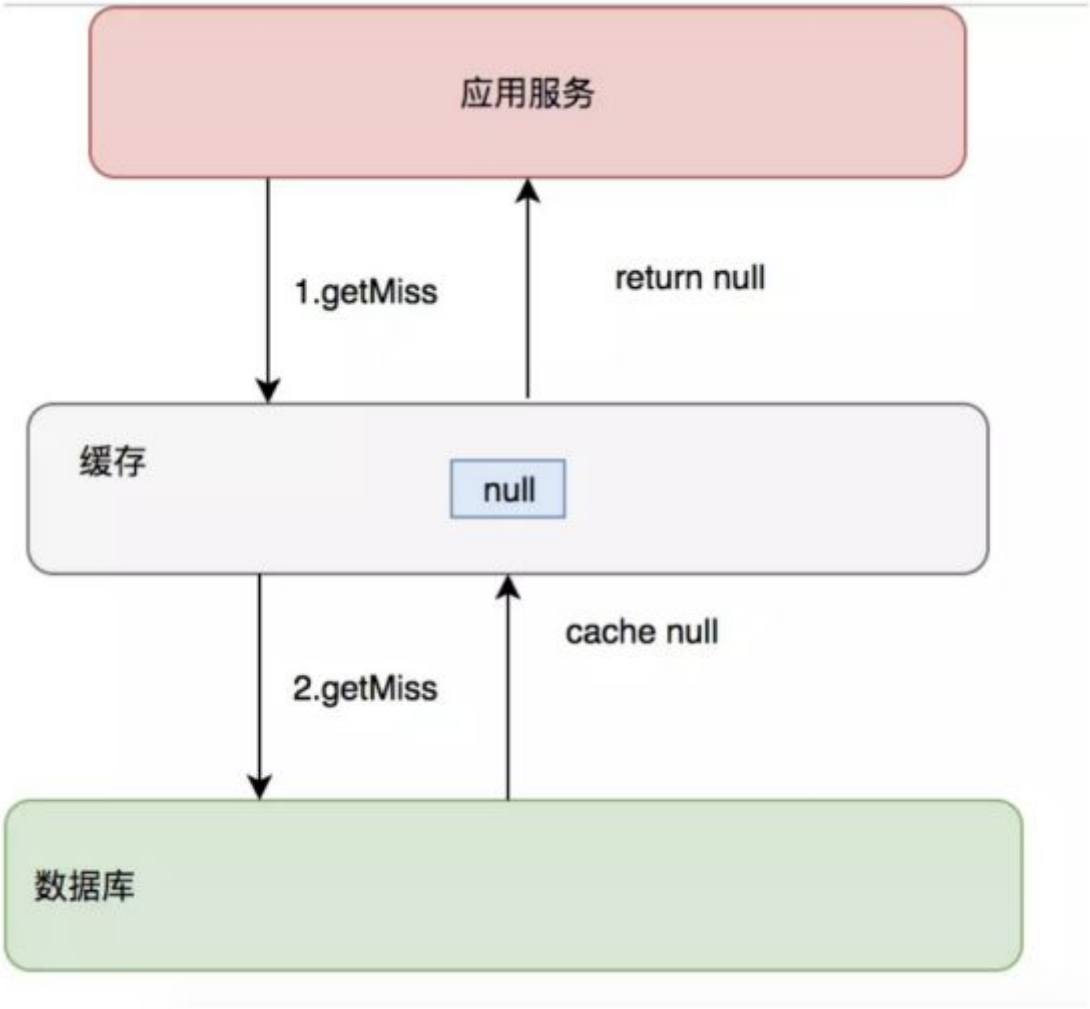
新启动的系统没有任何缓存数据，在缓存重建数据的过程中，系统性能和数据库负载都不太好，所以最好是在系统上线之前就把要缓存的热点数据加载到缓存中，这种缓存预加载手段就是缓存预热。



4 缓存穿透

4.1 概念

如果某个key对应的数据不存在，而又未对该key做缓存，所以每次请求都会穿过缓存直接到数据库进行查询，并发量高的情况下进而导致数据库直接宕机，这就是缓存穿透。



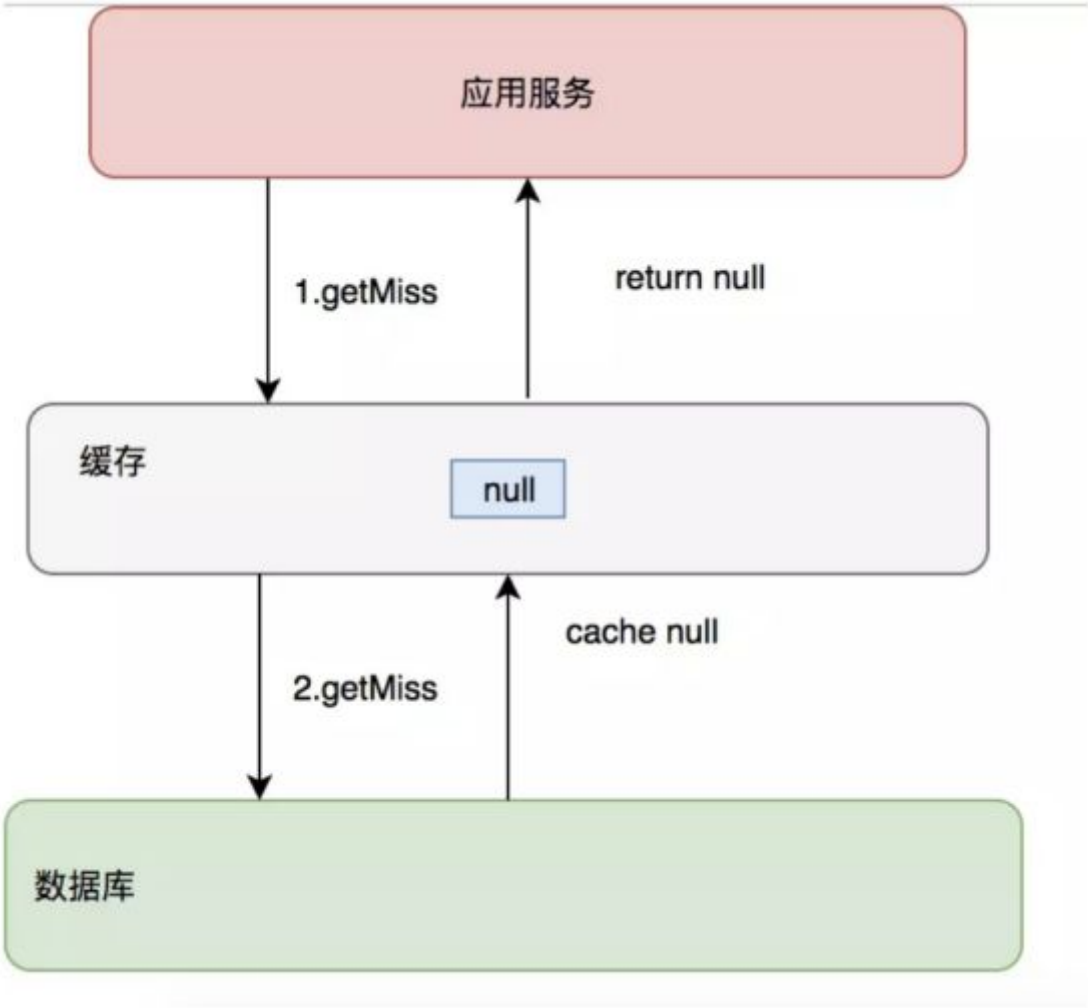
4.2 解决方案

1. 对空值缓存：如果一个查询返回的数据为空（不管数据是否存在），我们仍然把这个空结果缓存，设置空结果的过期时间会很短，最长不超过5分钟。
2. 设置白名单：使用bitmaps类型定义一个可以访问的名单，用户id作为偏移量，每次访问查询是否在白名单中，如果不存在，则拒绝访问。
3. 布隆过滤器：类似一个hash set，用来判断某个元素（key）是否在某个集合中。
   和一般的hash set不同的是，这个算法无需存储key的值，对于每个key，只需要k个比特位，每个存储一个标志，用来判断key是否在集合中。

5 缓存击穿

5.1 概念

某一个热点 key，在缓存过期的一瞬间，同时有大量的请求打进来，由于此时缓存过期了，所以请求最终都会走到数据库，造成瞬时数据库请求量大、压力骤增，甚至可能打垮数据库。



5.2 解决方案

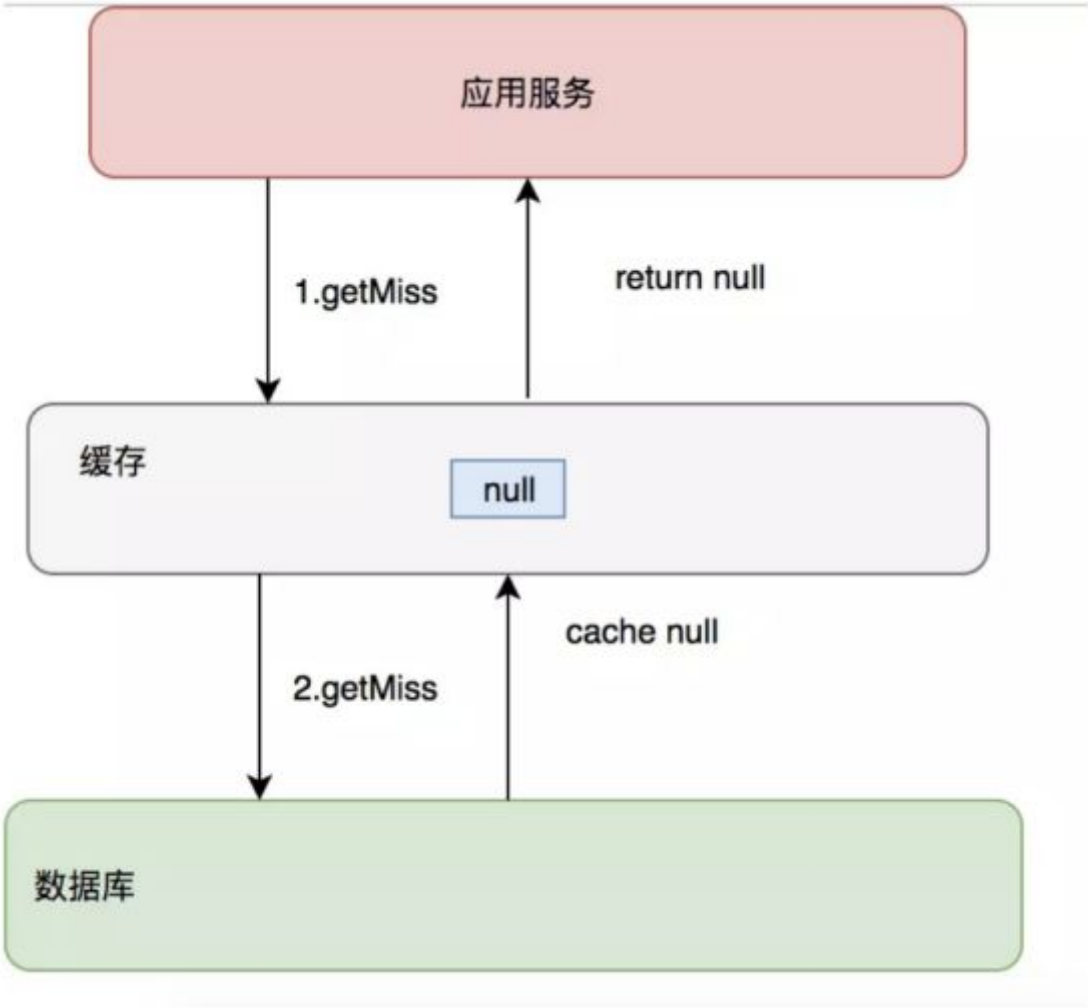
1. 加互斥锁：在并发的多个请求中，只有第一个请求线程能拿到锁并执行数据库查询操作，其他的线程拿不到锁就阻塞等着，等到第一个线程将数据写入缓存后，其他线程直接查询缓存。
2. 热点数据不过期：直接将缓存设置为不过期，然后由定时任务去异步加载数据，更新缓存。

6 缓存雪崩

6.1 概念

缓存雪崩是指在我们设置缓存时采用了相同的过期时间，导致缓存在某一时刻同时失效，请求全部转发到DB，DB瞬时压力过重雪崩。

和缓存击穿不同的是，缓存击穿指并发查同一条数据，缓存雪崩是不同数据都过期了，很多数据都查不到从而查数据库。



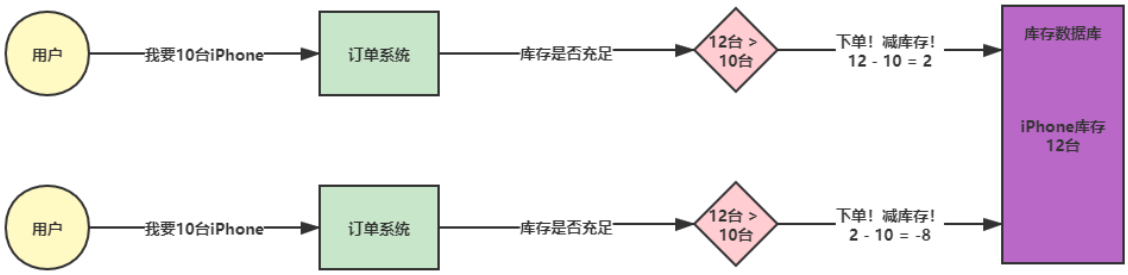
6.2 解决方案

1. 缓存数据的过期时间设置随机，防止同一时间大量数据过期现象发生。
2. 如果缓存数据库是分布式部署，将热点数据均匀分布在不同搞得缓存数据库中。
3. 设置热点数据永远不过期。

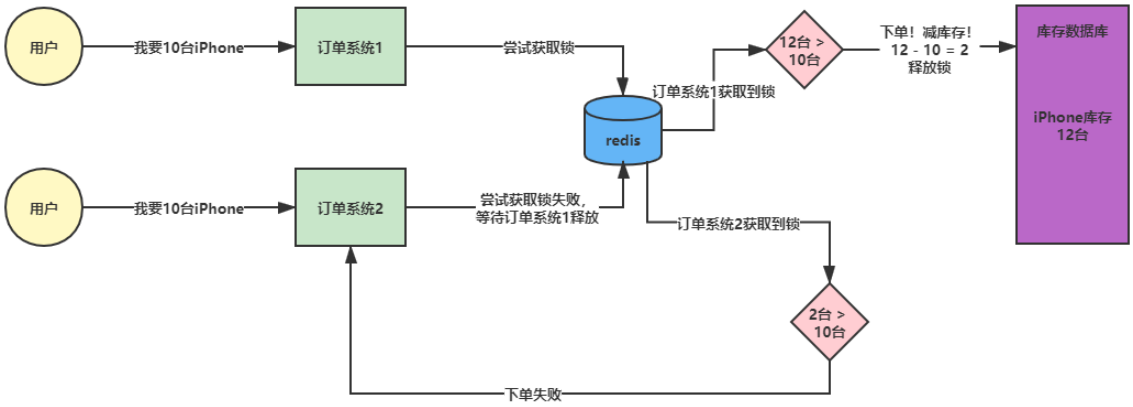
7 分布式锁

随着业务发展的需要，原单机部署的系统被演化成分布式集群系统，由于分布式系统多线程、多进程且分布在不同的机器上，这将使原单机部署情况下的并发控制锁策略失效，单纯的Java API并不能提供分布式锁的能力。为了解决这个问题需要一种跨JVM的互斥机制来控制共享资源的访问，这就是分布式锁要解决的问题！

- 假设订单系统部署两台机器上，不同的用户都要同时买10台iphone，分别发了一个请求给订单系统。
- 接着每个订单系统实例都去数据库里查了一下，当前iphone库存是12台。
- 12台库存大于了要买的10台数量啊！
- 于是每个订单系统实例都发送SQL到数据库里下单，然后扣减了10个库存，其中一个将库存从12台扣减为2台，另外一个将库存从2台扣减为-8台。



增加分布式锁



分布式锁主流的实现方案：

1. 基于数据库实现分布式锁
2. 基于缓存（Redis等）
3. 基于Zookeeper

每一种分布式锁解决方案都有各自的优缺点：

1. 性能：Redis最高
2. 可靠性：Zookeeper最高

7.1 设置锁和过期时间

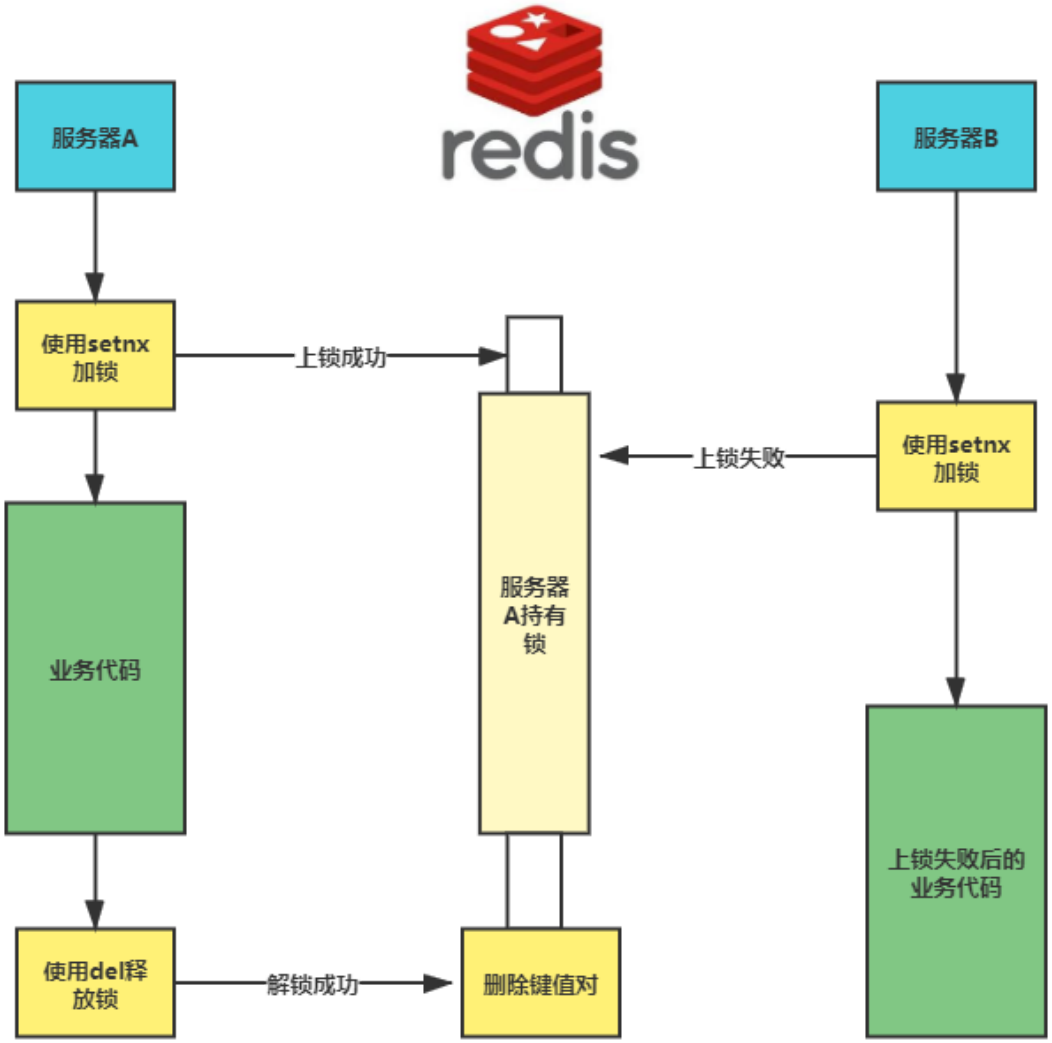
1. 通过setnx上锁

   由于setnx只有不存在该key的时候，可以设置成功，并返回1，否则设置失败，并返回0。

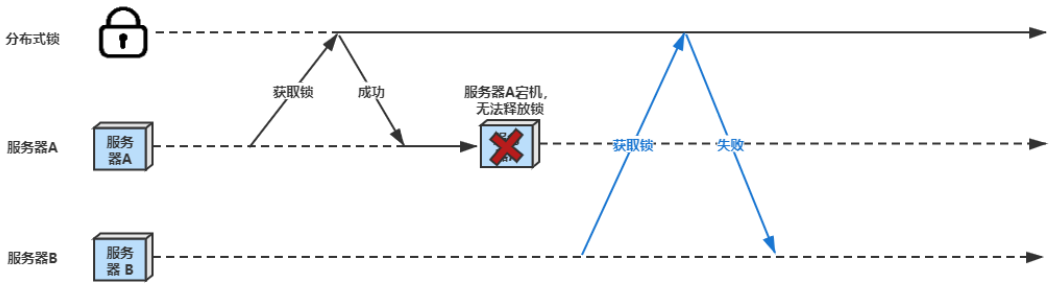
   ```
   setnx lock A //获取锁，并对lock上锁
   setnx lock B //其他服务器试图获取锁时，失败
   ```

2. 通过del释放锁

   ```
   del lock //释放锁，此时其他服务器可以获取锁
   ```



3. 如果锁一直不释放，需要增加过期时间，防止资源浪费。

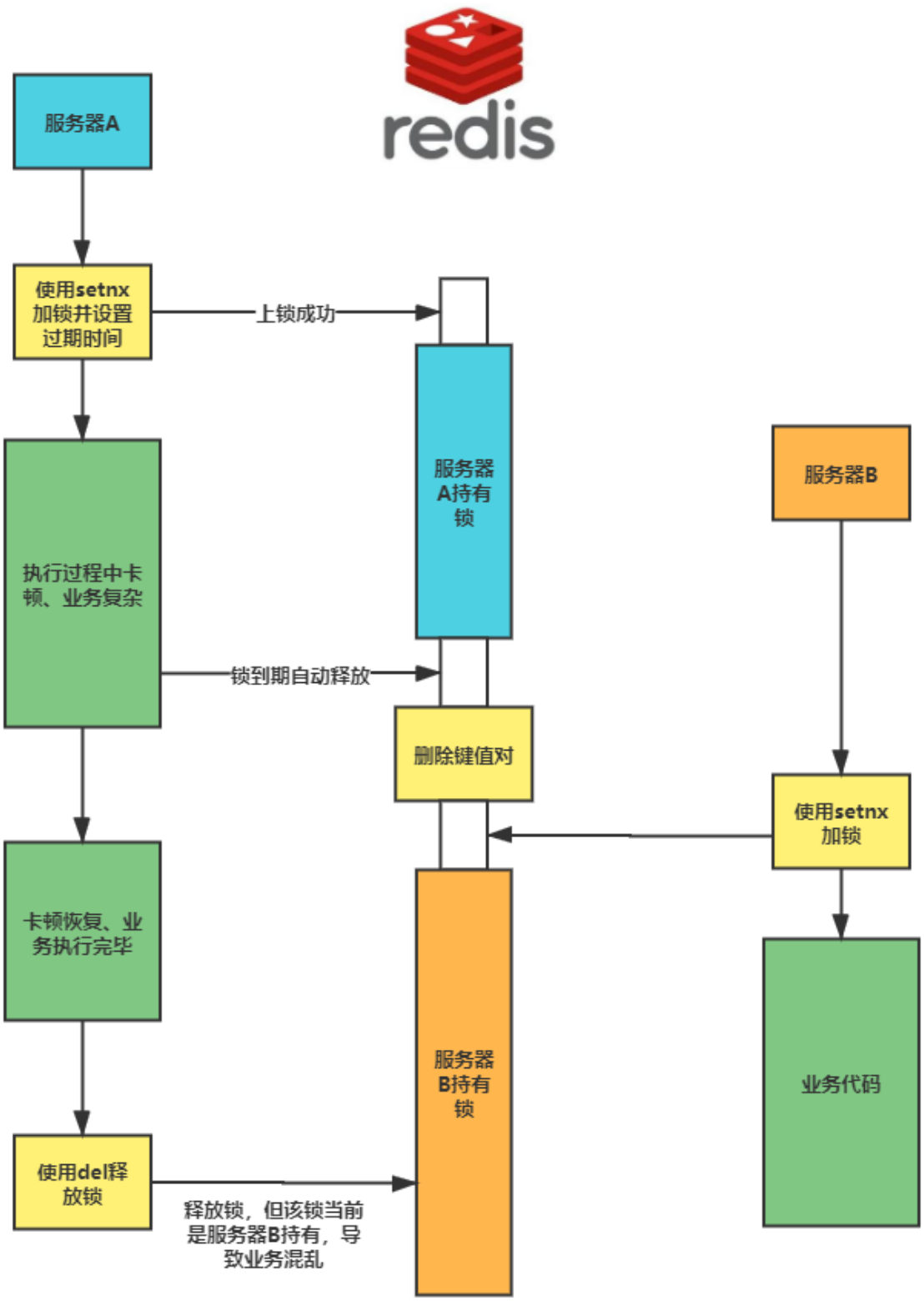



```
expire lock 10 //给锁添加个过期时间
```

4. 如果在上锁之后，设置过期时间之前，服务器异常，就无法设置过期时间，可以在上锁的同时设置过期时间。

```
set lock 1 nx ex 10 //上锁的同时设置过期时间
```

7.2 防止误删

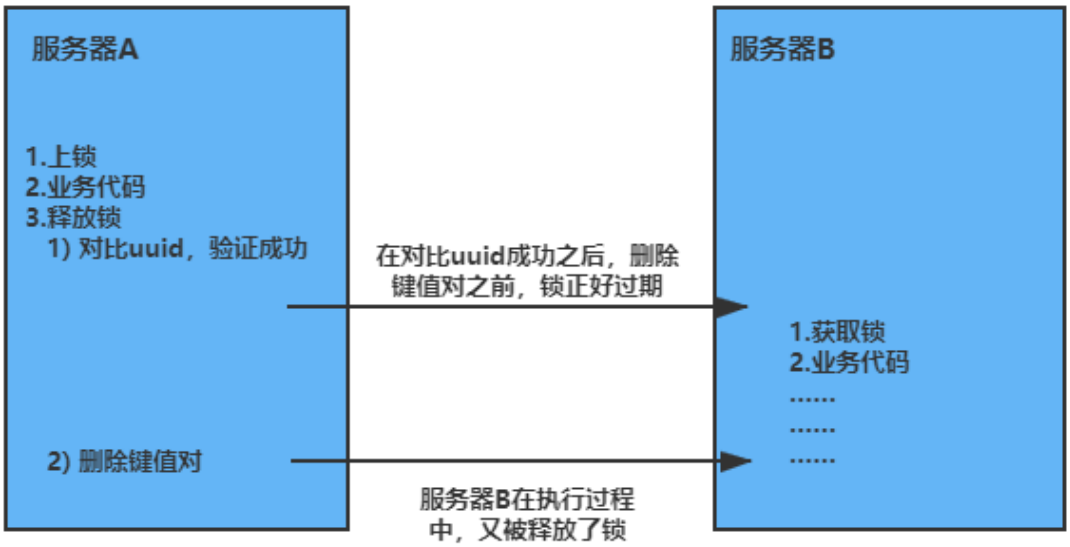


避免误删情况出现，可以在加锁过程中添加一个加锁的唯一id，通过跟该id对比，阻止误删的情况出现。

```
        //连接到自己的redis服务器
        Jedis jedis = new Jedis("192.168.56.31",6379);
        UUID uuid = UUID.randomUUID();
        jedis.setnx("lock",uuid.toString());
        /*
        执行业务代码
```

```
        * */
        //释放锁的时候，通过uuid对比下是不是自己加的锁
        String lockuuid = jedis.get("lock");
        if (uuid.toString().equals(lockuuid))
        {

            //如果是自己加的锁，则释放
            jedis.del("lock");
        }
```

7.3 保证删除原子性



Lua脚本

Lua 是一种轻量小巧的脚本语言，用标准C语言编写并以源代码形式开放， 其设计目的是为了**嵌入应用程序**中，从而为应用程序提供灵活的扩展和定制功能。

Redis中引入lua的优势：

- 减少网络开销：多个请求通过脚本一次发送，减少网络延迟
- **原子操作**：将脚本作为一个整体执行，中间不会插入其他命令，无需使用事务
- 复用：客户端发送的脚本永久存在redis中，其他客户端可以复用脚本
- 可嵌入性：可嵌入JAVA，C#等多种编程语言，支持不同操作系统跨平台交互

lua进行比较uuid，对比成功后删除键值对的代码：

```
if redis.call('get', KEYS[1]) == ARGV[1]
    then
        return redis.call('del', KEYS[1])
    else
        return 0
end
```

if 中的比较如果是true , 那么 执行 del 并返回del结果；如果 if 结果为false 直接返回 0 。

其中的KEYS[1] , ARGV[1] 是参数，我们只调用 jedis 执行脚本的时候，传递这两个参数就可以了。

通过jedis执行lua脚本

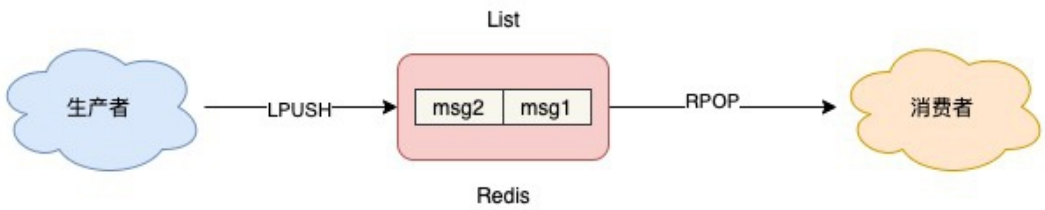
```
//连接到自己的redis服务器
Jedis jedis = new Jedis("192.168.56.31",6379);
UUID uuid = UUID.randomUUID();
jedis.setnx("lock",uuid.toString());
String lockuuid = jedis.get("lock");


String luaScript = "if redis.call('get', KEYS[1]) == ARGV[1] then return
redis.call('del',KEYS[1]) else return 0 end";// lua脚本，用来释放分布式锁
//第一个参数是lua脚本，第二个参数是需要判断的key，第三个参数是key所对应的value
jedis.eval(luaScript,Arrays.asList("lock"),Arrays.asList(lockuuid));
```

通过lua脚本进行比对删除，保证原子性操作，防止出现问题。

8 消息队列

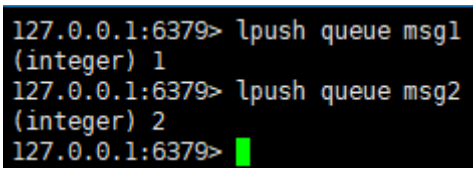
8.1 List消息队列

List 底层的实现就是一个「链表」，在头部和尾部操作元素，时间复杂度都是 O(1)，这意味着它非常符合消息队列的模型。



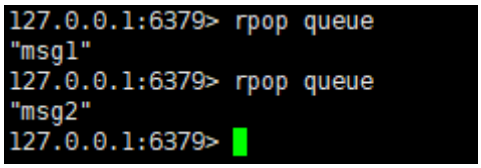
生产者使用 lpush发布消息

```
lpush queue msg1
lpush queue msg2
```



消费者这一侧，使用 rpop拉取消息：

```
rpop queue
rpop queue
```



当队列中已经没有消息了，消费者在执行 RPOP 时，会返回 NULL。

一般编写消费者逻辑时，通过一个“死循环”实现，如果此时队列为空，那消费者依旧会频繁拉取消息，造成资源浪费。

```
while(true)
{
    String msg = jedis.rpop("queue");
}
```

Redis 提供「阻塞式」拉取消息的命令：brpop / blpop，这里的 B 指的是阻塞（Block）。

brpop key timeout：移除并返回最后一个值，同时需要传入一个超时时间（timeout），如果设置为 0，则表示不设置超时，直到有新消息才返回，否则会在指定的超时时间后返回 NULL。

```
brpop queue 0//获取queue最后一个值，如果没有值，则一直等待
```

缺点：

- 不支持重复消费：消费者拉取消息后，这条消息就从 List 中删除了，无法被其它消费者再次消费，即不支持多个消费者消费同一批数据
- 消息丢失：消费者拉取到消息后，如果发生异常宕机，那这条消息就丢失了

消费者（Customer）：

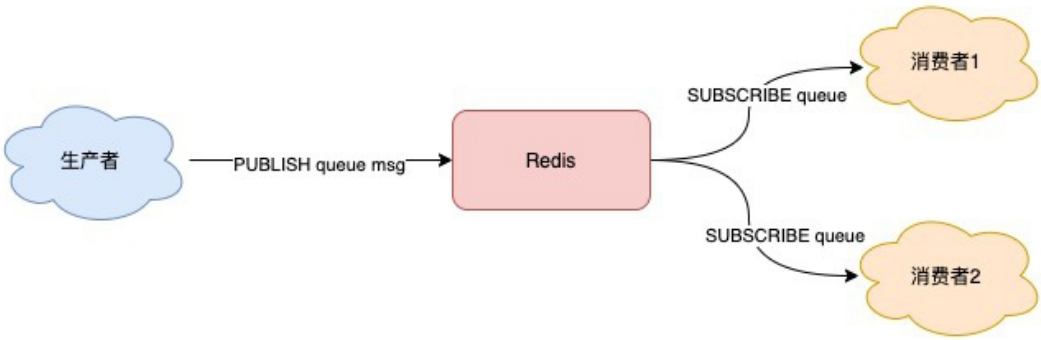
```
Jedis jedis = new Jedis("192.168.56.31",6379);
System.out.println("开始监听");
while (true)
{
    List<String> msg =  jedis.brpop(0,"queue");
    System.out.println("接受消息：");
    //一般来说 一条消息分为两部分，第一部分是list的key，第二部分为value
    for (String m : msg)
    {
        System.out.print(m + " ");
    }
}
```

生产者（Producer）：

```
Jedis jedis = new Jedis("192.168.56.31",6379);
Scanner sc = new Scanner(System.in);
while (true)
{
    System.out.println("输入发送的消息：");
    String msg = sc.next();
    jedis.lpush("queue",msg);
}
```

8.2 发布/订阅消息队列

Redis 提供了 PUBLISH / SUBSCRIBE 命令，来完成发布、订阅的操作。



多个消费者，同时消费同一批数据。

```
//多个客户端同时订阅queue频道
SUBSCRIBE queue
```

通过生产者，发布一条消息。

```
PUBLISH queue msg1
```

客户端接收到消息

```
SUBSCRIBE queue
// 收到新消息
1) "message"
2) "queue"
3) "msg1"
```

使用 Pub/Sub 这种方案，既支持阻塞式拉取消息，还很好地满足了多组消费者，消费同一批数据的业务需求。

但是该方案会引起**消息丢失**：

- 消费者下线
- Redis 宕机

消费者：通过jedis订阅频道，需要一个JedisPubSub子类对象，并重写onMessage方法用于接受消息

```
public class Customer extends JedisPubSub {
    public void onMessage(String channel, String message) {
        System.out.println("接收到消息:" + channel + ":" + message);
    }

    public static void main(String[] args) {
        Jedis jedis = new Jedis("192.168.56.31",6379);
        //通过jedis订阅频道，需要一个JedisPubSub子类对象，并重写onMessage方法用于接受消息
        jedis.subscribe(new Customer(),"queue");
    }
}
```

生产者：

```
Jedis jedis = new Jedis("192.168.56.31",6379); //第一个参数是ip地址，第二个参数是端口
Scanner sc = new Scanner(System.in);
while (true)
{
    System.out.println("输入发送的消息：");
    String msg = sc.next();
    jedis.publish("queue",msg);
}
```

9 数据一致性解决方案

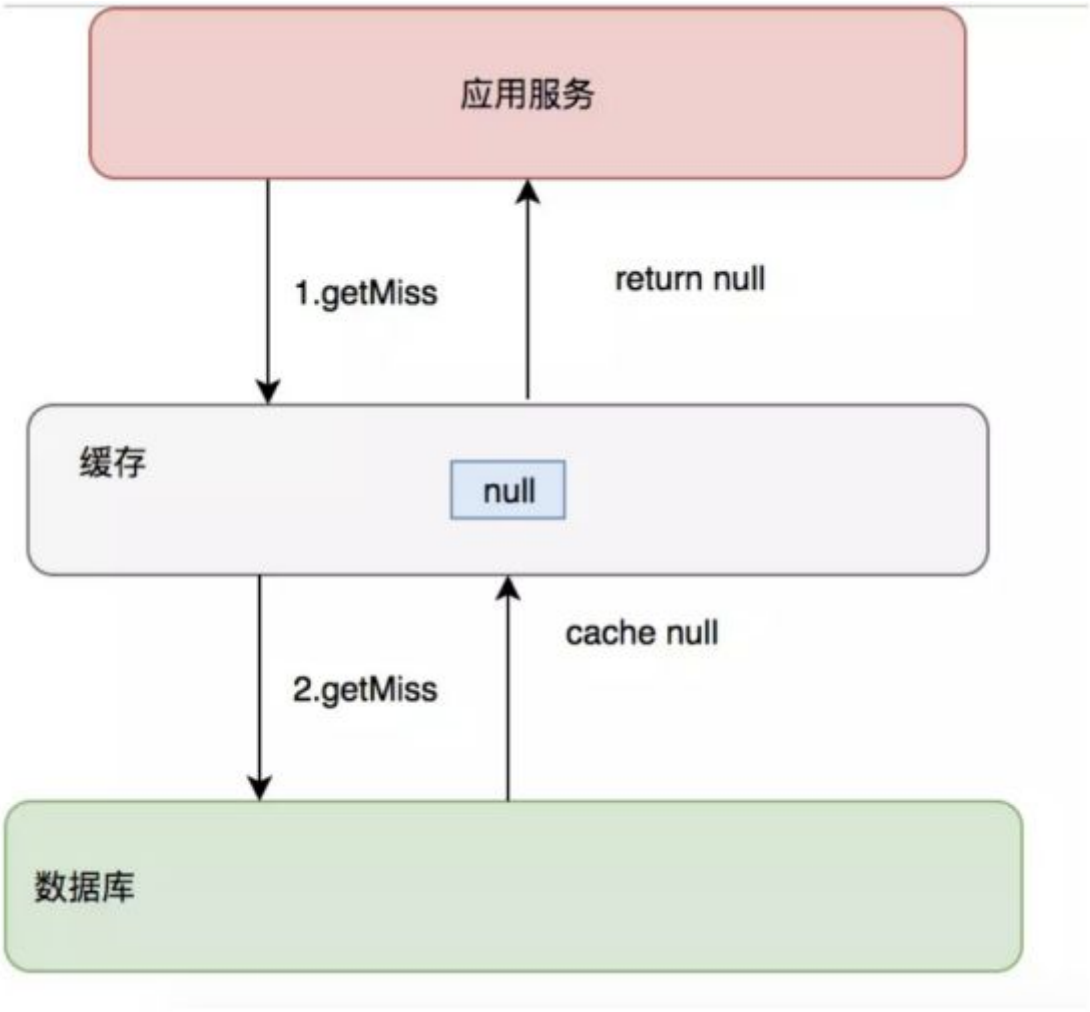
读取缓存步骤一般没有什么问题，但是一旦涉及到数据更新：数据库和缓存更新，就容易出现**缓存(Redis)和数据库（MySQL）间的数据一致性问题。**

不管是先写MySQL数据库，再删除Redis缓存；还是先删除缓存，再写库，都有可能出现数据不一致的情况。

例：

1. 如果删除了缓存Redis，还没有来得及写库MySQL，另一个线程就来读取，发现缓存为空，则去数据库中读取数据写入缓存，此时缓存中为脏数据。
2. 如果先写了库，在删除缓存前，写库的线程宕机了，没有删除掉缓存，则也会出现数据不一致情况。

因为写和读是并发的，没法保证顺序,就会出现缓存和数据库的数据不一致的问题。



9.1 延时双删策略

1. 先删除缓存。
2. 再写数据库。
3. 休眠500毫秒；
4. 再次删除缓存。

需要评估自己的项目的读数据业务逻辑的耗时。这么做的目的，就是确保读请求结束，写请求可以删除读请求造成的缓存脏数据。

当然这种策略还要考虑redis和数据库主从同步的耗时。最后的的写数据的休眠时间：则在读数据业务逻辑的耗时基础上，加几百ms即可。比如：休眠1秒。

缺点：结合双删策略+缓存超时设置，这样最差的情况就是在超时时间内数据存在不一致，而且又增加了写请求的耗时。

10 企业级持久化解决方案

在企业中不要仅仅使用RDB，因为那样会导致丢失很多数据。

也不要仅仅使用AOF，因为那样有两个问题：

1. 通过AOF做冷备，没有RDB做冷备，来的恢复速度更快；
2. RDB每次简单粗暴生成数据快照，更加健壮，可以避免AOF这种复杂的备份和恢复机制的bug。

综合使用AOF和RDB两种持久化机制，用AOF来保证数据不丢失，作为数据恢复的第一选择；

用RDB来做不同程度的冷备，在AOF文件都丢失或损坏不可用的时候，还可以使用RDB来进行快速的数据恢复。

如果RDB在执行snapshotting操作，那么redis不会执行AOF rewrite；如果redis再执行AOF rewrite，那么就不会执行RDB snapshotting。

如果RDB在执行snapshotting，此时用户执行BGREWRITEAOF命令，那么等RDB快照生成之后，才会去执行AOF rewrite。

10.1 RDB的生成策略

如果希望能确保RDB最多丢1分钟的数据，那么尽量就是每隔1分钟都生成一个快照。不过到底是10000条执行一次RDB，还是1000条执行一次RDB，这个根据需要根据自己的应用和业务的数据量来确定。

10.2 AOF的生成策略

AOF一定要打开，fsync方式选择everysec。一般可能会调整的参数可能就是下面俩参数了

```
auto-aof-rewrite-percentage 100
```

就是当前AOF大小膨胀到超过上次100%，上次的两倍。

```
auto-aof-rewrite-min-size 64mb
```

根据自己的数据量来定，16mb，32mb。

10.3 企业级的数据备份方案

RDB非常适合做冷备，每次生成之后，就不会再有修改。

数据备份方案：

1. 写定时调度脚本去做数据备份。
2. 每小时都copy一份rdb的备份，到一个目录中去，仅仅保留最近48小时的备份。
3. 每天都保留一份当日的rdb的备份，到一个目录中去，仅仅保留最近1个月的备份。
4. 每次copy备份的时候，都把太旧的备份给删了。
5. 每天晚上将当前服务器上所有的数据备份，发送一份到远程的云服务上去。